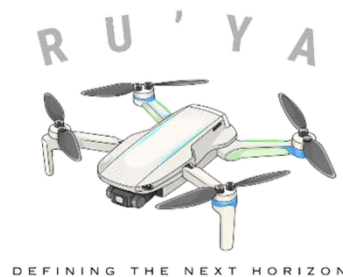


Kingdom of Saudi Arabia
Ministry of Higher Education
Imam Abdulrahman Bin Faisal University
College of Computer Sciences & Information Technology



RU'YA: A LIGHTWEIGHT AI MODEL FOR UAV-BASED CROWD DETECTION AND MONITORING

Final report

#	Name	ID	Role
1	Nora Abdullah Aljomuh	2220004452	Leader
2	Mozoon Alkhalis	2220002873	Member
3	Ritaj Alhamli	2210002809	Member
4	Joud Alahmari	2220001872	Member
5	Sarah Alashgar	2220005589	Member

Supervised by: Dr. Rabab Alkhalifa

ACKNOWLEDGEMENT

The project team would like to express sincere gratitude to Dr. Rabab Al Khalifa for her continuous guidance, valuable feedback, and encouragement throughout the development of this project. Her expertise and support were instrumental in shaping the project's direction and ensuring its successful progress.

We would also like to extend our heartfelt thanks to Mrs. Mehwash Farooqui, our co-supervisor, for her insightful suggestions, technical guidance, and consistent motivation that greatly contributed to improving the quality of this work.

We also extend our appreciation to the Department of Computer Science for providing the necessary resources, technical support, and a collaborative environment that facilitated our research and implementation.

Finally, we would like to thank our families and peers for their patience, motivation, and support during the completion of this work.

LIST OF ABBREVIATIONS

Table 1. List of Abbreviations

ABBREVIATION	FULL FORM
AI	Artificial Intelligence
UAV	Unmanned Aerial Vehicle
CNN	Convolutional Neural Network
FCN	Fully Convolutional Network
GAN	Generative Adversarial Network
RL	Reinforcement Learning
IOT	Internet of Things
GPU	Graphics Processing Unit
CPU	Central Processing Unit
MAP	Mean Average Precision
MAE	Mean Absolute Error
RMSE	Root Mean Square Error
MOTA	Multiple Object Tracking Accuracy
LOS	Line of Sight
RGB	Red, Green, Blue (color image channels)
TIR	Thermal Infrared
API	Application Programming Interface
PID	Proportional–Integral–Derivative (controller)
CV	Cross-Validation
FPS	Frames Per Second
TOPS	Trillion Operations Per Second
YOLO	You Only Look Once (object detection algorithm)
SVM	Support Vector Machine
RNN	Recurrent Neural Network
FCNN	Fully Convolutional Neural Network

SRS	Software Requirements Specification
SDS	Software Design Specification
STP	Software Testing Plan
SPMP	Software Project Management Plan
WBS	Work Breakdown Structure
SDAIA	Saudi Data and Artificial Intelligence Authority
GACA	General Authority of Civil Aviation
IAU	Imam Abdulrahman Bin Faisal University
ROS	Robot Operating System
RL	Reinforcement Learning
TPU	Tensor Processing Unit
TFW	Thermal Faces in the Wild (dataset)
PS-YOLO	Partial Convolution–Based YOLO
LCDNET	Lightweight Crowd Density Network
AIRSIM	Aerial Informatics and Robotics Simulation
GYM-PYBULLET-DRONES	Lightweight open-source quadcopter simulator
UAVDT	Unmanned Aerial Vehicle Detection and Tracking dataset
VISDRONE	Vision-based Drone Dataset
RM	Risk Management
SDLC	Software Development Life Cycle

Table of Contents

Chapter 1: Introduction	16
1.1 Introduction	16
1.2 Problem Statement	17
1.3 Motivation	18
1.4 Justification	18
1.5 Aims & Objectives	19
1.6 Scope / Limitation of the Study	20
1.7 Social, Professional, Legal, and Ethical Implications	20
1.8 Project Organization	21
Chapter 2: Background and Review of Literature	22
.....	24
2.1 AI Models for UAV Crowd Detection:	26
2.1.1 Heavyweight AI Models	26
2.1.2 Lightweight AI Models	29
2.1.3 Model Compression Techniques	32
2.2 Comparative Analysis of Datasets for Crowd Monitoring	34
2.3 Simulation as a Testbed for UAV Crowd Detection Models	37
2.4 Deployment of Embedded Hardware Platform for UAVs	41
Chapter 3: Software Project Management Plans	50
1. Project Overview	50
1.1 Purpose, Scope, and Objectives	50
1.2 Assumptions, Constraints and Risks	51
1.2.1 Assumptions	51
1.2.2 Constraints	51
1.2.3 Risks	52
1.3 Project Deliverables	53
1.4 Schedule and Budget Summary	54
1.5 Evolution of the Plan	54
1.6 Definitions and Acronyms	55
1.7 Document Structure	55

1.8 Business Criteria and Success Measures	56
2. Project Organization	57
2.1 External Interfaces.....	57
1.2 Internal Structure	57
2.3 Roles and Responsibilities	58
2.4 Stakeholder Identification	60
3. Managerial Process Plans	61
3.1 Start-up Plan	61
3.1.1 Estimates.....	61
3.1.2 Staffing	62
3.1.3 Project Staff Training	63
3.2 Work Plan	64
3.2.1 Work Breakdown Structure	64
3.2.2 Schedule Allocation	67
3.2.4 Budget Allocation	71
3.3 Project Tracking Plan	72
3.3.1 Requirements Management	72
3.3.2 Schedule Control.....	72
3.3.4 Quality Control.....	72
3.3.5 Reporting	72
3.3.6 Project Metrics	73
3.4 Risk Management Plan.....	73
3.5 Project Closeout Plan	75
4. Technical Process Plans.....	76
4.1 Process Model	76
4.2 Methods, Tools, and Techniques	77
4.3 Infrastructure	77
4.4 Product Acceptance	78
5. Supporting Process Plans	79
5.1 Documentation.....	79
6. Additional Plans	79

Chapter 4: Software Requirements Specification.....	80
1. Introduction.....	80
1.1 Purpose.....	80
1.2 Scope	80
1.3 Definitions, Acronyms, and Abbreviations	81
2. Overall description	82
2.1 Product perspective	82
2. Product functions.....	83
3. Specific requirements	83
3.1 External interface requirements.....	83
3.1.1 User interfaces	83
3.1.2 Hardware Interfaces	90
3.1.3 Software Interfaces.....	91
3.1.3.1 UAV Firmware and Control Software	91
3.1.3.2 AI Crowd Detection Module	91
3.1.3.3 Web Dashboard Software.....	91
3.1.3.4 Operating Environment	91
3.1.4 Communication Interfaces	92
3.1.4.1 UAV–Dashboard Link.....	92
3.1.4.2 Cloud and Database Synchronization.....	92
4. System Features.....	92
4.1 Flight Control.....	92
4.2 Aerial Data Acquisition	93
4.3 AI-Based Crowd Detection	93
4.4 Data Transmission	94
4.5 Dashboard Analytics	94
4.6 Data Storage.....	95
5.Non-Functional Requirements.....	95
Chapter 5: Software Design Specifications	97
1. Introduction.....	97
1.1 Purpose.....	97

1.2 Scope	97
1.3 Definitions, Acronyms, and Abbreviations.....	97
2. System overview	99
2.1 General description.....	99
2.2 System Functionality.....	99
2.3 Design Approach.....	100
3. Design Considerations	101
3.1 Assumptions and Dependencies.....	101
3.2 General Constraints.....	102
4. User Interface Design.....	103
4.1 Overview of User Interface	103
4.1.1 Administrator Interface	103
4.1.2 Operator Interface.....	103
4.2 Interface Design Rules	104
4.3 Screen Images	105
4.3.1 Login Page.....	105
4.3.2 Main Dashboard (Home Page)	106
4.3.3 Analytics Panel	107
4.3.4 Alert management panel.....	108
4.3.5 Flight control center	109
4.3.6 System settings page.....	110
4.3.7 Logs page	111
4.2 Screen Objects and Actions.....	112
5. System Architecture.....	114
5.1 Architectural Design Approach	114
5.2 Architectural Design	115
5.3 Subsystem Architecture.....	116
1. The Hardware Subsystem	116
2. The Processing Subsystem.....	117
3. The Visualization Subsystem	117
6. Data Design.....	119

6.1 Data Description	119
6.2 Data Dictionary	119
6.3 Database Description.....	122
7. Component Design	124
8. Detailed System Design	126
8.1 Classification, Definition, Responsibilities.....	126
8.2 Constraints.....	127
8.3 Composition	127
8.4 Uses/Interactions	128
8.5 Resources	132
8.6 Processing	133
8.7 Interface/Exports	136
9. Other Design Features	139
9 Requirements Traceability Matrix.....	140
Chapter 6: Implementation Phase	142
1. Introduction.....	142
2. Summary of the proposal	143
3. Changes from the proposal phase and justifications	144
4. Implementation Progress.....	145
4.1 System Overview	145
4.2 Dataset and Preprocessing	145
4.2.1 Dataset Selection	145
4.2.2 Data Preprocessing.....	146
4.3 Person Detection Model.....	148
4.3.1 Model Selection.....	148
4.3.2 Model Training	148
4.3.3 Model Evaluation.....	150
4.3.4 Inference Performance	151
4.3.5 Qualitative Failure Case Analysis	151
4.4 Crowd Density Estimation	154
4.4.1 Grid-Based Partitioning	154

4.4.2 Local Person Counting	154
4.4.3 Density Computation.....	154
4.4.4 Interpretation of Density Values	155
4.5 Risk Classification.....	155
4.5.1 Risk Classification Criteria	155
4.5.2 Risk Mapping and Visualization.....	155
4.5.3 Interpretation of Results.....	156
4.6 Hardware Setup and Integration	156
4.6.1 Hardware Assembly	156
1. Hardware Construction	157
1.1. Mechanical Assembly and Power Integration.....	157
1.2. Flight Controller and Electronics Integration	158
1.3. Safety and Final Components	159
2. Mission Planner Configuration.....	160
2.1. Mission Planner Setup and Connection	160
2.2. Initial UAV Setup.....	160
2.3. Sensor and Radio Calibration.....	161
2.3.1. Accelerometer Calibration.....	161
2.3.2. Compass Calibration	162
2.3.3. Radio Calibration	163
2.4. Flight Mode Setup and Motor Testing	163
2.4.1. Mode setup.....	163
2.4.2. Motor Testing	164
5. Issues Faced	166
5.1 Dataset Challenges	166
5.2 Model Training and Performance.....	166
5.3 Implementation and Integration.....	166
5.4 Real-Time Processing Constraints	166
5.5 Hardware Challenges	167
Chapter 7: Testing & Evaluation.....	168
7.1 Test Items	168

7.2 Features to be Tested	168
7.3 Approach.....	168
7.3.1 Data and Dataset Integrity Testing	168
7.3.2 Component Testing	169
7.3.3 Integration Testing	171
7.3.4 Interface Testing	172
7.3.5 Validation and Verification (V&V) Testing	172
7.3.6 Performance Testing.....	173
7.3.7 Constraints	173
7.3.8 Acceptance Testing	173
7.4 Testing Environment	174
7.5 Testing Hardware.....	174
7.5.1 ESC and Motor Testing	174
7.5.2 Power Distribution Testing.....	175
7.5.3 Pixhawk Integration Testing	175
Completion Criteria	175
7.5.4 Camera and Video Transmission Testing.....	175
Completion Criteria	176
7.6 Altitude & Weather Simulation Testing.....	177
8. Conclusion.....	185
Appendix 1: Response to evaluators' comments	190
Term 1 comments.....	190
Term 2 comments.....	192
Appendix 2: Biweekly Report: 1st semester	194
Appendix 3: Paper Submission to conference	205
Appendix 4: Patent Submission to Imam Abdulrahman Bin Faisal University	206
Appendix 5: Biweekly Report: 2nd semester	207

Table of Tables

Table 1. List of Abbreviations.....	2
Table 2. Organization of the Project Report	21
Table 3. Summary of recent UAV-based crowd analysis and detection models	30

Table 4. Coverage of AI Approaches in UAV-Based Crowd Detection	31
Table 5. Model Compression for UAV Deployment	32
Table 6. Comparative summary of real-world and synthetic UAV crowd monitoring datasets (from [19], [21], [29]).	36
Table 7. Comparative Evaluation of UAV Simulators	39
Table 8. AI Hardware for UAV Crowd Detection	42
Table 9. Summary of Literature under Theme 0 – Background and Problem Statement	45
Table 10. Summary of Literature under Theme 1 – AI Models for UAV Crowd Detection...	46
Table 11. Summary of Literature under Theme 2 – Comparative Analysis of Datasets for Crowd Monitoring	47
Table 12. Summary of Literature under Theme 3 – Simulation as a Testbed for UAV Crowd Detection Models	48
Table 13. Summary of Literature under Theme 4 – Deployment of Embedded Hardware Platforms for UAVs	48
Table 14. Project Risks	52
Table 15. Project Deliverables	53
Table 16. The project Schedule	54
Table 17. Terms and Definitions	55
Table 18. Roles and Responsibilities	59
Table 19. Stakeholder Identification	60
Table 20. Human Resources and Duration	62
Table 21. Work Breakdown Structure (WBS) for the Ru'ya Drone Project	64
Table 22. Project Schedule and Activity Timeline	67
Table 23. Hardware, Software, and Written Resources for the Ru'ya Project	71
Table 24. Budget Allocation table	71
Table 25. Project Metrics	73
Table 26. Risk Management Plan	74
Table 27. The Waterfall Phases	76
Table 28. Methods, Tools, and Techniques Used in the Ru'ya Project	77
Table 29. Infrastructure Components of the Ru'ya Project	78
Table 30. Project Documentation Plan for the Ru'ya Project	79
Table 31. Key Terms and Abbreviations Used in the Ru'ya System	81
Table 32. User Roles	90
Table 33. Definitions, Acronyms, and Abbreviations	98
Table 34. Screen Objects and Actions	112
Table 35. Major Identified Roles Of The System.	114
Table 36. Information Domain Process	119
Table 37. Data Dictionary	120
Table 38. Component, Classification, Definition and Responsibility	126
Table 39. Components Constrains	127

Table 40:Components Composition	127
Table 41. Summary of hardware, software, and data resources utilized by the Ru'ya Drone System.....	132
Table 42. Concurrency and Dependency Management Strategies in the Ru'ya Drone System	133
Table 43. Description of “Login” Component	136
Table 44. Description of “Dashboard” Component.....	136
Table 45. Description of “Analytics” Component	136
Table 46. Description of “Alerts” Component.....	137
Table 47. Description of “Flight Control” Component	137
Table 48. Description of “Settings” Component	137
Table 49. Description of “Logs” Component	138
Table 50. Description of “Admin Control” Component	138
Table 51. Traceability Matrix	140
Table 52: Training Configuration	149
Table 53: Observed Failure Patterns	151
Table 54. Data and Dataset Integrity Test Cases	169
Table 55 Component Test Case for YOLOv8 Detection Model	170
Table 56 Component Test Case for Density and Risk Classification Module	171
Table 57.Comparison of Crowd Detection Accuracy Across Different UAV Altitudes	179
Table 58. Weather-Based Evaluation of the RU'YA Crowd Detection System	181
Table 59: Evaluators comments term1	190
Table 60: Evaluators comments term2.....	192

Table of Figure

Figure 1. Distribution of fatalities per accident across five-year intervals (1975–2019) (adapted from [5])	17
Figure 2. Structure of literature review.....	22
Figure 3. weather impacts: dense fog, rain, and strong light degrade UAV perception (Reproduced from [11]).....	24
Figure 4. Visual (top) and thermal (bottom) images under different lighting conditions: (a) normal illumination, (b) low illumination, and (c) complete darkness. (Reproduced from [13])	24
Figure 5. Summary of UAV crowd monitoring challenges and proposed solutions.	25
Figure 6. Initial images on the left and the corresponding detected crowd. (Reproduced from [16]).	27
Figure 7. Example video frame of a video demonstrating the proposed crowd detection method results on a previously unseen UAV-captured video. URL: http://bit.ly/crowd_det_results . (reproduced from [17])	28
Figure 8. Example samples from DroneCrowd (rows 1–2, real UAV images) and AirSimCrowd (rows 3–4, synthetic UAV images), (from [21])	35

Figure 9. Crowd detection results on DroneCrowd (rows 1–2, real UAV images) and AirSimCrowd (rows 3–4, synthetic UAV images), (From [21]).	35
Figure 10. A snapshot from AirSim shows an aerial vehicle flying in an urban environment. The inset shows depth, object segmentation, and front camera streams generated in real time (reproduced from [32]).	37
Figure 11. multi-drone simulation in gym-pybullet-drones (reproduced from [33]).	38
Figure 12. Context Diagram of the Ru'ya Project External Interfaces	57
Figure 13. Internal structure	58
Figure 14. Software Resource Estimates	61
Figure 15. Training Framework	63
Figure 16. Ru'ya Project Gantt Chart (2025–2026)	70
Figure 17. The Waterfall Model	76
Figure 18. The Ru'ya System	80
Figure 19. High-Level Architecture of the Ru'ya System	82
Figure 20. Dashboard Interface	84
Figure 21. Analytics Panel	85
Figure 22. Alert Management Panel	86
Figure 23. Flight Control Center	87
Figure 24. System Settings Page	88
Figure 25. Logs Page	89
Figure 26. Login Page	105
Figure 27. Main Dashboard (Home Page)	106
Figure 28. Analytics Panel	107
Figure 29. Alert management panel	108
Figure 30. Flight control center	109
Figure 31. System settings page	110
Figure 32. Logs page	111
Figure 33. The Architectural Design of Ru'ya	115
Figure 34. Top-Level Data Flow Diagram (DFD Level 0)	116
Figure 35. The Hardware Subsystem (DFD Level 1)	116
Figure 36. The Processing Subsystem (DFD Level 1)	117
Figure 37. The Visualization Subsystem (DFD Level 1)	117
Figure 38. Structural Decomposition of the Ru'ya System	118
Figure 39. Conceptual Entity Relationship Diagram	122
Figure 40. Relational Entity Relationship Diagram	123
Figure 41. Login component Pseudocode	124
Figure 42. Dashboard Component Pseudocode	124
Figure 43. Analytics Component Pseudocode	124
Figure 44. Alerts Component Pseudocode	125
Figure 45. Flight Control Component Pseudocode	125

Figure 46. Settings Component Pseudocode	125
Figure 47. The “Login” sequence diagram.....	129
Figure 48. The “Alert & Detection” sequence diagram	129
Figure 49. The “Analytics” sequence diagram.....	130
Figure 50. The "Flight Control" sequence diagram.....	130
Figure 51. The "Setting and Admin" sequence diagram	131
Figure 52. The "System Log" sequence diagram.....	131
Figure 53. Example of a raw drone-captured crowd image illustrating the initial visual input used by the Ru’ya Drone System for real-time monitoring and analysis.	133
Figure 54. Preprocessing Pipeline for the Ru’ya Drone-Based Crowd Monitoring System	134
Figure 55. YOLO-based detection output	134
Figure 56. Example of automatic face-blurring for privacy protection in crowd monitoring systems Reproduced from [50]	139
Figure 57. Comparison between CCTV (left) and UAV drone (right) views for crowd monitoring reproduced from [51]	142
Figure 58: Overall system pipeline for UAV-based crowd detection and risk assessment ..	145
Figure 59: Example of class filtering applied to the VisDrone dataset.	147
Figure 60. Sample images from the VisDrone2019 dataset showing small-scale human instances in aerial scenes	148
Figure 61 Validation loss	149
Figure 62 Training loss	149
Figure 63: Performance metric curves	150
Figure 64: Training loss curves.	
Figure 65: Mean Average Precision (mAP) curves.	150
Figure 66: Detection results of the YOLOv8s model showing identified individuals in aerial imagery with bounding boxes and confidence scores	151
Figure 67: Dense aerial imagery creates many overlapping detections. In such cases, very small or partially occluded people may be missed, which is consistent with the model's moderate recall value.	152
Figure 68: Detection examples across altitude levels show the trade-off between scene coverage and object detail. At high altitude, people occupy fewer pixels, making missed detections more likely.	152
Figure 69: Low-light conditions reduce visual contrast and lead to weaker confidence scores or missed detections, especially for small people in crowded regions.	152
Figure 70: Weather-based evaluation confirms that clear scenes achieve the highest detection rate, while dusty and low-light conditions reduce the average detection rate.	153
Figure 71: Grid-based partitioning of the input frame into 6×6 cells for localized density estimation	154
Figure 72: Crowd risk visualization showing grid-based partitioning with low-risk regions highlighted across the frame.....	156

Figure 73.Final Ru'ya UAV system overview.....	157
Figure 74. ESCs Soldered to the Power Distribution Board (PDB)	157
Figure 75. Bullet connectors soldered to motor wires for connection with the ESC.	158
Figure 76. GPS and Pixhawk aligned facing forward.....	158
Figure 77. Ru'ya UAV powered on showing active system status	159
Figure 78. Mission Planner showing UAV connection and GPS location.	160
Figure 79. Frame type selection in Mission Planner.	160
Figure 80. UAV orientation during accelerometer calibration.	161
Figure 81. Accelerometer calibration interface in Mission Planner	161
Figure 82. Compass calibration interface in Mission Planner.	162
Figure 83. UAV movement directions and telemetry interface.	162
Figure 84. Radio calibration signals in Mission Planner.....	163
Figure 85. Flight mode configuration in Mission Planner.....	163
Figure 86. Quadrotor motor rotation directions (CW and CCW)	164
Figure 87. Motor testing interface in Mission Planner	164
Figure 88. UAV connection and input signals in Mission Planner.....	165
Figure 89.Motor direction diagram (CW/CCW layout).....	174
Figure 90. Example of the generated synthetic aerial crowd used in the simulation.....	177
Figure 91. Conceptual illustration of drone altitude effect on camera field of view. Low altitude provides clearer object details but limited coverage, high altitude increases coverage but reduces object detail, while medium altitude provides the best	178
Figure 92. Detection Performance of the RU'YA System Across Different UAV Flight Altitudes	179
Figure 93. Detection results at different altitudes. Medium altitude provides the best balance between coverage and detection accuracy.	180
Figure 94. Crowd Detection Output Under Clear Weather Conditions Using the RU'YA System.....	182
Figure 95. Crowd Detection Output Under Rainy Weather Conditions Using the RU'YA System.....	182
Figure 96. Crowd Detection Output Under Foggy Weather Conditions Using the RU'YA System.....	183
Figure 97. Crowd Detection Output Under low light Weather Conditions Using the RU'YA System.....	183
Figure 98.Crowd Detection Output Under Dusty Weather Conditions Using the RU'YA System.....	183
Figure 99. Weather-Based Evaluation of YOLOv8s Crowd Detection Performance	184
Figure 100.ICIGP 2026 Conference.....	205
Figure 101.Patent Submission Acknowledgment Email for the UAV Compliance Enforcement Device	206

Chapter 1: Introduction

This chapter provides a concise introduction to the project, outlining its key aspects and presenting an overview of the problem's scope and constraints. It also discusses the underlying causes and justifications for undertaking the project, followed by the specific goals and objectives that are intended to be achieved. Finally, the chapter concludes with an explanation of how the overall project is organized and structured.

1.1 Introduction

Saudi Arabia is thriving and gleaming, rising to new heights under the transformative framework of Saudi Vision 2030. The main goal of the Kingdom's vision is to build a vibrant society, a flourishing economy, and an ambitious nation [1]. As part of this journey, the Kingdom is preparing to welcome the world through remarkable initiatives, from hosting the FIFA World Cup 2034 to annually managing the massive operations of the Hajj season, and even more [2] [3]. To achieve these milestones effectively and maintain people's safety, the demand for new technological advancements for large-scale projects has never been greater.

Huge events often need managing massive crowds in dynamic and complex environments. Current crowd monitoring methods rely on security guards and fixed CCTV surveillance systems, which can be limited by visibility, slow response times, and human error. These challenges increase safety risks and reduce emergency responses, highlighting and focusing on the need for more advanced and reliable monitoring systems [4].

In response to this need, our project **Ru'ya**, meaning "Vision" in Arabic introduces a multi-environment drone system for crowd management and control. Our project foresight is to build a lightweight autonomous aerial system model capable of operating across different environments to monitor crowds, enhance safety, and deliver real-time data. By combining unmanned aerial vehicle (UAV) technology with artificial intelligence, this system will align with our Kingdom's vision, especially as it will enhance the user experience through rapid real-time response and wide aerial visibility.

Our project intention and expectation are to support authorities such as the Saudi Data and Artificial Intelligence Authority (SDAIA) and event organizers in improving safety measures, optimizing resources, and ensuring smoother large-scale event operations, with the aspiration of being part of the Kingdom's journey.

1.2 Problem Statement

People have been gathering at major events for centuries; however, in recent years, crowd management and monitoring are needed more than ever due to numerous factors, including people moving to urban areas, globalization, religious events such as the Hajj, and, most prominently, worldwide events including the FIFA World Cup. An article by Feliciani et al. [5] examines the severity of crowd incidents to raise awareness on this matter. The authors took the initiative to collect and analyze data covering more than a century.

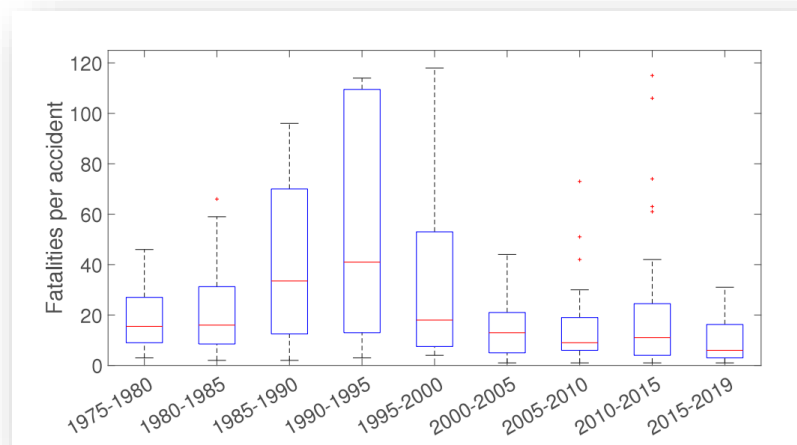


Figure 1. Distribution of fatalities per accident across five-year intervals (1975–2019) (adapted from [5])

Figure 1 shows a boxplot illustrating how fatalities can vary drastically across time periods, mirroring the unpredictable and high-risk nature of dense crowd gatherings. Furthermore, the article also discusses the purpose of those gatherings and the type of events. It discovers a relation clearly demonstrating that religious and sports events have the highest count of crowd accidents [5].

The Kingdom of Saudi Arabia is preparing to host the FIFA World Cup 2034, which is proven to attract a major crowd consisting of millions of fans from all around the world, and such risks must be addressed and mitigated. Today's CCTV system has blind spots, and it cannot track large, moving crowds. Also, existing UAV-based detection solutions, while promising, typically require high GPU resources and are still struggling to provide reliable real-time performance in outdoor conditions. These challenges highlight the need for an improved and more adaptive monitoring approach. Our AI-driven UAV solution is designed to avoid and prevent such accidents by detecting and forecasting crowd densities in real time, providing authorities with early warnings to tackle such situations.

Traditionally, UAVs are operated using non-AI techniques such as manual remote control, GPS-based waypoint navigation, and classical control algorithms like proportional–integral derivative (PID) controllers. Visual data from onboard cameras is typically streamed to a

ground control station for human monitoring or processed using simple rule-based methods. While effective for basic navigation and surveillance, these approaches cannot autonomously interpret complex scenes or adapt in real time, motivating the integration of AI-based perception models for onboard crowd analysis.

1.3 Motivation

1. Reducing the potential safety risks and operational challenges associated with managing massive crowds during large events in Saudi Arabia, such as the FIFA World Cup 2034.
2. Incorporating the latest technological innovations into the objectives of the Smart Infrastructure and Smart City initiatives, aligned with Saudi Vision 2030.
3. Assisting authorities and event organizers in monitoring crowds by providing real-time aerial data and intelligent analysis.
4. Utilizing drone-based systems and AI-driven crowd monitoring to support the kingdom's economic vision through optimizing resources and minimizing operational costs.
5. Expanding our academic knowledge by learning and studying artificial intelligence from different aspects, including computer vision, data analysis, and autonomous systems, and applying this knowledge to real-world challenges.

Based on these motivations, the justification explains why this direction is suitable

1.4 Justification

Public safety at mass gatherings is growing to be a global concern. Existing drone-based solutions are heavy and require high-end resources, and some of the existing drones are humanly controlled without implementing AI methods. Currently, it has become clear that new solutions are needed to manage large crowds in an efficient and responsible way. Saudi Arabia's Vision 2030 focuses on advancing digital technologies and enhancing public safety. With the Kingdom preparing to host an international event. **Our project is justified because it explores lightweight AI-driven drones not only as a technological solution to improve monitoring and quick response, but it also introduces a smarter and more sustainable approach as a way to support the country's strategic goals.** Through this project, our academic knowledge will be strengthened by hands-on practical experiences in utilizing UAVs and facing the real-world challenges. Moreover, we aim to witness Saudi Arabia soar as a global leader in innovation and safety. The following objectives translate this rationale into specific measurable goals.

1.5 Aims & Objectives

The main aim of this project is to develop a lightweight AI model that can be deployed on a UAV for real-time crowd detection and analysis. The system is intended to process live footage directly on the drone to enhance monitoring performance and reduce dependence on external computing power.

Objectives:

1. **Collect and analyze datasets:** Identify and prepare UAV-based crowd datasets to train, validate, and benchmark the AI model.
 - ✓ *Expected outcome:* The datasets should include different lighting conditions, crowd sizes, and environments so our model performs well in all situations.
2. **Develop a lightweight AI model:** Design and train a small, efficient deep learning model that works smoothly on limited drone hardware without losing much accuracy.

Work planned: Use model compression techniques to make it faster and lighter.

 - ✓ *Expected outcome:* The model should run in real time with accuracy above 85%.
3. **Test the model in simulation:** Use UAV simulators to safely test model performance before real-world deployment.
 - ✓ *Expected outcome:* The model should process frames quickly (under 200 ms) and detect crowds accurately in different conditions.
4. **Integrate the model into the UAV system:** After simulation, we will connect the trained model to the UAV system so it can detect crowds live and adjust its movement automatically.
 - ✓ *Expected outcome:* The UAV should handle detection on its own without needing external hardware like a GPU.
5. **Deploy and test in a controlled environment:** Fly and test the UAV in a controlled environment at Imam Abdulrahman bin Faisal University while following GACA safety rules.
 - ✓ *Expected outcome:* The UAV should detect crowds correctly and fly steadily during tests.
6. **Evaluate performance:** Measure how accurate, fast, and efficient the system is by checking precision, recall, and frames per second (FPS).
 - ✓ *Expected outcome:* The model's results in the real test should match or be close to the ones from simulation.
7. **Ensure ethical and data compliance:** Follow privacy and data protection standards during model training and testing to prevent data misuse.

- ✓ *Expected outcome:* All data is used safely, and no private information is exposed.

Demonstrate system impact: In the final stage, we will present how our system can actually help improve safety and management during crowded events.

- ✓ *Expected outcome:* The drone should show faster response and better situational awareness in real test cases.

The project is expected to produce the following key deliverables by the end of the development phase:

- A working **lightweight AI model** for crowd detection
- A **UAV prototype** with the model fully integrated and tested.
- A public **GitHub repository** with our code, datasets, and documentation.
- A **final evaluation report** showing how well the system performed.

To maintain focus, the scope defines the boundaries and testing environment of the project.

1.6 Scope / Limitation of the Study

The scope of the project is to develop a Lightweight AI-powered UAV system for real-time crowd detection and density analysis to enhance safety and situational awareness. The system will be deployed and put to test in Imam Abdulrahman Bin Faisal University (IAU) under supervised circumstances to guarantee safety and adherence to the rules of GACA. As the availability of real-world crowd data on a large scale is a limited case, model training and testing are conducted with simulated data and environments. Additionally, in case outdoor trials cannot be carried out in the ongoing phase, the project will be developed with full simulation-based testing to confirm the performance and guarantee the readiness to be implemented on a large scale in the future. Moreover, Because the system involves people and data, ethical and legal factors which will be addressed in the following section.

1.7 Social, Professional, Legal, and Ethical Implications

The project was developed to enhance public safety and support authorities in managing large crowds during the FIFA World Cup 2034. Integration of (UAVs) with artificial intelligence (AI)-driven crowd monitoring provides real-time insights that mitigate overcrowding risks, improve emergency response, and optimize resource allocation. This initiative supports Saudi Vision 2030 by advancing smart city technologies, strengthening national safety infrastructure, and increasing the Kingdom's capacity to host global events securely and efficiently.

The system is designed to comply with aviation regulations and data protection laws in Saudi Arabia, ensuring safe UAV operation and responsible management of collected data. Ethical considerations are prioritized, with measures implemented to protect privacy, prevent

algorithmic bias, and maintain transparency in system usage. The project demonstrates high standards of software engineering, reliability, and accountability, ensuring responsible application of technology. By integrating innovation with legal and ethical responsibility, the project aims to foster public trust and contribute positively to societal well-being. Finally, the organization section summarizes how the report is structured.

1.8 Project Organization

The organization of this project is divided into structured chapters, each addressing a critical aspect of development and ensuring clarity in both technical and managerial processes. The chapters are summarized as follows:

Table 2. Organization of the Project Report

Chapter 1: Introduction	Provides an overview of the project, including motivation, justification, problem statement, aims, and objectives. It also explains the scope and implications of the work.
Chapter 2: Background and Review of Literature	Covers related work and existing studies on UAVs in public safety and crowd monitoring. It highlights recent approaches, technologies, and challenges, while identifying gaps that our project aims to address.
Chapter 3: Software Project Management Plan (SPMP)	outlines the management aspects of the project, including scope, objectives, assumptions, risks, resources, scheduling, and deliverables.
Chapter 4: Methodology/Software Requirement Specification (SRS)	Defines the system's functional and non-functional requirements, data needs, and user interactions. This chapter also describes the methodology guiding development.
Chapter 5: Software Design Specification (SDS)	Details of the architecture and design of the system, illustrating how different modules communicate. It also explains interface designs and user interaction flows.
Chapter 6: Implementation	Covers the practical development of the UAV-based crowd monitoring system. This includes datasets used, AI models applied, and integration of hardware and software components.
Chapter 7: Software Testing Plan (STP)	Presents the testing strategy, including component, security, and acceptance testing, ensuring the system meets functional and performance requirements.
Chapter 8: Conclusion	Summarizes the outcomes of the project, contributions made, lessons learned, and possible directions for future research and system enhancements.

In the next chapter “**Background and Review of Literature**” we will review previous studies and technologies that will help structure the project design.

Chapter 2: Background and Review of Literature

This chapter builds on the introduction and reviews related research and restate the problem to clarify how existing research connects to our project's objectives. In this literature review we will explore the key components of UAV-based crowd monitoring, structured into four themes, shown in Figure 2, for clarity and focus. It begins with exploring AI Models for UAV Crowd Detection, the core of this research, followed by a comparative Study of Datasets for Crowd Monitoring, assessing their suitability and limitations. Subsequently, the third theme, UAV Simulation Platforms, covers tools for model validation. And finally, the fourth theme, Embedded Hardware Platforms which addresses deployment constraints and integration challenges. Together, these themes build a clear foundation linking lightweight model design to real-world UAV implementation.

Structure of the Literature Review

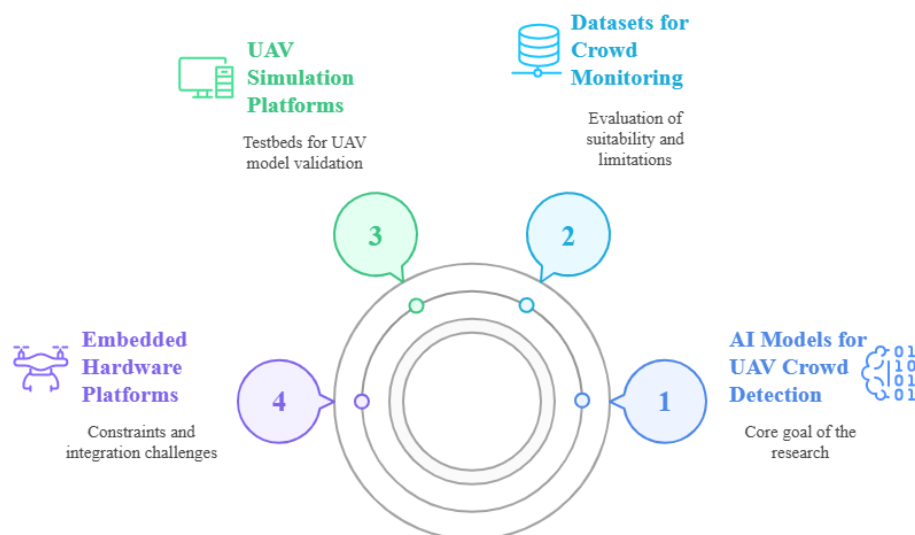


Figure 2. Structure of literature review

In most events, we need crowd management (CM), which is planned and real-time coordination of people's movement to maintain safety during mass gatherings such as religious events, protests, or concerts, aiming to keep densities safe, manage flow, and prevent incidents. Therefore, relying solely on fixed closed-circuit television (CCTV) and ground patrols alone struggles to meet the CM requirements due to blind spots and can't redeploy when flows shift, so engineers introduced drones in public safety and crowd management.

Drones are unpiloted aircraft (unmanned aerial vehicles, UAVs) that function without a pilot, relying on human control to autopilot autonomy using embedded sensors for safe navigation. Although UAVs were initially developed for military purposes, in the past decade their rapid civilian commercialization has transformed them into widely available tools for public safety and services. In crowd contexts, drones turn crowds from a blur into a plan: UAVs cover large spaces and produce high-resolution visuals that enable faster detection and better supervision. UAVs help in crowd counting, density flow mapping that creates heatmaps, and spotting unusual behavior. Compared with traditional methods, they offer faster deployment, fewer blind spots, handling different tasks easily, and improved line-of-sight (LoS) because of flying above, which proves how aerial perspectives and redeploy situational awareness and support a quicker response [6].

Although UAVs provide unique advantages for crowd monitoring, their real-world deployment faces several technical, ethical, and regulatory challenges.

Ethical and Privacy Concerns:

According to [7], the use of drones for human monitoring involves ethical concerns around surveillance, permission, and confidentiality. For example, people in public places might not be aware that they are being monitored, which raises questions about how their personal information is stored and used. Similarly, [8] explains that even if drones can technically monitor people, doing so without proper safeguards may lead to public distrust. Duffy et al. [10] further emphasize that UAV deployment requires careful pre-flight planning and community engagement, particularly in sensitive environments.

Battery Life and Endurance:

Moreover, low battery capacity and limited flight endurance are one of the most common challenges in UAV-based crowd monitoring. This limitation directly impacts coverage, as areas remain unmonitored whenever a UAV leaves to recharge. According to [9], alternative solutions such as battery switching, UAV rotation, or wireless charging exist but are impractical for large-scale deployment. Duffy et al. [10] also highlights that battery redundancy is important to prevent UAV downtime during critical tasks.

Environmental and Illumination Limitations:

Also, UAV-based monitoring is highly sensitive to weather, such as rain, fog, and snow can reduce sensor accuracy, which will degrade detection performance. Zhang et al. [11] demonstrate that severe weather poses significant challenges for UAV perception systems, while strong winds may affect UAV stability [10]. Figure 3 shows how dense fog, rainfall, and strong light bias affect RGB and LiDAR outputs, leading to degraded detection reliability.

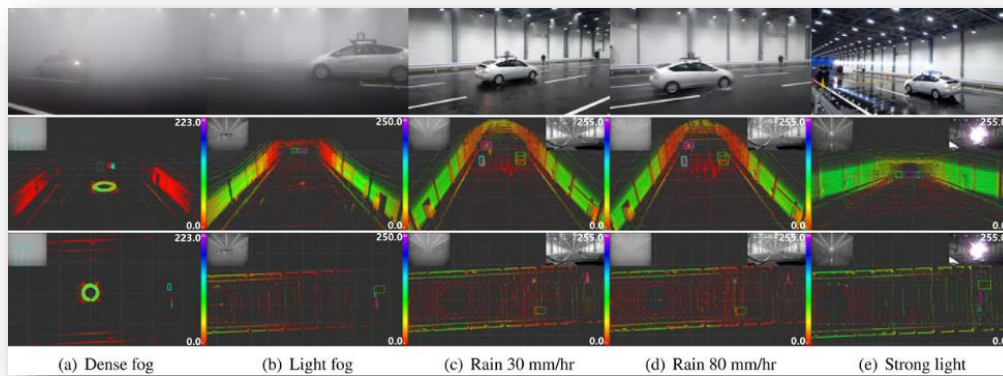


Figure 3. weather impacts: dense fog, rain, and strong light degrade UAV perception (Reproduced from [11])

Beyond weather, illumination remains a critical limitation for UAV-based monitoring, especially in low-light or night-time. One strategy to overcome this challenge is the integration of thermal infrared (TIR) cameras with conventional RGB sensors. TIR imaging enables trustworthy detection of human presence under even under poor lighting. As shown in Figure 4, thermal imagery maintains visibility when camera's quality degrade. Previous studies have used both computer vision and deep learning approaches to boost thermal-based crowd detection [12].

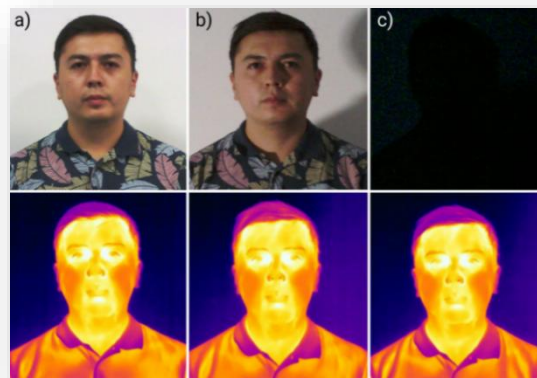


Figure 4. Visual (top) and thermal (bottom) images under different lighting conditions: (a) normal illumination, (b) low illumination, and (c) complete darkness. (Reproduced from [13])

Regulatory and Security Constraints

In Saudi Arabia, UAV operations are regulated by the General Authority of Civil Aviation (GACA) under *Part 107*, which requires registration, licensing, and prohibits flights over densely populated or restricted areas without prior approval [14]. While these regulations are essential for safety, they limit rapid deployment in crowd monitoring contexts. At (IAU), additional institutional rules require security clearance and adherence to internal guidelines for research-related UAV use, adding another layer of oversight. Beyond formal regulations, UAV misuse remains a security concern: unauthorized drones may threaten privacy.

The challenges identified in UAV-based crowd monitoring highlight that hardware alone cannot guarantee safe and effective deployment. Several mitigation strategies have been proposed in our project to accomplish our vision.

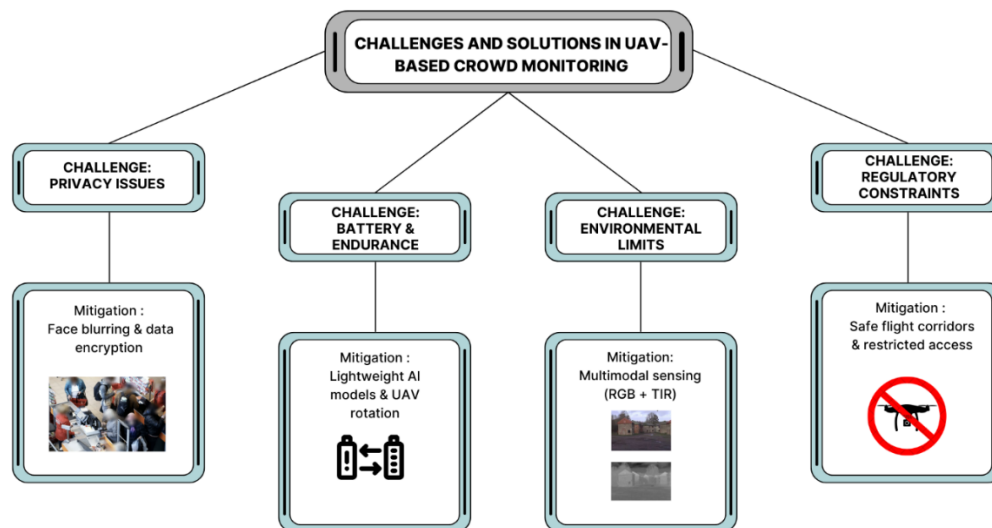


Figure 5. Summary of UAV crowd monitoring challenges and proposed solutions.

While these measures mitigate risks, they also emphasize the need for AI integration on board processing that balances accuracy, efficiency, and compliance. This naturally transitions to the role of artificial intelligence (AI) in UAV-based crowd detection, where intelligent models can enable real-time results, reliable perception, and decision-making under such constraints.

In this chapter, the terms heavyweight and lightweight AI models are used based on computational complexity. Heavyweight models are characterized by high computational cost, typically measured by a large number of floating-point operations (FLOPs), which leads to increased inference latency. In contrast, lightweight models require fewer computations, enabling faster inference and making them suitable for real-time deployment on embedded UAV hardware. With the structure of the chapter established and the problem is defined, our review will begin with Theme 1, which examines existing AI models used for UAV-based crowd detection

2.1 AI Models for UAV Crowd Detection:

Public-safety UAVs have emerged as an important area of research in recent years. However, intelligent control system integration became a critical part of a complete public-safety drone system. A series of recent studies has shown that AI algorithms enable drones to make complex decisions. The scope of a review done by Caballero-Martin et al. [15] was to identify and examine in detail the most salient AI algorithms that have proven effective in various drone operations. This state-of-the-art review categorizes various AI methodologies, including machine learning, Deep learning, and Reinforcement Learning (RL). Unlike supervised learning approaches, RL focuses on decision-making through trial and error in dynamic environments, making it particularly well-suited for UAV tasks such as path planning, obstacle avoidance, and multi-drone coordination. While their focus was on logistics applications, their findings emphasized that AI enables drones to operate autonomously in uncertain and evolving environments. Hence, crowd detection systems have evolved from classical image processing to powerful smart solutions using Artificial Intelligence integration, which has significantly increased accuracy and helped with crowd monitoring and management. In this section, we are going to discuss various AI-driven models for crowd detection, categorized into three main subsections: heavyweight models, lightweight models, and, lastly, model compression techniques that optimize heavyweight models to become more fitting to real-world scenarios.

2.1.1 Heavyweight AI Models

Recent advances in crowd detection systems have been driven by the adoption of heavyweight AI architectures. These models include object detection frameworks such as YOLOv5, graph-based convolution networks, and encoder-decoder architectures such as UNet and DeepLab. They have shown impressive results in handling complex visual tasks, including people counting and crowd detection.

Tzelepi and Tefas [16] introduced a CNN-based approach that generates crowd-presence heatmaps on a dataset combining drone and YouTube images. Their method formulates the optimization problem by combining the Softmax Loss (L_s) with the Euclidean Loss (L_e), expressed As defined in [16, eq. (10)], the total loss is:

Equation 1. total loss

$$L = L_s + \sum_i \eta_i L_e$$

noting that the Euclidean loss can be attached to deep neural layers. Hence, for a deep learning model with N_L layers, it will produce a total loss in the regularized training scheme. This

formula is considered a novel mathematical piece, as it defines the full mathematical problem that their network solves. Graph Embedded CNN in Human Crowd Detection achieved nearly 95% accuracy with 5-fold cross-validation, which is superior compared to other methods discussed in the study, such as standard L1 and L2 regularization schemes. The output of this approach is illustrated in Figure 6.

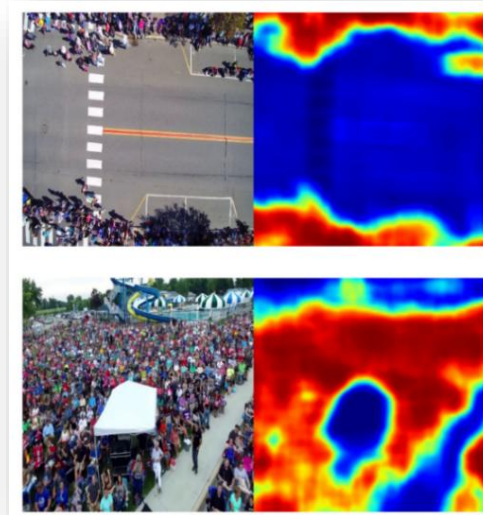


Figure 6. Initial images on the left and the corresponding detected crowd. (Reproduced from [16]).

The limitation of this study is its limited dataset, which comprises 11,840 images that only include daytime scenes. This paper is significant and extremely relevant to our research on crowd monitoring and management. While the authors also explored a fully convolutional CNN variant to reduce computational load, the core Graph Embedded CNN architecture remains computationally intensive and is therefore categorized as a heavyweight model in UAV-based crowd monitoring [16]. Similarly, Herrera et al. [17] employed YOLOv5 for large-scale crowd monitoring tasks, utilizing over 35,000 UAV images collected across 24 different beaches. Their framework implemented two AI models: one dedicated to counting individuals on land and in water, and another to classify usage types (e.g., recreational vs. active engagement). The people-counting model achieved 91–95% accuracy, while the classification model reached 85–92% accuracy the heatmap is shown in Figure 7. These results underscore the efficiency of object detection models in providing fast and reliable analysis of large gatherings, highlighting UAVs' potential in real-time public safety applications.



Figure 7. Example video frame of a video demonstrating the proposed crowd detection method results on a previously unseen UAV-captured video. URL: http://bit.ly/crowd_det_results. (reproduced from [17])

Despite these promising results, the study faced certain limitations, especially its restricted time frame of five months, which may not account for performance under unusual conditions such as adverse weather, seasonal fluctuations, or special events. Nevertheless, this work demonstrates the scalability and robustness of deep learning–based detection methods in practical, real-world environments, strengthening the case for AI integration into UAV-based crowd monitoring [17].

Recent work has also advanced the integration of AI in UAV systems by addressing both real-time density estimation and safe operation in crowd scenarios. Ptak et al. [18] developed an end-to-end onboard system using encoder–decoder architectures such as UNet, UNet++, and DeepLabV3+ to generate density maps without cloud reliance. Deployed on lightweight hardware platforms (Jetson Xavier NX, Raspberry Pi 4, Intel NCS2), their model processed VisDrone 2020 images in real time, achieving an average error of 18–20 people per frame. While this demonstrated the feasibility of edge-based crowd monitoring, it faced challenges in computational overhead at higher resolutions.

Early approaches to UAV-based crowd analysis focused on accuracy-oriented architectures with less consideration of deployment constraints. Castellano et al. [19] introduced an approach that couples spatial graphs with fully convolutional networks (FCNs) for crowd density estimation in aerial imagery. By capturing spatial dependencies alongside pixel-level features, the approach achieved high accuracy; however, its computationally demanding nature restricted its use in real-time UAV applications. Similarly, Elharrouss et al. [20] introduced Drone-SCNet, a scaled cascade network to address scale variation in drone images. While the cascaded structure improved accuracy through iterative density map refinement, the model remains computationally intensive, hence not compatible with embedded platforms.

Rather than resorting to cascaded structures, Zhao et al. [21] extended this line of research by proposing a hybrid point–density network, a unified framework that combines localization and

counting. Although the dual representation enhances interpretability and accuracy, its memory and processing requirements remain high. Collectively, these works highlight the accuracy-efficiency trade-off of heavyweight crowd analysis models.

Overall, these approaches highlight the strengths of heavyweight deep learning models in UAV-based crowd detection. However, their computational intensity and reliance on high-end processing hardware often makes them not as feasible for real-world use.

2.1.2 Lightweight AI Models

In contrast to heavyweight architecture, recent studies have also focused on lightweight and embedded AI models that balance accuracy with the strict computational and energy constraints of UAV platforms. According to this, Papaioannidis et al. [22] proposed a multi-task deep CNN to define no-fly zones by detecting crowd areas. Tested on DroneCrowd and AirSimCrowd datasets, their system reached 86% accuracy and was optimized for embedded deployment. Although limited to binary classification (crowd vs. non-crowd), this work shows how AI supports UAVs in both analyzing and safely navigating populated environments.

Khan et al. [23] proposed LCDnet, a lightweight CNN-based model that reduces the number of parameters without compromising performance. Its real-time performance on Jetson Nano/Xavier platforms demonstrates the feasibility of lightweight crowd density estimation in real-world UAV applications. Starting from object detection frameworks, Alhawsawi et al. [24] enhanced YOLOv8 with the addition of a context augmentation module so that the model can learn complex crowd patterns at the cost of inference speed. Similarly, Yallamraju and Jana [25] blended RegionViT elements in a multi-scale YOLOv8 pipeline, striking a balance between transformer power and YOLO speed. Although this comes at small overhead compared to pure YOLO models, the hybrid approach remains feasible for UAV applications. These light-weight approaches show that careful design in architecture is able to facilitate real-time UAV crowd surveillance without compromising substantial accuracy.

There is a clear gap in the development of lightweight AI models for crowd detection; most existing articles make great improvements in heavyweight architectures, which provide high accuracy but are difficult to deploy due to complex hardware. Our work addresses this gap by developing a lightweight AI model that balances performance and efficiency, enabling it to fit a wide real-world uses.

Table 3. Summary of recent UAV-based crowd analysis and detection models

Study	Model	Application	Accuracy	Deployment	Complexity Level
[16]	Graph Embedded CNN	Crowd detection (heatmaps, no-fly zones)	~95% accuracy (5-fold CV)	Limited dataset (daytime, ~11k images). Not optimized for UAV onboard	Heavyweight
[17]	YOLOv5 (dual-model pipeline)	Large-scale crowd monitoring & classification	Counting: 91–95%; Classification: 85–92%	Real-world UAV deployment but limited timeframe (5 months).	Heavyweight
[18]	UNet, UNet++, DeepLabV3+	Density estimation, crowd counting	Avg. error: 18–20 ppl/frame	Deployed on Jetson Xavier NX, RPi4, Intel NCS2 (edge devices). Real-time feasible at lower resolution	Heavyweight to medium
[19]	FCN + Spatial Graphs	Crowd density estimation from aerial images	Accuracy 87.95%, Precision 88.37%, Recall 87.76%	Evaluated on aerial imagery datasets (not embedded hardware)	Heavyweight
[20]	Drone-SCNet	Crowd counting on drone images	MAE: 58.5–93.9, MSE: 99.1–231.6	Focused on accuracy; no embedded deployment reported	Heavyweight
[21]	PDNet (Point + Density Hybrid)	Crowd counting & localization on UAV imagery	localisation metrics (L-mAP, L-AP@10, L-AP@15, L-AP@20)	Algorithmic validation, not optimized for edge deployment	Heavyweight
[22]	Multi-task Deep CNN	Detect crowd areas → no-fly zones	~86% (DroneCrowd, AirSimCrowd)	Optimized for embedded deployment. Binary classification only.	Lightweight
[23]	LCDnet	Real-time crowd density estimation	MAE: 21.4 (DroneRGBT), 13.1 (CARPK)	Deployed on Jetson Nano/Xavier in real time	Lightweight
[24]	Enhanced YOLOv8 + Context Module	Crowd counting in complex drone imagery	mAP@50: 82.1%, mAP@70: 76.2%;	Suitable for UAV inference; real-time feasible	Lightweight
[25]	RegionViT–YOLOv8	Object detection & tracking for UAV surveillance	Accuracy 96%, Precision 95%	Designed for UAV surveillance, tested on UAV imagery	Lightweight (hybrid)

Table 4. Coverage of AI Approaches in UAV-Based Crowd Detection

Study	Crowd Detection	Heatmap / Density Estimation	Real-Time Onboard Processing	Safety / Regulation	Identified Limitations	Model Type
[16]	✓	✓	✗	✗	limited dataset (daytime only)	Heavyweight
[17]	✓	✗	✗	✗	Short dataset time frame	Heavyweight
[18]	✓	✓	✓	✗	Higher computational cost at higher resolutions	Lightweight (onboard Jetson/RPi/NCS2)
[19]	✓	✓	✓	✗	Limited testing in crowded urban areas	Heavyweight
[20]	✓	✓	✓	✗	Computationally expensive due to the cascade structure	Heavyweight
[21]	✓	✓	✗	✗	High energy consumption on UAV hardware	Heavyweight
[22]	✓	✗	✓	✓	Binary classification only (crowd vs. non-crowd)	Lightweight (embedded-friendly)
[23]	✓	✓	✓	✗	Focused on fixed cameras; needs UAV integration	Lightweight (LCDnet CNN)
[24]	✓	✓	✓	✗	High complexity in dense scenes	Lightweight (Enhanced YOLOv8)
[25]	✓	✗	✓	✗	High computation for multi-scale detection	Lightweight (RegionViT-YOLOv8)
Ru'ya	✓	✓	✓	✓	-	Lightweight real time AI model

As summarized in Table 4, existing studies demonstrate the potential of AI for UAV-based crowd detection, but none provide a complete solution that balances accuracy, lightweight deployment, and regulatory compliance. Current research is either constrained by limited datasets, high computational requirements, or narrow focus areas such as binary classification or logistics applications. Our project addresses these gaps by developing lightweight AI architectures optimized for UAV hardware, training on diverse datasets that cover varying

crowd densities, lighting, and weather conditions, and ensuring real-time onboard processing with minimal computational overhead. Additionally, the project incorporates safety and regulatory considerations, such as privacy-preserving detection and compliance with IAU regulations.

2.1.3 Model Compression Techniques

In addition to purpose-built lightweight architecture, another orthogonal research area is to convert heavyweight models to light forms. Pruning and quantization are common practices where pruning removes redundant neurons or convolutional filters and quantization reduces floating-point weights to lower-bit representations (e.g., INT8), thereby reducing memory footprint and inference latency. Knowledge distillation also enables a large teacher model to share its knowledge with a smaller student model that can be more easily deployed. For instance, a model such as Drone-SCNet [20] can be reduced in size via pruning and quantization to meet nearly real-time deployment on UAV hardware. Similarly, FCN-based architectures such as those in Castellano et al. [19] can serve as teachers in distillation pipelines to learn student networks with LCDnet-like performance.

Table 5. Model Compression for UAV Deployment

Study	Model	Technique	Dataset	Efficiency Gains	Accuracy Impact	UAV Relevance
[26]	Distilled lightweight crowd model	Weighted Knowledge Distillation	Multiple crowd datasets (edge/UAV focus)	Significant reduction in student model size	Improved accuracy over baseline lightweight models	Explicitly targeted for UAV/edge deployment
[27]	SlimYOLOv3	Channel pruning	VisDrone2018	FLOPs ↓90.8%, Params ↓92%, ~2× faster	Comparable to YOLOv3	Real-tonboard detection with reduced compute load
[28]	PS-YOLO	Compact design	VisDrone	Params ↓41.3%, +26.7 FPS, FLOPs ↓6.1%	mAP ↑1.3% over YOLOv11-s	Lightweight by design, balances efficiency & accuracy

Table 5 demonstrates a comparison of common techniques used in crowd detection models. Khan et al. [26] recently introduced a weighted knowledge distillation approach for edge crowd counting and demonstrated how student networks learned under this scheme are capable of significantly improving accuracy without being too burdensome on UAV and low-power deployment. These optimization methods are a key bridge between accuracy-focused research and real-world application of UAVs, making models effective while hardware-friendly.

Zhang et al. proposed SlimYOLOv3, a compact object detection framework optimized for UAV applications. Using channel pruning with L1 regularization, as a result, SlimYOLOv3 achieved a 90.8% reduction in FLOPs and a 92% decrease in parameters while maintaining accuracy compared to the original YOLOv3 [27]. As well as Zhong et al. also introduced PS-

YOLO, a lightweight redesign using partial convolution and an optimized feature fusion module. Compared to YOLOv11-s on VisDrone, it reduced parameters by 41.3%, improved inference speed by +26.7 FPS, and even increased mAP by 1.3%. This shows how compact model design can deliver both efficiency and accuracy for UAV applications [28].

The effectiveness of AI-based UAV systems depends heavily on the datasets that train and validate them. Crowd monitoring datasets differ in scale, annotation style (bounding boxes, head labels, density maps), and environmental coverage (day/night, weather). Synthetic datasets have also emerged as valuable supplements, enabling model training in controlled scenarios.

Based on the reviewed literature, this project will use model compression techniques, including quantization, structured pruning, and knowledge distillation to enable efficient deployment on embedded UAV hardware. These techniques will be applied to reduce computational cost and inference latency while preserving detection accuracy, making them suitable for real-time UAV-based crowd monitoring. Theme 2 will therefore present a comparative analysis of existing datasets, highlighting their characteristics, strengths, and shortcomings, and explaining how our project leverages them to build more robust UAV-based crowd monitoring solutions.

2.2 Comparative Analysis of Datasets for Crowd Monitoring

Datasets are the most important part of every study on UAV-based crowd detection, the kind of data we use to train our models determines how effectively they will operate in the real world. One of the most common real datasets is **VisDrone**, which has over 10,000 annotated images collected from different urban and suburban areas. It mainly uses bounding boxes to label people, vehicles, and other objects, making it suitable for classification and detection tasks where the model learns to recognize and locate multiple object types. The performance on VisDrone is usually measured using mean Average Precision (mAP), which evaluates how accurately objects are detected across classes [19]. Another important dataset is **UAVDT**, which has more than 40,000 frames. Like VisDrone, it uses bounding boxes, but it is more focused on urban traffic scenes, making it useful for tracking tasks, where the goal is to detect and follow objects across multiple frames. Its performance is also commonly evaluated using mAP and MOTA (Multiple Object Tracking Accuracy), which measure detection and tracking quality together. However, UAVDT is less effective when the goal is dense crowd analysis [29].

On the other hand, the **DroneCrowd** dataset was designed specifically for human crowd monitoring. It contains about 213,000 frames, with head-point annotations and density maps, which are used for regression-based crowd density estimation. This means the model predicts continuous values, such as the number of people or density per frame, rather than class labels. Its performance is usually measured using MAE (Mean Absolute Error) and RMSE (Root Mean Square Error), which show how close the predicted crowd count is to the real one. DroneCrowd is highly useful for precise crowd counting and localization but is more complex and time-consuming to annotate [21]. To address issues of scalability and annotation costs, researchers have also introduced synthetic datasets such as **AirSimCrowd**, which generate UAV footage in simulation environments with automatic annotations. AirSimCrowd can be used for both classification and regression tasks, depending on how it is trained. The evaluation typically uses MAE and RMSE, similar to DroneCrowd, to assess counting accuracy. While synthetic data is not as realistic as real-world footage, it offers advantages such as low cost, flexibility, and the ability to model dangerous crowd scenarios that would be difficult to capture in practice [21].

Since our project is mainly about crowd monitoring, we focus on **DroneCrowd** and **AirSimCrowd** datasets. DroneCrowd is particularly well-suited for dense real-world crowds, while AirSimCrowd provides synthetic data that is scalable and safe for controlled testing. By combining them, we balance realism with flexibility. Figure 8 shows examples from both datasets, comparing real DroneCrowd images with synthetic AirSimCrowd images and their corresponding annotations.

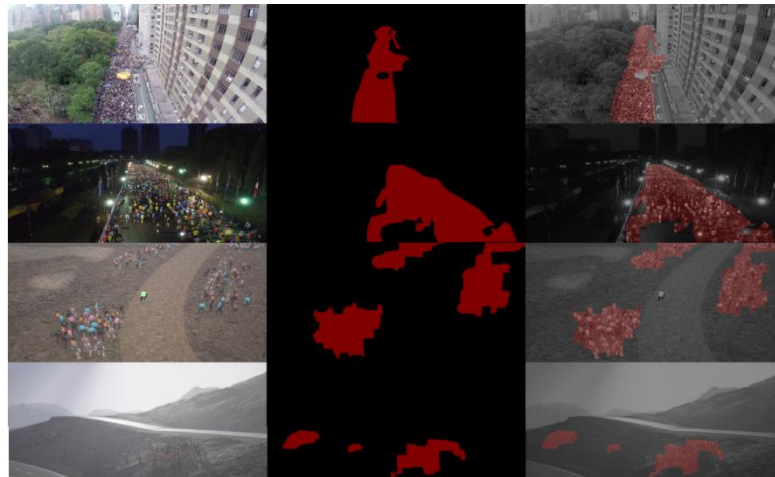


Figure 8. Example samples from DroneCrowd (rows 1–2, real UAV images) and AirSimCrowd (rows 3–4, synthetic UAV images), (from [21])

Figure 9 goes further by showing detection results on both datasets, including the input image, ground truth, and model prediction. These figures highlight why we chose DroneCrowd and AirSimCrowd as the foundation for our work, since together they support both realistic benchmarking and scalable experimentation.

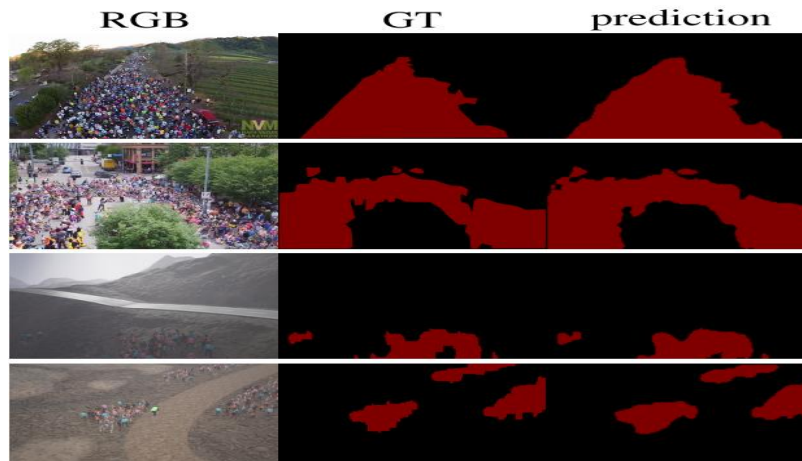


Figure 9. Crowd detection results on DroneCrowd (rows 1–2, real UAV images) and AirSimCrowd (rows 3–4, synthetic UAV images), (From [21]).

Table 6. Comparative summary of real-world and synthetic UAV crowd monitoring datasets (from [19], [21], [29]).

Dataset	Year	Size / Frames	Modality	Annotation Type	Strengths	Weaknesses	Train / Val / Test Split	Problem Type
VisDrone	2018	261,908 frames & 10,209 static images	RGB	Bounding boxes	Large-scale, diverse environments	Limited focus on dense crowds	6,471 / 548 / 3,190	Classification
UAVDT	2017	40k+ frames	RGB	Bounding boxes	Urban/traffic-rich environments	More traffic-oriented than crowds	30 / 10 / 10 sequences	Tracking
DroneCrowd	2020	213k frames	RGB	Head-points, density maps	Designed for dense crowd scenarios	Annotation complexity	112 / 12 / 111	Regression
AirSimCrowd	2021	Flexible	Synthetic	Bounding boxes, density maps	Scalable, flexible, safe for dangerous scenarios	Lacks full real-world realism	User-defined	Regression

For our project, we plan to combine the strengths of DroneCrowd (for dense real-world benchmarking) with AirSimCrowd (for simulation-driven flexibility). This dual approach ensures that our model evaluation is both realistic and scalable, supporting robust UAV-based crowd detection pipelines. Still, since each type of dataset has its own limits, we see simulation platforms as the next step to bridge this gap. They allow us to test models safely and under controlled conditions before moving to real-world deployment, which leads directly into our next theme.

2.3 Simulation as a Testbed for UAV Crowd Detection Models

Advancements in the fields of artificial intelligence usually require an environment to learn from and adapt to its dynamic states. A real-world environment is high-cost and hard to manage due to its trial-and-error basis. In recent years, there have been numerous developments in simulated environments. Robotic simulators are vital for academic research as well as the deployment of safety-critical implementations. With gaps between the real environment and the simulator being minimized, it can enable learning while overcoming the drawbacks of using a real-life environment [30]. An inclusive review of reinforcement learning-based drone simulators was conducted in [30] that discusses various drone simulators in terms of their technical features. The review concludes that an ideal UAV simulator with AI integration must have accurate sensors and computational efficiency to reduce training cost, effort, and, most importantly, time [30]. When working with UAV model research, selecting the best applicable UAV simulator is challenging and crucial. A review discussed prominent drone simulators and helped in narrowing down the key criteria for deciding which simulator fits best. They considered many aspects, for instance, integration of sensors, integration with other platforms, visual fidelity, and many more [31].

In this literature review, we shall focus on four simulators: AirSim, gym-pybullet-drones, Gazebo (*ROS2*), and MATLAB UAV Toolbox. By comparing and contrasting their Strengths and weaknesses in terms of their AI integration capabilities and how they apply to a crowd monitoring UAV. First, Aerial Informatics and Robotics Simulation (AirSim), is an open-source, cross-platform simulator for UAVs and other vehicles or objects. Developed by the Microsoft team, it has an environment that can interact with programming in computer vision and deep learning. AirSim offers a twelve-kilometer urban environment, shown in the Figure 10, and an API that receives data and controls drones in an independent manner [32]. Having a near photo-real rendering, even under varied lighting, makes it ideal for providing realistic input for crowd-detection or other computer-vision tasks. The APIs are available through many programming languages, such as C++, Python, and Java. However, AirSim is slower to run compared to other simulators and needs high computational resources, such as a GPU and a high-end PC [32].



Figure 10. A snapshot from AirSim shows an aerial vehicle flying in an urban environment. The inset shows depth, object segmentation, and front camera streams generated in real time (reproduced from [32]).

Subsequently, gym-pybullet-drones is a lightweight, open-source, quadcopter simulator, shown in the figure below. A quadcopter is a drone with four rotors; hence, it can move up, down, forward, backward, or rotate. The simulator is written in Python on top of the Bullet physics engine, and then it's packaged as an OpenAI Gym environment. The system is tailored to be compatible with AI and robotics projects; therefore, reinforcement-learning libraries can be used directly without the need for extra wrappers [33].

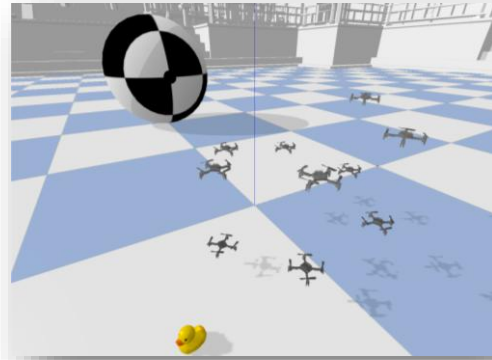


Figure 11. multi-drone simulation in gym-pybullet-drones (reproduced from [33]).

The environment model has many noticeable features, including realism; hence, it ensures realistic flight physics and an implemented PID control that delivers ease of use. It scales to multiple UAVs and provides parallel environments; thus, it can run faster, making it an excellent choice for RL experiments [33]. Its gym-style API shown in Figure 11, makes it a great choice for early testing of detection algorithms, such as crowd detection. Overall, gym-pybullet-drones remains a great lightweight simulator option, even when compared to heavier models such as AirSim. Next is Gazebo, an open-source 3-dimensional dynamic simulator. It is one of the earliest well-known models proposed in the field of robotics simulators. The environment consists of models and sensors, models being any object that sustains a physical representation, and a sensor, such as cameras, LiDAR, and IMUs. Gazebo provides a solid basis for developers and programmers to test their work and run their AI code in a safe virtual environment. It has a tight integration with the robot operating system, meaning algorithms can be translated from simulation to real hardware with subtle adjustments. It supports tasks from single-drone to multiple-drone scenarios, and its benefits include saving time and reducing risks during the development phase. Gazebo is considered a reliable simulator for AI-driven robotics in education and research [30] [31]. This makes Gazebo a well-suited testbed simulator for vision-based models, including crowd detection cases, enabling AI algorithms to be carefully tested before flying a drone in the real world. Moving on to another simulator, MATLAB UAV Toolbox is a modern simulation and design platform released as part of MATLAB, which offers various tools for model development, simulating, and analyzing the UAV behavior. The toolbox includes out-of-the-box flight scenarios and sensor simulation, which can feed object detection and crowd monitoring algorithms, allowing programmers to validate the AI pipelines before real-world tests [34].

Developers can design, navigate, and control algorithms in MATLAB, test the models in a virtual world, and finally, they can deploy them in a real-world environment. A main advantage of MATLAB UAV Toolbox is that it runs entirely on MATLAB; hence, it offers a smooth workflow for integrating AI by allowing to combine the drone modelling, data analysis, and algorithm testing all in a single platform. Nevertheless, a disadvantage of MATLAB is that it requires a paid license, and it is more restricted in terms of customization than other open-source simulators [34].

Table 7. Comparative Evaluation of UAV Simulators

Simulator	Strengths	Weakness	Ref
AirSim	✓ Very realistic city scenes for vision training ✓ Wide camera/segmentation outputs ✓ Multi-language APIs	✗ Heavy GPU and memory load ✗ Slower training and testing	[32]
gym-pybullet-drones	✓ Lightweight, easy to set up ✓ built-in interface with RL and computer-vision libraries ✓ supports multiple UAVs in parallel ✓ Ideal for rapid AI algorithms prototyping	✗ Limited realism and sensors ✗ not ideal for final vision accuracy testing	[33]
Gazebo (ROS2)	✓ Large sensor library (LiDAR, RGB, IMU) ✓ strong pipeline to move from virtual to real drone ✓ handles multi-drone and complex environments	✗ Higher setup complexity ✗ Visuals are less realistic and detailed than AirSim	[30],[31]
MATLAB UAV Toolbox	✓ complete MATLAB workflow ✓ ready to use flight and sensor scenarios ✓ good for quick proof-of-concept and AI algorithms	✗ Commercial license ✗ Limited customization and flexibility	[34]

Table 7 summarizes the key strengths and weaknesses of the four prominent simulators in the field. While considering our project needs, highlighting a lightweight AI crowd-monitoring drone, and our team's limitations on hardware resources, the gym-pybullet-drones simulator provides a good balance of its technicality and feasibility. It has low computational trail, which allows rapid execution. However, many reinforcement learning environments trade off realism for speed, such as gym-pybullet-drones, making the virtual environment too simple; thus, producing a model that is not reliable in real-world scenarios. Other simulators mentioned can also support our work, but they requires high-end resources, which contradicts our lightweight model needs. Nevertheless, AirSim can be used in an optional late stage if resources are available.

Based on the selected simulation platforms, the simulation testing process is conducted as follows. In simulation-based testing, the AI model will receive input from the UAV's RGB camera stream, with frames captured via simulations and preprocessed before inference. Testing will be conducted under controlled variations in altitude, crowd density, and lighting/weather conditions to evaluate robustness

In conclusion, gym-pybullet-drones offers the most practical environment for developing a lightweight UAV model for crowd detection and monitoring. AirSim, Gazebo

(ROS 2), and MATLAB UAV Toolbox remain useful for optional testing and future deployment stages. This layered approach ensures efficient research and an easier transition to real-life operations. After simulation, the following section addresses hardware deployment on real UAVs.

2.4 Deployment of Embedded Hardware Platform for UAVs

The use of artificial intelligence (AI) on embedded (UAV) platform hardware is a fast-evolving research area in UAV-based crowd counting. While detection and counting algorithms have been established to work accurately, applying these methods from controlled environments to real UAV platforms comes with inherent limitations that degrade performance tremendously. Unlike desktop or cloud environments, UAVs have limited lightweight embedded hardware with minimal computational and energy resources, and deployment is the real point for the viability of AI-based aerial surveillance [35][36].

One of the universal issues in recent literature is how to balance mission requirements with computational efficiency. Since UAVs are battery-powered, mission duration and flight duration are limited by them. In parallel, onboard processors such as NVIDIA Jetson modules or Raspberry Pi boards provide a fractional percentage of the processing capability of earth-bound GPUs [13]. As a result, AI algorithms tuned for laboratory precision only may fail to meet real-time latency, energy, and robustness requirements in UAV use cases. To close such gaps, researchers increasingly turn to hardware-aware optimizations like quantization and pruning in a bid to reduce computational and energy cost without compromising performance [35].

Hardware platform selection is central to enabling AI on UAVs. Early work focused on UAV airframe types such as fixed-wing, rotary-wing, and multicopters. More recent work focuses on embedded processors to enable deep learning inference with strict resource constraints. Payload selection, such as high-definition cameras or LiDAR sensors, also has an impact on deployment because extra weight reduces the endurance of flight [37].

Case studies highlight embedded AI modules to perform on-board processing. Ptak et al. [17] proposed a crowd density estimation system based on UAV that runs only on embedded hardware, e.g., encoder–decoder models on Jetson Xavier NX, Raspberry Pi 4, and Intel Neural Compute Stick 2. The system of the authors made real-time inference with a mean counting error of 18–20 individuals, though scalability was limited by dataset size and optical flow computation. Similarly, Papaioannidis et al. [21] employed a multi-task CNN for flight safety of UAV on Jetson Xavier, with near 86% accuracy in crowd area detection on synthetic and real data. Both the experiments confirm responsive UAV operations possible on embedded hardware.

A case study used the Jetson TX1 as the onboard processor in a UAV system built to monitor abnormal crowd behaviors. The drone used a FLIR infrared camera to capture scenes and an STM32 controller to handle flight, while the Jetson TX1 processed the data directly onboard. Although the TX1 is an older and less powerful device, it was still able to run the detection system in real-time and achieved over 90% accuracy in identifying behaviors such as crowd gathering or sudden escapes [38]. In contrast, the Jetson Orin Nano demonstrates how significant advancements these devices have undergone. With a far more powerful GPU and CPU, it can perform up to 40 trillion operations per second (TOPS) and still stay compact and energy-efficient enough to fit on a drone. In tests, it achieved around 25 frames per second,

nearly 80 times faster than the older Jetson Nano, making it much better suited for demanding real-time analysis [39].

To facilitate deployment decisions, Table 8 integrates the specifications, power consumption, advantages, disadvantages, and aptness of widely used embedded platforms. This comparison encapsulates the tradeoffs between computing power, energy efficiency, and integration ease, which have immediate implications for UAV crowd monitoring feasibility. encapsulates the tradeoffs between computing power, energy efficiency, and integration ease, which have immediate implications for UAV crowd monitoring feasibility.

Table 8. AI Hardware for UAV Crowd Detection

study	Device	CPU/GPU	Power Consumption	Pros	Cons	UAV suitability
[17]	Jetson Nano	Quad-core ARM Cortex-A57, 128-core Maxwell GPU	~5–10 W	Low-cost, lightweight, large community support	Limited performance for heavy models	Suitable for lightweight CNNs and prototyping
[21]	Jetson Xavier NX	6-core Carmel ARM CPU, 384-core Volta GPU	~10–15 W	High inference speed, supports advanced CNNs	Higher cost, increased power consumption	Suitable for real-time deployment with endurance trade-offs
[37]	Raspberry Pi 4 + Coral TPU	Quad-core ARM Cortex-A72 + Edge TPU	~3–5 W	Very low cost, energy efficient	Limited memory, cannot handle large CNNs	Suitable for basic detection and edge tasks
[17]	Intel Neural Compute Stick 2 (NCS2)	Intel Myriad X VPU	~1 W	Portable, USB plug-and-play, low power	Limited ecosystem, less optimized than Jetson	Suitable for rapid prototyping and simple inference
[38]	Jetson TX1	Quad-core ARM Cortex-A57 CPU, 256-core Maxwell GPU, 4 GB LPDDR4 RAM	~10–15 W	Reliable early embedded AI device, proven real-time onboard use	Older hardware, limited memory and compute	Suitable for basic UAV AI tasks (infrared, detection, behavior monitoring)
[39]	Jetson Orin Nnao	6-core ARM Cortex-A78AE CPU, 1024-core Ampere GPU + 32 Tensor Cores	~7–15 W	Very high performance for its size up to 40 TOPS, energy-efficient, compact	Higher cost, more power draw than Nano (affects endurance)	Excellent for advanced UAV AI (real-time crowd monitoring, complex detection)

These comparisons illustrate that successful deployment of AI on UAVs involves more than comparing the "fastest" or most powerful processor. Successful missions are predicated on balancing hardware performance, power consumption, and algorithm complexity with

endurance flight and payload requirements. Board-level modules like Jetson Xavier NX or Orin Nano offer outstanding performance enhancement, enabling real-time computation as well as advanced detection, but at the expense of higher power consumption, which may decrease flight duration. In contrast, offerings like Jetson Nano, Raspberry Pi 4 with Coral TPU, or Intel NCS2 are power-frugal and lightweight, hence suitable for simple designs or rapid prototyping. Overall, these findings emphasize the significance of using cautious, context-aware hardware selections and optimization methodologies to attain reliable, real-time crowd monitoring under practical operation limitations in UAV-based AI systems.

Conclusion

The review shows that AI-based UAVs have promising benefits in crowd monitoring by detecting, counting, and estimating the density of crowds in real-time. The literature highlights both their potential and the restrictions, such as privacy, regulation, battery limitations, and environmental sensitivity. To address this, our project incorporates safety and regulatory considerations and ensures compliance with IAU regulations. Another key insight is the critical importance of datasets, with current limitations including the lack of large, diverse, and weather-resilient data. For this, we will train on both real DroneCrowd and synthetic AirSimCrowd datasets to ensure robustness across densities, lighting, and weather conditions. Research also shows that simulators provide safe and cost-effective testbeds, but at a trade-off between realism and efficiency. In response, we will use gym-pybullet-drones for rapid, low-resource development and optionally validate with AirSim or Gazebo for added realism. Furthermore, the review shows that most AI models have demonstrated excellent accuracy in the task of crowd detection through UAVs, but some of them remain too bulky to be used in real-time on small drones. They also need strong hardware and consume a lot of energy and hence cannot be used in real-world environments. This literature review shares a lot of gaps such as Dataset limitation, model complexity such as the number of parameters or the architecture. We believe that overcoming these issues is possible by modifying and simplifying the existing AI models so that they can execute on embedded UAV processors with good accuracy. The model will be resilient to varying light, weather, and crowd conditions by utilizing both real and synthetic datasets, as well as enabling a dependable and regulation-compliant UAV monitoring system. In the following chapter, we outline the project's methodology, including how each component will be developed, tested, and integrated. This part will bring together the topics addressed in the literature review and transform them into a practical framework. The insights from this review support the Software Project Management Plan described in the next chapter.

Theme 0: Background and Problem Statement

Table 9. Summary of Literature under Theme 0 – Background and Problem Statement

	Author/s	Year	Title	Technique/s	Results
[6]	Khalifa AL-Dosari, Ziad Hunaiti, Wamadeva Balachandran	2023	<i>Systematic Review on Civilian Drones in Safety and Security Applications</i>	Systematic review of 45 publications (2011–2021); examined sensors (GPS, cameras, LiDAR) and software	Most studies relied on GPS (71%) and cameras (64%), with >50% integrating advanced sensors/software; drones shown effective in safety/security applications
[7]	Resnik, D. B., & Elliott, K. C.	2019	<i>Using Drones to Study Human Beings: Ethical and Regulatory Issues</i>	Ethical analysis and regulatory framework discussion	Highlighted ethical concerns including surveillance, permission, and confidentiality. Emphasized need for safeguards to maintain public trust.
[8]	Shakhatreh, H., Sawalmeh, A. H., Al-Fuqaha, A., et al.	2019	<i>Unmanned Aerial Vehicles (UAVs): A Survey on Civil Applications and Key Research Challenges</i>	Comprehensive survey of civil UAV applications	Summarized challenges in UAV deployment including energy limitations, navigation, and communication, provided overview of applications in surveillance, delivery, agriculture, and monitoring.
[9]	Qin, Y., Kishk, M. A., & Alouini, M.-S.	2020	<i>Performance Evaluation of UAV-Enabled Cellular Networks With Battery-Limited Drones</i>	Analytical modeling and simulations	Identified energy consumption as a major issue. Showed battery limits reduce UAV availability and coverage and discussed trade-offs in UAV network design.
[10]	Duffy, J.P., Cunliffe, A.M., DeBell, L., Sandbrook, C., Wich, S.A., Shutler, J.D., Myers-Smith, I.H., Varela, M.R., Anderson, K.	2017	<i>Location, location, location: considerations when using lightweight drones in challenging environments</i>	Field-based assessment and synthesis of UAV operational challenges across multiple ecosystems	Identifies key UAV field challenges such as pre-flight planning, environmental constraints, battery management, and ethics; emphasizes redundancy, weather assessment, and community engagement for safe and effective drone deployment
[11]	Zhang, Y., Carballo, A., Yang, H., Takeda, K.	2023	<i>Perception and sensing for autonomous vehicles under adverse weather conditions: A survey</i>	Comprehensive survey of autonomous perception systems in adverse weather	Demonstrates how rain, fog, and snow degrade RGB and LiDAR performance, highlighting implications for UAV perception and detection reliability under challenging conditions.

[12]	Jiang, C., Yang, X., Wan, Y., Zeng, T., Nie, M., Liu, Z.	2022	<i>Object detection from UAV thermal infrared images and videos using YOLO models</i>	Deep learning (YOLO) on thermal infrared imagery	Shows that combining RGB and TIR improves UAV-based crowd detection under low-light conditions, achieving robust human detection performance using deep models
[13]	Kuzdeuov, A., Aubakirova, D., Koishigarina, D., Varol, H.A.	2022	<i>TFW: Annotated thermal faces in the wild dataset</i>	Dataset creation and benchmarking with deep learning	Introduces a thermal face dataset enabling improved model training and detection accuracy in low-illumination scenarios, supporting thermal-based UAV monitoring applications .
[14]	General Authority of Civil Aviation (GACA), Saudi Arabia	2024	<i>GACAR Part 107 – Operation of Unmanned Aircraft Systems</i>	Regulatory framework (Part 107, Version 4.0)	Defines legal requirements for UAV operation in Saudi Arabia

Theme 1: AI Models for UAV Crowd Detection:

Table 10. Summary of Literature under Theme 1 – AI Models for UAV Crowd Detection

	Author/s	Year	Title	Technique/s	Results
[15]	A. D. Martín <i>et al.</i>	2024	<i>Artificial Intelligence Applied to Drone Control: A State of the Art</i>	Survey of AI methods for UAV control	Proves that AI methods improve UAVs
[16]	Maria Tzelepi and Anastasios Tefas	2021	<i>Graph Embedded Convolutional Neural Networks in Human Crowd Detection for Drone Flight Safety</i>	tested how the CNN algorithm produces a crowd-presence heatmap	achieved nearly 95% accuracy with 5-fold cross-validation
[17]	M. A. Husman <i>et al.</i>	2023	<i>Drone Insights: Unveiling Beach Usage through AI-Powered People Counting</i>	Two YOLOv5 models: one counting people, and another for usage types	First scored 91-95% accuracy, second model scored 85–92% accuracy.
[18]	B. Ptak <i>et al.</i>	2022	<i>On-Board Crowd Counting and Density Estimation Using Low Altitude Unmanned Aerial Vehicles—Looking beyond Beating the Benchmark</i>	CNN encoder–decoder (UNet, UNet++, DeepLabV3+)	the average counting error is roughly 18–20 people
[19]	G. Castellano, C. Castiello, C. Mencar, and G. Vessio	2020	<i>Crowd Detection in Aerial Images Using Spatial Graphs and Fully-Convolutional Neural Networks</i>	Combined spatial graph modeling with FCNN to identify crowd regions in aerial views	Showed accurate detection in structured environments but limited performance in complex, dense crowds
[20]	O. Elharrouss <i>et al.</i>	2021	<i>Drone-SCNet: Sealed Cascade Network for Crowd Counting on Drone Images</i>	Proposed a sealed cascade CNN framework for precise density estimation	Outperformed baseline models on drone-based datasets but required high computational power
[21]	L. Zhao, Z. Bao, Z. Xie, G. Huang, and Z. U. Rehman	2022	<i>A Point and Density Map Hybrid Network for Crowd Counting and Localization Based on UAVs</i>	Hybrid approach integrating point-based and density map estimation	Achieved high localization accuracy with balanced density estimation but increased onboard energy use

[22]	Christos Papaioannidis, Ioannis Mademlis, Ioannis Pitas	2021	<i>Autonomous UAV Safety by Visual Human Crowd Detection Using Multi-Task Deep Neural Networks</i>	multi-task CNN with two branches: the Semantic Image Segmentation Branch and the Image-to-Image Translation Branch	The model achieved nearly 86%
[23]	M. A. Khan, H. Menouar, and R. Hamila	2022	<i>LCDnet: a lightweight crowd density estimation model for real-time video surveillance</i>	Lightweight CNN for real-time crowd density estimation in video surveillance	Achieves real-time performance with reduced computational cost; suitable for deployment on resource-constrained platforms
[24]	A. N. Alhawsawi, S. D. Khan, and F. U. Rehman	2024	<i>Enhanced YOLOv8-Based Model with Context Enrichment Module for Crowd Counting in Complex Drone Imagery</i>	Modified YOLOv8 with context enrichment to improve feature representation	Improved detection accuracy in complex backgrounds while maintaining real-time processing
[25]	V. R. Yallamraju, S. Jana	2025	<i>Dynamic Object Detection and Tracking System on Unmanned Aerial Vehicles for Surveillance Applications Using RegionViT-Based Adaptive Multi-Scale YOLOv8</i>	Hybrid RegionViT + YOLOv8 (RV-AMYOLOv8); optimized with Fitness-based Random Variable Elk Herd Optimizer (FRVEHO).	Achieved 96% accuracy and 95% precision on UAV video tracking; superior performance compared to CNN-LSTM and GM-YOLO; robust to motion blur and background noise.
[26]	M. A. Khan, Hamid Menouar, Ridha Hamila, and Adnan Abu-Dayya	2025	<i>Crowd Counting at the Edge Using Weighted Knowledge Distillation</i>	Introduced an edge-optimized crowd counting model using teacher-student knowledge distillation to reduce model size	Achieved high counting accuracy with 4× reduction in computation cost, demonstrating efficiency for UAV and IoT edge devices
[27]	Pengyi Zhang, Yunxin Zhong, and Xiaoqiong Li	2019	<i>SlimYOLOv3: Narrower, Faster and Better for Real-Time UAV Applications</i>	Channel pruning with L1 regularization applied to YOLOv3; creates “slim” networks by removing less-informative feature channels	Achieved ~90.8% reduction in FLOPs and ~92% fewer parameters while maintaining accuracy compared to YOLOv3
[28]	Han Zhong, Yan Zhang, Zhiguang Shi, Yu Zhang, and Liang Zhao	2025	<i>PS-YOLO: A Lighter and Faster Network for UAV Object Detection</i>	Partial convolution-based lightweight YOLO redesign with optimized feature fusion	Reduced parameters by 41.3%, improved inference speed by +26.7 FPS, and increased mAP by +1.3% compared to YOLOv11-s on VisDrone

Theme 2: Comparative Analysis of Datasets for Crowd Monitoring

Table 11. Summary of Literature under Theme 2 – Comparative Analysis of Datasets for Crowd Monitoring

	Author/s	Year	Title	Technique/s	Results
[19]	G. Castellano, C. Castiello, C. Mencar, G. Vessio	2022	<i>Crowd Detection in Aerial Images Using Spatial Graphs and Fully-Convolutional Neural Networks</i>	Uses Fully Convolutional Neural Networks (FCN) with Spatial Graphs and Clustering Coefficient to detect and measure crowd density, applying	Improved accuracy in detecting crowded vs. non-crowded regions; effective in capturing density

				dual-loss training and Grad-CAM heatmaps for crowd localization.	distribution patterns in aerial images with limited computational cost.
[21]	C. Papaioannidis, I. Mademlis, I. Pitas	2021	<i>Autonomous UAV Safety by Visual Human Crowd Detection Using Multi-Task Deep Neural Networks</i>	Uses a multi-task CNN combining semantic segmentation and image-to-image translation (GAN) with shared ResNet-18 backbone and skip connections; trained with a multi-task loss to detect human crowds in UAV footage for safe-flight planning in real time.	Achieved $\approx 86\%$ accuracy detecting crowd zones
[29]	Khan et al.	2018	<i>The Unmanned Aerial Vehicle Benchmark: Object Detection and Tracking</i>	Uses multiple deep learning-based object detection and tracking models (Faster R-CNN, SSD, R-FCN, SORT, DeepSORT, MDNet) to benchmark UAV footage.	results show low mAP and MOTA due to small object size, motion blur, and occlusion, highlighting the challenge of aerial detection and tracking.

Theme 3: Simulation as a Testbed for UAV Crowd Detection Models

Table 12. Summary of Literature under Theme 3 – Simulation as a Testbed for UAV Crowd Detection Models

	Author/s	Year	Title	Technique/s	Results
[30]	Chan, Liu, Chen, Sagar, Kim	2024	<i>Reinforcement Learning-Based Drone Simulators: Survey, Practice, and Challenge</i>	Review of RL simulators; deep RL methods	Compared many simulators; RL better than supervised for drones
[31]	Dimmig, Silano, McGuire, Gabellieri, Hönig, Moore, Kobilarov	2023	<i>Survey of Simulators for Aerial Robots</i>	Review of 44 simulators; 14 in detail	Provided guidance for simulator selection
[32]	Shah, Dey, Lovett, Kapoor	2017	<i>AirSim: High-Fidelity Visual and Physical Simulation for Autonomous Vehicles</i>	Unreal Engine, HITL, APIs, realistic graphics	Worked well for quadrotor tests; good vision and sensor data.
[33]	Panerati, Zheng, Zhou, Xu, Prorok, Schoellig	2021	<i>Learning to Fly—Gym Environment with PyBullet Physics</i>	Gym API, PyBullet physics, multi-agent RL	Supports PID and RL and runs many quadcopters.

Theme 4: Deployment of Embedded Hardware Platform for UAVs:

Table 13. Summary of Literature under Theme 4 – Deployment of Embedded Hardware Platforms for UAVs

	Author/s	Year	Title	Technique/s	Results
[35]	M. H. Rahman, M. A. S. Sejan, M. A. Aziz, et al.	2024	A Comprehensive Survey of UAV Detection and Classification Using Machine Learning: Challenges, Solutions, and Future Directions	Comprehensive survey summarizing UAV detection and classification using machine learning methods	Highlighted limitations of current datasets, lack of real-time edge performance studies, and absence of

				such as CNNs, SVMs, and ensemble learning.	unified benchmarks for diverse UAV environments.
[36]	T. Ahmad et al.	2025	Future UAV/Drone Systems for Intelligent Active Surveillance and Monitoring	Survey on integrating AI, IoT, and edge computing for next-generation UAV surveillance and monitoring.	Provided future design directions but no experimental evaluation; identified a gap in standardized testing frameworks and real-world deployment validation.
[37]	H. Hildmann and E. Kovacs	2019	Review: Using Unmanned Aerial Vehicles (UAVs) as Mobile Sensing Platforms (MSPs) for Disaster Response, Civil Security, and Public Safety	Literature review of UAV sensing systems, communication technologies, and control architectures for emergency and public safety applications.	Conceptual review without deep learning implementation; lacked quantitative results or evaluation of AI-based UAV perception systems.
[38]	Yanhua Shao, Wenfeng Li, Hongyu Chu, Zhiyuan Chang, Xiaoqiang Zhang , Huayi Zhan	2020	<i>A Multitask Cascading CNN with MultiScale Infrared Optical Flow Feature Fusion-Based Abnormal Crowd Behavior Monitoring UAV</i>	Used a Cascading CNN with Infrared Optical Flow on a Jetson TX1 for detecting abnormal crowd behavior.	Reached over 90% accuracy and worked in real time using infrared video on the UAV.
[39]	Xiaoyang Qiu,Yajun Chen ,Wenhao Cai,Meiqi Niu, Jianying Li	2024	<i>LD-YOLOv10: A Lightweight Target Detection Algorithm for Drone Scenarios Based on YOLOv10</i>	Designed LD-YOLOv10, a lightweight version of YOLOv10, tested on Jetson Orin Nano for drone use.	Achieved 25 FPS with reduced parameters while keeping high detection accuracy.

Chapter 3: Software Project Management Plans

1. Project Overview

This section discusses the project's main purpose, scope, objectives, assumptions, potential risks, and constraints. It also outlines the expected outcomes, deliverables, budget, and overall development roadmap.

1.1 Purpose, Scope, and Objectives

The plan begins by defining the overall purpose and scope.

1.1.1 Purpose

Our goal is to make the UAV capable of running these AI models directly on its hardware, so it can process live video without needing constant connection to a computer. To achieve this, we plan to use lightweight AI architectures and optimize them to run on embedded systems

The system will combine several technologies:

- Deep learning for real-time crowd detection and counting.
- Edge computing to process images onboard the UAV with minimal delay.
- UAV flight control integration so that the AI model works smoothly with the drone's navigation system.
- Simulation tools to safely test and evaluate performance before any real flight.
- Establish a communication pipeline between the UAV and a monitoring dashboard via wireless transmission for visualizing live analytics and alerts.

In short, the purpose of Ru'ya is to technically demonstrate how AI and UAV systems can work together to analyze crowds efficiently and safely, providing a foundation for smart monitoring solutions.

1.1.2 Scope

For now, the system will be implemented and tested only (IAU), due to the strict UAV flight regulations in Saudi Arabia. Therefore, all experiments and demonstrations will be performed in a controlled university environment, ensuring both safety and compliance with GACA guidelines.

1.1.3 Objectives

The main goals we aim to achieve during the project are:

- Develop a deep learning model trained to detect and estimate crowd density from aerial images.
- Train and optimize the model for real-time performance on edge devices.
- Simulate and test the system before running it on the actual UAV.
- Integrate the AI model into a UAV and test it inside (IAU) only.

1.2 Assumptions, Constraints and Risks

The following section outlines the main assumptions, constraints, and risks.

1.2.1 Assumptions

To successfully complete the Ru'ya project, several assumptions have been made. The main assumptions are as follows:

- Access to open-source UAV simulation tools, and public datasets.
- All team members have the required technical expertise in AI, Python, and UAV simulation.
- The university provides sufficient computational resources for model training.
- Project tasks are completed following the planned Waterfall model.
- Team members will maintain continuous communication and collaboration to ensure the smooth execution of project tasks.

1.2.2 Constraints

Some factors may limit the project's progress and performance. These include:

- Regulatory restrictions prevent UAV testing outside the university.
- Hardware limitations such as limited GPU memory and processing power may affect model complexity and inference speed.
- Time constraints due to the academic schedule may reduce the opportunity for extended testing and model fine-tuning.
- Limited number of participants involved in the project restricts the workload distribution.

1.2.3 Risks

Several risks could affect the project's progress or results. These risks, if not managed properly, could impact on the schedule, quality, or overall success of the system. Table 14 summarizes the probable risks and their impact.

Table 14. Project Risks

<i>Risk</i>	<i>Description</i>	<i>Impact</i>
<i>Hardware limitations or failures</i>	Drone components or onboard devices (like Jetson Nano or Raspberry Pi) might overheat, fail, or face compatibility issues during integration.	May delay testing or reduce system performance and stability.
<i>Model performance issues</i>	The lightweight AI model might not achieve the expected accuracy or real-time detection speed in real-world tests.	Could affect detection reliability and overall project success.
<i>Simulation-to-real gap</i>	Results in simulators might differ from real-world testing due to lighting, weather, or sensor noise.	Can cause unexpected behavior or inaccurate detection during flight.
<i>Regulatory and safety restrictions</i>	UAV flight rules (GACA) or university policies might limit where and when we can test the drone.	Could delay or restrict real flight experiments.
<i>Team coordination and time management</i>	Team members may have different workloads or schedules, slowing progress.	May cause missed deadlines.
<i>Data privacy and ethics</i>	Some datasets or collected videos might contain personal or sensitive information.	Could lead to ethical issues or data misuse if not handled properly.
<i>Overfitting of AI model</i>	The model may perform well on training data but fail to generalize on unseen crowd scenarios.	Poor real-world accuracy and unreliable predictions.

1.3 Project Deliverables

Project deliverables summarize the expected outputs working on Ru'ya project.

Table 15. Project Deliverables

<i>Deliverable</i>	<i>To whom</i>	<i>Delivery Media</i>	<i>Date</i>
<i>Project Proposal</i>	Dr. Rabab Alkhalifa	Softcopy	31 st August 2025
<i>Literature Review</i>	Dr. Rabab Alkhalifa and Mrs. Mehwash Farooqui		8 th October 2025
<i>Mid Semester Report</i>	Dr. Rabab Alkhalifa and Mrs. Mehwash Farooqui		16 th October 2025
<i>Submit Software Project Management Plan (SPMP)</i>	Dr. Rabab Alkhalifa		18 th October 2025
<i>Methodology</i>	Dr. Rabab Alkhalifa		23 rd October 2025
<i>Submit System Requirements Specification (SRS)</i>	Dr. Rabab Alkhalifa		29 th October 2025
<i>Submit Software Design Specification (SDS)</i>	Dr. Rabab Alkhalifa		5 th November 2025
<i>First Semester Final Report</i>	Dr. Rabab Alkhalifa		19 th December 2025
<i>First Semester Final Presentation</i>	Dr. Rabab Alkhalifa		21 th December 2025

1.4 Schedule and Budget Summary

In terms of the budget, our team estimates that the total cost of the project will be around 5,000 to 6,000 SAR. This amount should cover the main hardware components, software tools, and materials needed for testing and documentation. Since this is a university project, we plan to make the most of the resources already available at IAU, such as labs, simulation tools, and open-source software. This will help us stay within budget and spend mainly on essential equipment required for the technical implementation. To ensure feasibility, time and cost allocations are summarized in Table 16.

Table 16. The project Schedule

The Project Phase	Duration
Proposal Discussion	1 week
Research about crowd detection and drones	3 days
Dataset analysis	3 days
Literature Review	3 weeks
Mid Semester Report	6 weeks
SPMP	2 weeks
Methodology	1 week
SRS	5 days
SDS	1 week
First Semester Final Report	2 weeks
Implementing/Coding	20 weeks
STP	2 weeks
System Manual	2 days
Final Project Report	1 week
Presentation	2 days

1.5 Evolution of the Plan

This section of the document presents the preliminary Software Project Management Plan (SPMP) for the **Ru'ya** project, which follows the IEEE standards. The plan outlines the phases, deliverables, and procedures that will guide the project's development under the Waterfall model.

If no revisions are required, the team will continue the project based on the plans proposed in this document. However, any modifications or adjustments will only be made after careful discussion and agreement among all team members. All updates or revisions must be reviewed and approved by our supervisor, **Dr. Rabab**, before being officially applied to ensure that the project remains aligned with its academic and technical objectives.

1.6 Definitions and Acronyms

This section defines all technical terms and abbreviations that appear in this document to ensure clarity and consistency throughout the report

Table 17. Terms and Definitions

<i>Term</i>	<i>Definition</i>
UAV	A drone or aircraft that operates without a human pilot onboard, typically controlled autonomously or remotely for surveillance, mapping, or monitoring tasks [8].
SPMP	A formal document that defines the project's objectives, scope, schedule, resources, risks, and management processes to guide software development [40].
SRS	A detailed description of a software system's functional and non-functional requirements, serving as a contract between stakeholders and developers [41].
SDS	A document that describes the software architecture, components, interfaces, and data necessary to implement the SRS [42].
STP	A plan detailing the test strategy, objectives, resources, schedule, and procedures to ensure software meets its requirements [43].
Waterfall Model	A sequential software development model with distinct stages: requirements, design, implementation, testing, and maintenance [44].
WBS (Work Breakdown Structure)	A hierarchical decomposition of a project into smaller, manageable tasks or phases to organize and track work efficiently [45].

1.7 Document Structure

This document is organized as follows:

- **Project Overview:** Summarizes the project's purpose, scope, and objectives. It outlines assumptions, constraints, risks, deliverables, and provides a high-level summary of the schedule and budget. Acronyms, references, and the document structure are also included.
- **Project Organization:** Describes the project's internal structure, external interfaces, and the roles and responsibilities of each team member.
- **Managerial Process Plans:** Divided into five parts.
 1. Project Initiation: Covers staffing, staffing estimates, and employee training.
 2. Work Plan: Details the job breakdown structure, schedule, resource allocation, and budget.
 3. Project Tracking: Explains requirements management, schedule control, quality control, reporting, and project metrics.
 4. Risk Management: Defines the risk management plan.
 5. Project Close-out: Outlines strategies for completing and closing the project.
- **Technical Process Plans:** Describes the process model, methodology, tools and techniques, infrastructure, and product acceptance criteria.

- **Supporting Process Plans:** Includes documents that assist in the development and execution of project processes.

1.8 Business Criteria and Success Measures

The success of the Ru'ya project is evaluated based on its ability to deliver operational value while remaining feasible and cost-effective. The system is designed to minimize reliance on expensive cloud computing by performing AI inference onboard the UAV, reducing operational costs.

Scalability is a key business criterion, as the system should support future expansion to multiple UAVs and large-scale events. The project aligns with Saudi Vision 2030 by contributing to smart city initiatives and public safety technologies.

The system is considered successful if it improves situational awareness, reduces response time during crowd monitoring, and demonstrates readiness for real-world deployment with minimal operational overhead.

2. Project Organization

2.1 External Interfaces

Our work interacts with various external resources and entities that assist in advancing the project to its full potential. The Ru'ya project is supervised by Dr. Rabab Alkhalifa and co-supervised by Mrs. Mehwash Farooqui. It will operate under the administration of the Research and Development Department at Imam Abdulrahman Bin Faisal University, for internal safety and authorization procedures for the UAV, if required. Moreover, a cloud computing service such as Microsoft Azure or Amazon Web Services (AWS) is crucial, and it will be used for additional computational resources to ensure efficiency and reliability in our AI model training and testing. The Ru'ya project will also employ public datasets such as DroneCrowd and AirSimCrowd in its early stages for training, testing, and validating our AI-driven crowd detection algorithms. Furthermore, our program will be employed in the gym-pybullet-drones simulator to prove the system's performance before the actual physical implementation. Finally, our system's interface with the physical drone platform, which consists of the drone kit integrated with the flight controller (Pixhawk), Raspberry Pi as the computing unit, and several other essential components, that cooperatively guarantees a smooth transition to real-world use. Figure 12, shows all external interfaces working together to support all aspects of our Lightweight UAV-Based Crowd Detection and Monitoring project.

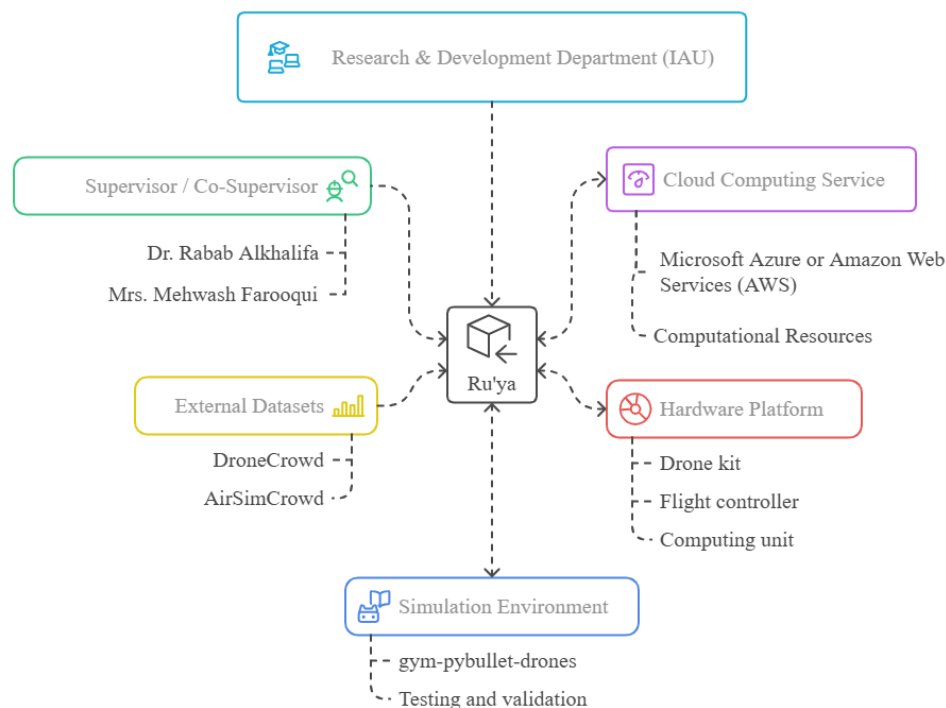


Figure 12. Context Diagram of the Ru'ya Project External Interfaces

1.2 Internal Structure

The project is supervised by Dr. Rabab Alkhalifa and co-supervised by Mrs. Mehwash Farooqui to provide our team with guidance and feedback to guarantee that the project is

aligned with the required criteria. Ru'ya will be developed by the project team members: Nora Aljomuh, Ritaj Alhamli, Joud Alahmari, Mozoon Alkhalis, and Sara Alashgar. Each team member has an assigned role and responsibility based on their skill, knowledge, and expertise to provide optimal results, which will be thoroughly explained in the following **“Roles and Responsibilities”** subsection. Figure 13 illustrates the internal organizational structure of the project.

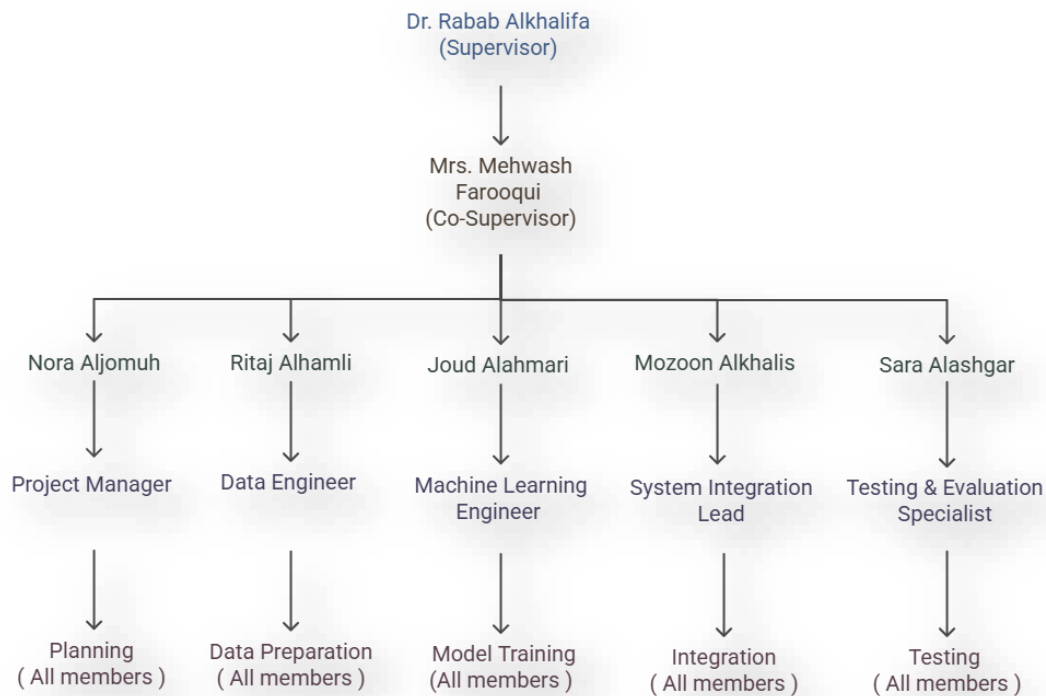


Figure 13. Internal structure

2.3 Roles and Responsibilities

Working on our project, each team member is responsible for a specific section assigned by the team leader, based on skill and expertise. The roles are allocated to cover all aspects and phases of our project. Even though the tasks are divided, our team ensures that all members are aware and involved in the progress of each task, highlighting the importance of communication and teamwork, to provide the best outcome for our project. Table 18 summarizes the roles and responsibilities of each team member.

Table 18. Roles and Responsibilities

Role	Responsibilities	Members
Project Manager	✓ Manages all project aspects. ✓ Directs the making of AI-focused documentation, reports, and presentations. ✓ Leads meetings and ensures smooth communication between team members. ✓ Ensures all submissions exhibit technical accuracy and professional standards.	Nora Aljomuh: leader All members
Data Engineer	✓ Gathers and prepares data used in training the AI model. ✓ Cleans, labels, and checks the quality of datasets. ✓ Keeps the data accurate, organized, and consistent. ✓ Works closely with the ML Engineer to modify and enhance data based on feedback regarding the AI model results.	Ritaj Alhamli: leader All members
Machine Learning Engineer	✓ Builds the AI model for crowd detection. ✓ Works on the integration of data to refine results. ✓ Improves accuracy and performance through hyperparameter tuning, validating, and testing.	Joud Alahmari: leader All members
System Integration Lead	✓ Runs the AI model in a simulated environment to test performance and reliability. ✓ Integrates the AI model with UAV hardware ✓ Ensuring the transition from simulated to physical systems is efficient.	Mozoon Alkhalis: leader All members
Testing Specialist	✓ Designs and runs test scenarios for the full AI system. ✓ Monitors the system's behavior in critical cases to spot weak points, delays, or inaccurate responses. ✓ Reviews and finalizes test data to approve the AI system before final deployment.	Sara Alashgar: leader All members

2.4 Stakeholder Identification

The Ru'ya system involves multiple stakeholders with different roles and interests. The primary stakeholders are government authorities and event organizers responsible for managing public safety during large gatherings such as Hajj and FIFA 2034. These stakeholders rely on real-time crowd analytics to support decision-making and emergency response.

Secondary stakeholders include UAV operators who monitor live feeds and respond to alerts through the system dashboard. Regulatory stakeholders such as the General Authority of Civil Aviation (GACA) and the Saudi Data and Artificial Intelligence Authority (SDAIA) are responsible for ensuring compliance with aviation and data protection regulations.

Academic and research stakeholders, including Imam Abdulrahman bin Faisal University, benefit from the system as a research platform for AI-driven UAV applications. Table 19 summarizes the main stakeholders.

Table 19. Stakeholder Identification

Stakeholder	Role	Interest
Event Organizers	Use analytics for crowd control	Public safety
Security Authorities	Monitor risks and alerts	Emergency response
UAV Operators	Control and monitor UAV	System operation
GACA	Regulate UAV flights	Legal compliance
SDAIA	Oversee AI & data usage	Data governance
University	Research & evaluation	Academic outcomes

3. Managerial Process Plans

3.1 Start-up Plan

In this section, we will highlight the key estimations, staffing, and training plans needed before starting the full development of the Ru'ya system. This plan will guarantee that all main technical resources, computational capacities, and team members' roles are clearly laid out from the early stages to ensure a smooth and reliable project workflow.

3.1.1 Estimates

Based on our skills and experience in working on previous AI systems, the following estimates were identified during the planning phase.

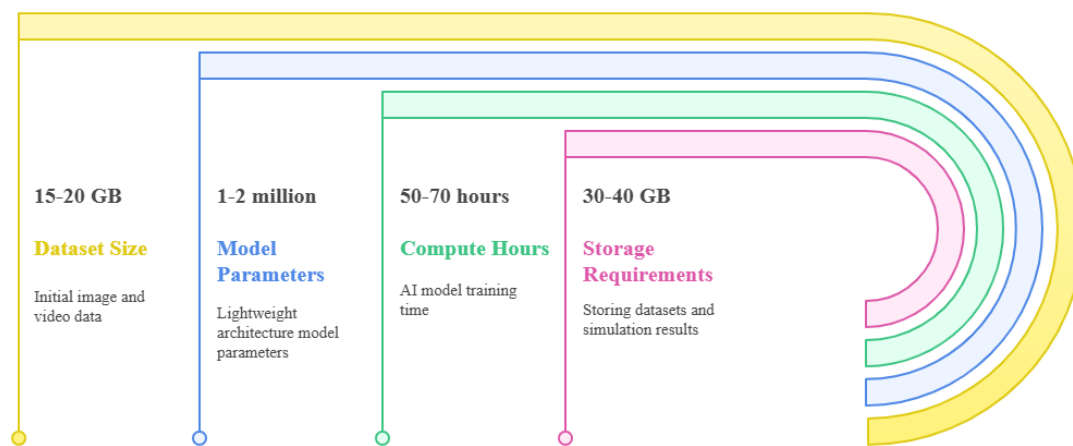


Figure 14. Software Resource Estimates

As illustrated in Figure 14, the estimated dataset size is approximately 15-20 GB, which consists of images and videos gathered from DroneCrowd and AirSimCrowd for the initial training and validating stages of our AI model. Moreover, the Ru'ya system's development and optimization phases are expected to have 1-2 million model parameters, as it is designed to have a lightweight architecture suitable for UAV deployments. The compute hours are valued to be 50-70 hours of training and hyperparameter tuning, which is logical and applicable for an AI model. Furthermore, the storage requirements are about 30-40 GB to store not only the dataset, but also the trained models and the simulation results. In addition to the software resources estimates, it is also worth mentioning that our project will later incorporate a UAV kit, controller unit, and computing unit as part of the hardware estimates for a physical integration after the successful software validation.

3.1.2 Staffing

The Ru'ya project will be developed by a team consisting of five senior AI students at Imam Abdulrahman Bin Faisal University, within the College of Computer Science and Information Technology. Every member has worked on various AI projects and is expected to have a solid foundation in artificial intelligence, data science, computer vision, and ML programming using Python. Each team member is assigned a specific technical role with the support of other members at every phase. Moreover, all members must have soft skills, including teamwork and decision-making, to ensure smooth cooperation.

Table 20. Human Resources and Duration

<i>Project Phase</i>	<i>Human Resource</i>	<i>Duration</i>
<i>Project Initiation</i>	Project Manager (Lead), supported by all members	2 weeks
<i>Planning</i>	Project Manager (Lead) supported by all members	1 week
<i>Dataset Preparation & Methodology</i>	Data Engineer (Lead) supported by all members	2 weeks
<i>Model Designing & Development</i>	Machine Learning Engineer (Lead) supported by all members	3 weeks
<i>Simulation & Integration</i>	System Integration (Lead) supported by all members	4 weeks
<i>Hardware Implementation</i>	System Integration (Lead) supported by all members	3 weeks
<i>Testing & Validation</i>	Testing Specialist (Lead) supported by all members	2 weeks
<i>Documentation & Presentation</i>	Project Manager (Lead) supported by all members	2 weeks

3.1.3 Project Staff Training

To achieve optimal results in the Ru'ya project, the team members must strengthen their skills and knowledge in deep learning, computer vision, and UAV concepts. Hence, a structured training framework was developed to enhance both theoretical and technical abilities. The following Figure demonstrates a series of training phases, each of which is designed to build upon the one before it, enabling members to progress easily and efficiently.

The training starts with improving our understanding of deep learning principles through reading and hands-on experiments using PyTorch and TensorFlow. Then in the computer vision phase, team members are going to use public tutorials and notebooks to learn and apply CNNs and OpenCV in small projects to test different crowd detection algorithms. Furthermore, in the model optimization techniques phase, the training will be research-based learning and implementation tests to explore quantization, pruning, and knowledge distillation, allowing the team to increase their knowledge in lightweight AI systems. Next, is the UAV simulation and hardware phase, which will focus on the practical implementation of our model. The training will be through the use of a simulator platform, such as gym-pypullet-drones, followed by learning the physical integration of the UAV system through publicly available tutorials and applied training. Finally, team members must learn to use collaboration tools, such as GitHub, for managing and updating project code efficiently, and Google Drive for sharing and storing all files. This will improve communication and help guide the team to work efficiently in a shared environment. Figure 15, shows the Ru'ya Project Training Framework.

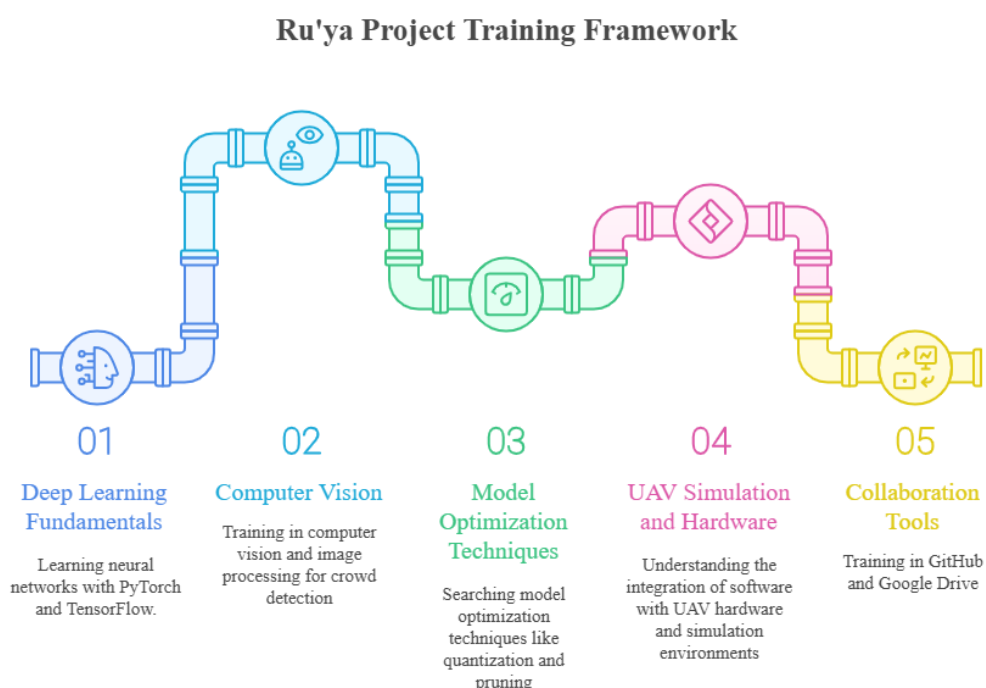


Figure 15. Training Framework

3.2 Work Plan

3.2.1 Work Breakdown Structure

Table 21 shows the work breakdown structure for this project.

Table 21. Work Breakdown Structure (WBS) for the Ru'ya Drone Project

Task	Activity	Resources		Timeframe	Deliverable	progress
		Software	Hardware			
1	Project Initiation			1 week	Project proposal & Supervisor Approval	Completed
1.1	Conduct initial face to face project meeting	MS word (meeting minutes)	PC	1 hour	Meeting minutes report	Completed
1.2	Research about crowd detection and drones	MS word, Web Browser (IEEE, MDPI, Scientific research)	PC	3 days	Research summary notes	Completed
1.3	Define project scope, objectives, and risks	MS word, Web Browser	PC	3 days	Project Scope Document	Completed
1.4	Identify UAV platform and challenges	MS word, Web Browser	PC	2 days	Technical Specification Sheet, Technical background	Completed
2	Work on Chapter 1			1 week	Mid-semester report	Completed
2.1	Write the Introduction Chapter, covering the project overview, problem statement, aims, and objectives.	MS word, Web Browser	PC	5 days	Draft of chapter 1	Completed
2.2	Add scope, justification, and motivation, linking the project to Vision 2030 and UAV applications.	MS word	PC	2 days	Finalized Chapter 1 content	Completed
2.3	Review and format the chapter 1	MS word	PC	3 hours	Polished Chapter 1	Completed
3	Literature Review			3 weeks	Mid-Semester Report	Completed
3.1	Search for relevant papers on UAV technology, crowd detection systems	MS word, Web Browser (IEEE, MDPI, Scientific	PC	2 days	Background research notes	Completed

		research), Shared Excel				
3.2	Work on Theme 0: Problem Statement and Challenges	MS word, Web Browser (IEEE, MDPI, Scientific research)	PC	1 day	Summary of research challenges	Completed
3.3	Work on Theme 1: AI Models: Review and compare crowd detection models			3 days	Summary table of AI models and findings	Completed
3.4	Work on Theme 2: Explore datasets and document relevant UAV-based datasets			3 days	Dataset analysis summary	Completed
3.4	Work on Theme 3: Simulation Tools			3 days	shortlist of preferred simulators	Completed
3.5	Work on Theme 4: Hardware Deployment			3 days	Hardware deployment summary	Completed
3.6	Summarize all reviewed papers	MS word	PC	2 days	Final literature summary	Completed
3.7	Overall revision	MS word	PC	1 day	Full Literature review	Completed
4	SPMP			2 weeks	Mid-semester report	Completed
4.1	Develop the project work plan, outlining responsibilities	Teams	PC	2 hours	Work plan draft	Completed
4.2	Compose the SPMP document	MS word	PC	5 days	Draft SPMP	Completed
4.3	Evaluate and refine the SPMP ensuring alignment with IEEE template	MS word	PC	3 days	Reviewed SPMP	Completed
4.4	Finalize and submit the SPMP	MS word	PC	2 days	Final Version	Completed
5	Submit Mid- semester report			2 hours	Mid-semester report final version	On Process
6	Work on the Methodology			1 week	Methodology	
6.1	Develop the project methodology	MS word	PC	3 days	Methodology draft	Completed
6.2	Review and refine project methodology	MS word	PC	2 days	Revised methodology	Completed
6.3	Submit the project methodology	MS word	PC	1 day	Finalized methodology	Completed
7	Software Requirements Specification (SRS)			5 days	SRS	Completed
7.1	Identify and analyze functional and non- functional requirements	MS word	PC	3 days	Draft SRS	Completed

7.2	Finalize and document detailed system requirements	MS word	PC	2 days	SRS document	Completed
7.3	Submit the SRS	MS word	PC	1 hour	Finalized SRS	Completed
8	Software Design Specification (SDS)			1 week	SDS	Completed
8.1	Design system architecture and simulator workflow	Draw.io / simulator	PC	3 days	System architecture draft	Completed
8.2	Create interface, use dataset, and AI model integration designs	Axure / MS Word	PC	3 days	Design diagrams	Completed
8.3	Finalize and submit the SDS	MS Word	PC	1 day	SDS Document	Completed
9	Progress to Final Report			2 weeks		Completed
9.1	Compile and update all project chapters and WBS tables	MS Word	PC	1 week	Updated report draft	Completed
9.2	Review and refine content based on supervisor feedback	MS Word	PC	3 days	Revised report	Completed
9.3	Prepare and submit the final version of the report	MS Word	PC	1 hour	Final report	Completed
9.4	Prepare and submit the final presentation slides	PowerPoint	PC	2 days	Final slides	Completed
10	Implementing the System			20 weeks	Ru'ya Drone	Completed
10.1	Preprocess the UAV dataset and perform data augmentation.	Python	PC	2 weeks	Clean dataset	Completed
10.2	Train and fine-tune AI models for crowd detection and density estimation	Python / TensorFlow	PC	3 weeks	Trained model	Completed
10.3	Evaluate and compare model performance metrics.	Python	PC	1 week	Evaluation report	Completed
10.4	Integrate trained models into the simulator environment for testing.	Simulator / Python	PC	3 weeks	Functional prototype	Completed
10.5	Develop the database and backend for data storage.	NoSQL / Python	PC	3 weeks	Database module	Completed
10.6	Develop and test the user interface for real-time monitoring.	Text Editor / Server	PC	3 weeks	Web UI	Completed
10.7	Deploy the system and conduct integration testing on hardware.	Python / Server	PC / UAV	5 weeks	Deployed system	Completed
11	Formulate the Software Testing Plan (STP)			2 weeks		Completed

11.1	Conduct unit and integration testing for AI model and database modules.	Python / Server	PC	5 days	Testing logs	Completed
11.2	Perform system-level testing using simulator and real UAV data.	Python / Simulator	PC	5 days	Testing results	Completed
11.3	Finalize and submit the Software Testing Plan (STP) document.	MS Word	PC	4 days	STP document	Completed
12	Submit the Final Report			1 week	Final Report	Completed
12.1	Compile and review the final report, ensuring all chapters are updated.	MS Word	PC	3 days	Final report draft	Completed
12.2	Develop the technical manual for system setup, testing, and demonstration.	MS Word	PC	2 days	Technical manual	Completed
12.3	Prepare and finalize the presentation slides	PowerPoint	PC	2 days	Final presentation	Completed
12.4	Submit the final report and presentation	MS Word	PC	1 hour	Final deliverables	Completed

3.2.2 Schedule Allocation

The table below outlines the scheduled activities, durations, and timelines for each phase of the Ru'ya Project.

Table 22. Project Schedule and Activity Timeline

No.	Activity	Start Date	Duration	End Date
1	Project Initiation	31st August 2025	1 week	7th September 2025
1.1	Conduct initial face-to-face project meeting	31 st August 2025	1 hour	7 th September 2025
1.2	Research about crowd detection and drones	1 st September 2025	3 days	3 rd September 2025
1.3	Define project scope, objectives, and risks	4 th September 2025	3 days	6 th September 2025
1.4	Identify UAV platform and challenges	7 th September 2025	1 day	8 th September 2025
2	Work on Chapter 1	9th September 2025	1 week	16th September 2025
2.1	Write the introduction chapter	9 th September 2025	5 days	13 th September 2025
2.2	Add scope, justification, and Vision 2030 link	14 th September 2025	2 days	15 th September 2025
2.3	Review and format chapter 1	16 th September 2025	3 hours	16 th September 2025
3	Literature Review	17th September 2025	3 weeks	8th October 2025
3.1	Search for relevant papers on UAV	17 th September 2025	2 days	18 th September 2025

	technology and crowd detection systems			
3.2	Work on Theme 0: Problem Statement and Challenges	19 th September 2025	1 day	19 th September 2025
3.3	Work on Theme 1: Review and compare AI-based crowd detection models	20 th September 2025	3 days	22 nd September 2025
3.4	Work on Theme 2: Explore and analyze UAV-based datasets	23 rd September 2025	3 days	25 th September 2025
3.5	Work on Theme 3: Evaluate simulation tools	26 th September 2025	3 days	28 th September 2025
3.6	Work on Theme 4: Hardware deployment and platform evaluation	29 th September 2025	3 days	1 st October 2025
3.7	Summarize all reviewed papers and perform overall revision	2 nd October 2025	3 days	4 th October 2025
4	SPMP (Software Project Management Plan)	5th October 2025	2 weeks	15th October 2025
4.1	Develop the project work plan and outline responsibilities	5 th October 2025	2 hours	5 th October 2025
4.2	Compose the SPMP document	6 th October 2025	5 days	10 th October 2025
4.3	Evaluate and refine the SPMP ensuring alignment with IEEE template	11 th October 2025	3 days	13 th October 2025
4.4	Finalize and submit the SPMP	14 th October 2025	2 days	15 th October 2025
5	Submit Mid-Semester Report	16th October 2025	2 hours	16th October 2025
6	Work on the Methodology	17th October 2025	1 week	23rd October 2025
6.1	Develop the project methodology	17 th October 2025	3 days	19 th October 2025
6.2	Review and refine project methodology	20 th October 2025	2 days	21 st October 2025
6.3	Submit the project methodology	22 nd October 2025	1 day	22 nd October 2025
7	Software Requirements Specification (SRS)	24th October 2025	~ 1.5 week	2nd November 2025
7.1	Identify and analyze functional and non-functional requirements	24 th October 2025	5 days	29 th October 2025
7.2	Finalize and document detailed system requirements	30 th October 2025	2 days	1 st November 2025
7.3	Submit the SRS	1 st November 2025	1 hour	2 nd November 2025
8	Software Design Specification (SDS)	3rd November 2025	~ 1.5 week	16 November 2025

8.1	Design system architecture and simulator workflow	3 rd October 2025	3 days	6 th November 2025
8.2	Create interface, use dataset, and AI model integration designs	7 th November 2025	4 days	11 th November 2025
8.3	Finalize and submit the SDS	12 th November 2025	4 days	16 th November 2025
9	Progress to Final Report	18th November 2025	1 month	19th December 2025
9.1	Compile and update all project chapters and WBS tables	18 th November 2025	1 week	25 th November 2025
9.2	Review and refine content based on supervisor feedback	25 th November 2025	1 week	2 nd December 2025
9.3	Prepare and submit the final version of the report	2 nd December 2025	1 week	9 th December 2025
9.4	Prepare and submit the final presentation slides	9 th December 2025	10 days	16 th December 2025
10	Implementing the System (Ru'ya Drone)	20th December 2025	20 weeks	8th April 2026
10.1	Preprocess the UAV dataset and perform data augmentation	20 th December 2025	2 weeks	3 rd January 2026
10.2	Train and fine-tune AI models for crowd detection and density estimation	3 rd January 2026	3 weeks	24 th January 2026
10.3	Evaluate and compare model performance metrics	25 th January 2026	1 week	31 st January 2026
10.4	Integrate trained models into the simulator environment for testing	1 st February 2026	3 weeks	21 st February 2026
10.5	Develop the database and backend for data storage	22 nd February 2026	3 weeks	11 th March 2026
10.6	Develop and test the user interface for real-time monitoring	12 th February 2026	3 weeks	4 th March 2026
10.7	Deploy the system and conduct integration testing on hardware	5 th March 2026	5 weeks	8 th April 2026
11	Formulate the Software Testing Plan (STP)	9th April 2026	2 weeks	22nd April 2026
11.1	Conduct unit and integration testing for AI model and database modules	9 th April 2026	5 days	13 th April 2026
11.2	Perform system-level testing using simulator and real UAV data	14 th April 2026	5 days	18 th April 2026
11.3	Finalize and submit the Software Testing Plan (STP) document	19 th April 2026	4 days	22 nd April 2026
12	Submit the Final Report	23rd April 2026	~3 weeks	17th May 2026

12.1	Compile and review the final report, ensuring all chapters are updated	23 rd April 2026	1 week	30 th April 2026
12.2	Develop the technical manual for system setup, testing, and demonstration	30 th April 2026	1 week	7 th May 2026
12.3	Prepare and finalize the presentation slides	7 th May 2026	1 week	14 th May 2026
12.4	Submit the final report and presentation	14 th May 2026	3 days	17 th May 2026

The Gantt chart presents the overall timeline and sequence of tasks for the Ru'ya Project. It starts with initiation and literature review in August 2025, followed by SPMP preparation and methodology development in September and October. The mid-semester milestone occurs in October, with subsequent phases: SRS, SDS, and progress reporting running through November. Core implementation spans December 2025 to March 2026, leading into testing and final report preparation in April, concluding with the final submission milestone. This schedule ensures clear workflow, efficient resource use, and well-defined milestones for tracking progress.

Ru'ya Project Gantt Chart (2025–2026)

Task	Aug	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr
Project Initiation									
Literature Review									
SPMP									
Mid-term report									
Methodology									
SRS									
SDS									
Progress Report									
Implementation									
Testing Plan									
Final Report									
Submission									

Figure 16. Ru'ya Project Gantt Chart (2025–2026)

Legend:

The Gantt chart color codes represent project progress as follows:

- **Green:** Completed tasks
- **Blue:** Tasks currently in progress
- **Gray:** Pending tasks
- **Diamond symbol:** Project milestones

3.2.3 Resource Allocation

The *Ru'ya Project* requires both hardware and software resources to support development and testing. Table 23 lists all essential components, including computing tools, UAV hardware, and reference materials used throughout the project.

Table 23. Hardware, Software, and Written Resources for the *Ru'ya Project*

Category	Resources
Hardware	- Personal Computer - Mobile Phone - Jetson Nano microcontroller - Pixhawk flight controller - UAV Drone (Ru'ya Drone) - Onboard Camera (for image and video capture) - Power supply and drone battery
Software	- Python - NoSQL Developer - Draw.io - Drone Simulator - Microsoft Office (Word, PowerPoint, Excel, Teams) - Zoom - OneDrive - TeamGantt
Written Resources	- research papers on crowd detection, drone technology, and AI-based surveillance systems. - Mendeley reference manager - Technical documentation and manuals - Grammarly

3.2.4 Budget Allocation

The *Ru'ya Project* budget includes estimated computing resources, software tools, and hardware components required for drone development and testing. Table 24 provides an overview of the expected expenses for each major project activity.

Table 24. Budget Allocation table

Category	Description	Estimated Cost
Computing Resources	High-performance PC for AI model training and testing	ﷲ 3000
Software Tools	Python (open-source), Draw.io, NoSQL Developer (free), Microsoft Office (free university license)	0
Hardware (Drone Components)	Jetson Nano, Pixhawk, Camera, Drone Frame, Battery, Cables	ﷲ 3000
Testing & Simulation Facilities	UAV Simulator environment and field testing setup	ﷲ 500
Administrative Support	Printing, documentation, and presentation materials	ﷲ 200
Travel & Meetings	In-town car rides for project meetings and drone hardware components	ﷲ 300
Total Estimated Cost	≈ 7000 Saudi Riyals	

3.3 Project Tracking Plan

The next section describes the project's requirements management, schedule control, quality control, reporting, and project metrics strategies.

3.3.1 Requirements Management

To maintain project consistency and quality, all requirements in the *Ru'ya Project* are clearly defined, documented, and prioritized. Any proposed modifications are carefully reviewed to evaluate their effect on the project's scope, schedule, and technical design. Significant changes will only be applied after discussion and approval from both team members and the supervisor, ensuring alignment with project objectives. Minor adjustments that do not affect major milestones may be implemented based on available time and resources.

3.3.2 Schedule Control

The *Ru'ya Project* schedule is monitored regularly to ensure steady progress in line with the planned timeline outlined in Table 21. Progress at both major and minor milestones is tracked using the project Gantt chart, allowing comparison between planned and actual completion dates. Regular team discussions, face-to-face meetings, and weekly check-ins help review progress and identify any deviations early.

In case of delays, corrective actions such as rescheduling tasks or reallocating responsibilities are applied to maintain alignment with the plan. Tools like TeamGantt and Microsoft Excel are used to measure and visualize progress, while milestone achievements are assessed based on timeliness, completeness, and quality of deliverables.

3.3.4 Quality Control

Quality of the project will be assessed in several fundamental areas:

- Compliance with all stated project specifications and objectives.
- Work progress and deliverables check via review meetings and approval by supervisors.
- Verification that all documents are complete, accurate, and consistent.
- The capability to detect and correct any errors or inconsistencies early on.
- Maintaining clear records for every stage of the project to ensure traceability and reliability.

3.3.5 Reporting

The project reporting process will allow for communication and tracking of progress. The main reporting activity is:

- **Weekly Meetings:** Conducted either in person or through Zoom to review next steps, problems, and progress.
- **Weekly Reports:** Summary of findings from each meeting and delivered to the supervisor.
- **Advisor Discussions:** Weekly meetings with the supervisor to review the project milestones and obtain advice.
- **Documentation:** Reports and minutes of meetings are stored in a shared folder and are easily accessible to be opened and consulted by all members of the team.

Having standardized report structure keeps the team and the supervisor aware of the schedule, activities, and quality of the project.

3.3.6 Project Metrics

Table 25 lists the project metrics that will be used to evaluate progress and ensure the project remains aligned with its objectives.

Table 25. Project Metrics

Metric	Description	Frequency
Timeframe	The progress of the project is measured against the planned schedule to ensure timely completion of tasks.	Ongoing
Requirement Fulfilment	The development of the project is checked regularly to ensure all requirements are being addressed.	Continuous
Change Tracking	Any requested changes are recorded and evaluated to ensure alignment with the project scope.	After each major task
Quality Review	Any identified errors or inconsistencies are corrected to maintain the quality of documents and work products.	After each review
Accuracy and Performance Indicators	Measures such as accuracy, F1 score, ROC, and latency will later be used in the implementation phase to assess the AI model's performance.	Implementation phase

3.4 Risk Management Plan

A risk management plan is implemented to identify potential difficulties that may affect the progress or quality of the project. Each risk is assessed in terms of likelihood and impact, with predetermined mitigation actions to reduce or eliminate negative consequences. Table 26 illustrates the potential risk throughout the project lifecycle.

Table 26. Risk Management Plan

No.	Potential Risk	Probability	Impact	Action	Prevention
1	Insufficient or biased dataset	Medium	Medium–High: Could reduce model accuracy and generalization	Retrain model with corrected/augmented dataset; validate with cross-validation	Collect diverse datasets; perform data augmentation; monitor dataset quality
2	Overfitting of AI model	Medium	High: Model performs poorly on new data	Adjust hyperparameters; increase training data	Use regularization, dropout, early stopping; cross-validation
3	Hardware/GPU failure or limited compute resources	Low–Medium	Medium: Delays training and experiments	Use cloud GPU/TPU services; reschedule experiments	Schedule GPU usage in advance; maintain backups
4	Model bias leading to unfair predictions	Low–Medium	High: Ethical and regulatory concerns	Retrain model with bias mitigation techniques	Monitor fairness metrics; include diverse features
5	Security/data privacy breaches	Low	High: Legal/regulatory consequences	Immediate investigation; notify stakeholders; patch vulnerabilities	Anonymize sensitive data; follow security protocols
6	Team member turnover or skill gaps	Medium	Medium: Could delay milestones	Reassign tasks; provide training; adjust timeline	Cross-train team; document processes
7	Delays in schedule due to unexpected technical issues	Medium	Medium: Potential impact on project delivery	Reschedule tasks; reprioritize features	Buffer in schedule; milestone tracking
8	Incompatibility with deployment environment	Low–Medium	Medium: Could prevent final system deployment	Adjust codebase; update environment	Test deployment early; use containerization (Docker)
9	Stakeholder requirement changes	Medium	Medium–High: Could impact scope and deliverables	Update requirements; adjust project plan	Regular communication with stakeholders; requirement review
10	Failure to achieve performance targets (accuracy, latency)	Medium	High: Could reduce project success and acceptance	Optimize model; tune hyperparameters	Benchmark early; continuous evaluation

3.5 Project Closeout Plan

Closeout plan for the project entails key steps to enable proper closure and documentation of all work:

- Assigning team members to other functions or tasks upon project closure.
- Archiving all datasets, source code, trained models, experiment logs, and documentation.
- Documenting project metrics such as model performance, use of resources, and schedule compliance.
- Post-mortem debriefings to document lessons learned and problems faced.
- Developing a final report summarizing targets achieved, project results, and recommendations for future studies.

4. Technical Process Plans

4.1 Process Model

The project follows the Waterfall Process Model, in which the development is divided into consecutive stages. Each stage is completed and tested before proceeding to the next stage, and each stage's output serves as the foundation for the subsequent stage. The model aligns with the two-semester project structure, wherein analysis and design occur during the first semester, and implementation and evaluation during the second semester. The stages of the project process model are shown in Figure 17.



Figure 17. The Waterfall Model

Table 27. The Waterfall Phases

Phase	Goal / Activities	Deliverable
Requirements Analysis and Definition	• Gather and analyze system requirements. • Define system scope and constraints. • Specify functional and non-functional requirements.	Deliver the Software Requirements Specification (SRS) document. 30 th October 2025
System and Software Design	• Design system architecture and data flow. • Develop interface layouts and module interactions.	Deliver the Software Design Specification (SDS) document. 27 th November 2025

	• Finalize AI model and UAV system design.	
Implementation and Unit Testing	• Implement the system components and train AI models. • Develop and integrate the UAV simulation environment. • Conduct unit testing for individual modules.	Deliver the Initial Implementation and Unit Test Results . 8 th April 2026
Integration and System Testing	• Integrate all modules and perform full system testing. • Validate system accuracy, latency, and reliability in simulation.	Deliver the System Integration and Testing Report . 22 nd April 2026
Operation and Maintenance	• Finalize documentation and prepare for project submission. • Maintenance phase is beyond current project scope.	Deliver the Final Project Report and Presentation. 30 th April 2026

4.2 Methods, Tools, and Techniques

The project uses selected tools and frameworks across each development phase to ensure accurate model design, testing, and deployment. Cloud platforms, ML frameworks, and collaboration tools support the complete workflow from planning to implementation. Table 28 summarizes the methods, tools, and techniques used in each phase.

Table 28. Methods, Tools, and Techniques Used in the Ru'ya Project

Phase	Tools
Planning	○ Microsoft Teams ○ Microsoft Word ○ Microsoft Excel ○ WhatsApp ○ Zoom
Requirement analysis	○ Microsoft Word
System design	○ Microsoft Word ○ Draw io ○ Simulators
Implementation	○ Python (Pytorch & TensorFlow) ○ Google Collab and Anaconda ○ AWS/GCP for cloud training

4.3 Infrastructure

The project relies on both local and cloud-based infrastructure to support training, testing, and deployment. Hardware, software, and networking resources are organized to ensure efficiency and scalability. Table 29 outlines the main infrastructure components used in the project.

Table 29. Infrastructure Components of the Ru'ya Project

Hardware	○ Personal laptop ○ UAV Drone ○ Mobile phone
Software	○ Python (TensorFlow, PyTorch, OpenCV). ○ Google collab. ○ Draw.io. ○ Drone simulator.
Operating System	○ Any.
Network	○ Cloud Services. ○ High speed network.
Facilities	○ University.

4.4 Product Acceptance

The deliverables of projects shall be accepted within testing, analysis and demonstration to ensure that the deliverables match the defined performance and quality requirements. Objective requirements include achieving the necessary detection accuracy, ensuring the speed of real-time processing, and adhering to safety and privacy standards applicable to drones. The supervisor and Co-supervisor, Dr. Rabab Alkhalifa and Ms. Mehwash Farooqui will verify a formal agreement on the acceptance criteria. The final approval will occur after all deliverables have passed through the technical reviews and satisfy all the requirements.

5. Supporting Process Plans

5.1 Documentation

Throughout the project's timeline, various documents will be prepared and submitted at specific stages. The entire team will collaborate on developing, reviewing, and finalizing these documents. Table 30 lists the documents to be produced during the project.

Table 30. Project Documentation Plan for the Ru'ya Project

Document	Format Standard	Prepare Document	Review Document
Introduction and Literature review	Materials provided by the supervisor.	All team members.	Project supervisor Dr.Rabab Alkhalifa.
Mid semester report	Materials provided by the supervisor.		
SPMP	Reference to IEEE Standard 1058-1998		
SRS	Reference to IEEE Standard 1058-1998		
Methodology	Materials provided by the supervisor.		
SDS	Reference to IEEE Standard 1058-1998		
First Report	Materials provided by the supervisor.		
Software Test Plan	Reference to IEEE Standard 1058-1998		
User Manual	Materials provided by the supervisor.		
Final Project Source Code and Report	Materials provided by the supervisor.		

6. Additional Plans

The project will ensure that the development and implementation of AI take place within the framework of established ethical standards and regulations. Among ethical AI practices, the protection of data privacy, fairness, and transparency will be considered important factors. Furthermore, the project will adhere to data protection laws, such as the GDPR, to ensure that information is handled safely and responsibly.

Chapter 4: Software Requirements Specification

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to define and explain all functional and non-functional requirements for the **Ru'ya Drone Project**, which's an intelligent Unmanned Aerial Vehicle (UAV) system that utilizes Artificial Intelligence (AI) and Computer Vision for real-time crowd detection and density estimation. This document serves as a reference for developers and evaluators to ensure a shared understanding of the system's objectives, behavior, and performance expectations throughout the software development lifecycle.

1.2 Scope

The Ru'ya system is designed to enhance crowd monitoring and safety management by integrating UAV technology with advanced AI-based image analysis. The drone captures aerial footage of large gatherings, processes it using deep learning models, and provides real-time insights such as crowd size, density, and movement patterns. The system includes three main components:

1. **Drone Module**
responsible for video capture and data transmission.
2. **AI Processing Unit**
handles image preprocessing, detection, and density estimation using trained neural networks.
3. **User Dashboard**
displays analytical results, visual alerts, and historical records.

The system aims to assist security personnel, event organizers, and emergency teams in decision-making and crowd management operations.

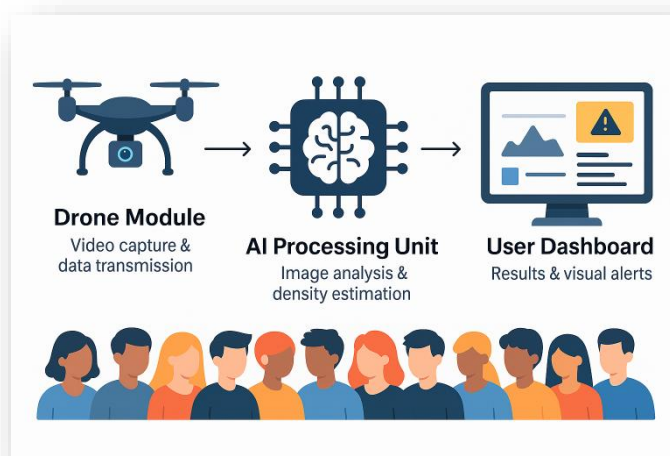


Figure 18. The Ru'ya System

1.3 Definitions, Acronyms, and Abbreviations

This section explains the main terms and abbreviations used in this document. It helps readers understand the technical words that appear throughout the Ru'ya system description.

Table 31. Key Terms and Abbreviations Used in the Ru'ya System

<i>Term / Acronym</i>	<i>Definition</i>
<i>AI</i>	Artificial Intelligence, the technology used to enable the UAV to perform real-time crowd detection and density estimation.
<i>UAV</i>	Unmanned Aerial Vehicle, a drone that captures aerial footage and sends it for analysis.
<i>Quadcopter</i>	A type of UAV that uses four rotors for lift and movement, providing stable and precise flight control.
<i>Ru'ya</i>	The proposed system that integrates AI and UAV technologies to detect and monitor crowds in real time.
<i>GUI</i>	Graphical User Interface, the visual component that allows users to interact with the system through dashboards and control panels.
<i>GPS</i>	Global Positioning System, used by the drone to determine its exact location and maintain flight stability.
<i>RTL</i>	Return to Home, a safety feature that lets the drone automatically return to its takeoff point.
<i>Dashboard</i>	The main user interface that displays analytics, alerts, and mission data from the UAV.
<i>SDK</i>	Software Development Kit, a set of tools and libraries that allow developers to build, customize, or integrate new features into the UAV system.
<i>AES-256</i>	Advanced Encryption Standard (256-bit), a strong encryption method used to secure communication and protect data shared between the UAV and dashboard.
<i>Telemetry</i>	The automatic process of collecting and transmitting data from the UAV to the ground system, such as altitude, speed, battery level, and GPS status.
<i>Dataset</i>	A collection of labeled images used to train and test the AI crowd detection model.
<i>Crowd Density</i>	The number of people detected within a specific area captured by the UAV's camera.
<i>Vision 2030</i>	The Kingdom of Saudi Arabia's national development plan emphasizing innovation, safety, and smart technologies.

2. Overall description

2.1 Product perspective

2.1.1 Context and Origin

The Ru'ya project is developed as an AI engineering senior project, aiming to ensure public safety, and crowd management in mass gatherings including Hajj, and FIFA 2034 in alignment with the Saudi vision 2030. Our motivation behind this project arose from analysing the weaknesses of a fixed CCTV systems such as, its limited coverage, having blind spots, and lacking in real time analytics abilities. Our project takes the idea of monitoring crowd to the next level. Ru'ya presents a new, self-contained product that combines UAV mobility with onboard deep learning to provide designated authorities with crowd density approximation, and real time aerial detection and monitoring.

2.1.2 System Scope

The Ru'ya system is a standalone product, not a part of an existing larger system. However, we will design the system to be flexible, enabling integration with external cloud platforms or remote dashboards, if needed in future real-world deployment. The system incorporates an AI Module in a UAV platform, and an integrated dashboard for visualization of crowd estimations, and analytics. The drone captures and analyses visual data, then stream the results to the dashboard. Figure 19 shows the major components of the overall system, and future integration capabilities.

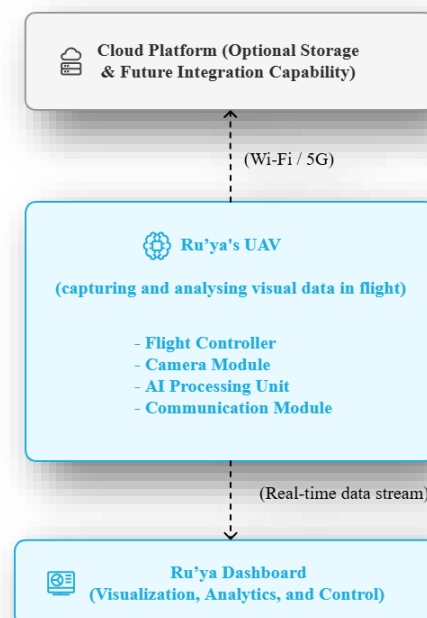


Figure 19. High-Level Architecture of the Ru'ya System

2. Product functions

The Ru'ya system will provide an AI-based crowd detection using the camera module integrated into the UAV platform and stream the captured visual data to the monitoring dashboard. As illustrated in Figure 2 shown in Section 2.1.2. The following are the primary functions of the Ru'ya system:

1. Flight Control

Enables autonomous or semi-autonomous UAV operations.

2. Aerial Data Acquisition

Captures visual data including live videos and images using the camera module.

3. AI-based Crowd Detection

Preprocesses and analyses the captured aerial data and provide a real-time estimate of the crowd gathering.

4. Data Transmission

Transmits the AI module results to the dashboard for user-friendly visualization.

5. Dashboard Analytics

Display visualization and analytics of the crowd estimate and provide alerts and warnings if a certain threshold is exceeded.

6. Data Storage

Archives previous results, and flight logs for review and system evaluation.

3. Specific requirements

3.1 External interface requirements

This section explains how users will interact with the Ru'ya system. It describes the main components of the user interface and how users can operate, monitor, and review the UAV system. Each interface is designed to be simple, clear, and responsive, especially during real-time operations.

3.1.1 User interfaces

The Ru'ya UAV Crowd Detection and Monitoring System includes a responsive, web-based GUI for authorized administrators and operators. The interface allows users to monitor real-time crowd density, manage UAV missions, configure alert thresholds, and analyze historical performance data through an intuitive, visually consistent dashboard.

Main Interface Components

1. Login Page

Provides secure authentication for administrators and operators through username and password fields. Also, ensures password visibility toggle and field validation before access.

2. Dashboard (Home)

- ✓ Central monitoring hub showing mission summary cards: Crowd Density, Total People, Alert Status, and Flight Status.
- ✓ Displays a live video feed from the UAV camera with AI-generated detection overlays (bounding boxes and percentage labels).
- ✓ Includes a Flight Path & Location panel for live UAV GPS tracking and mission time.
- ✓ Provides a real-time status bar showing connection, battery level, GPS signal, and timestamp.

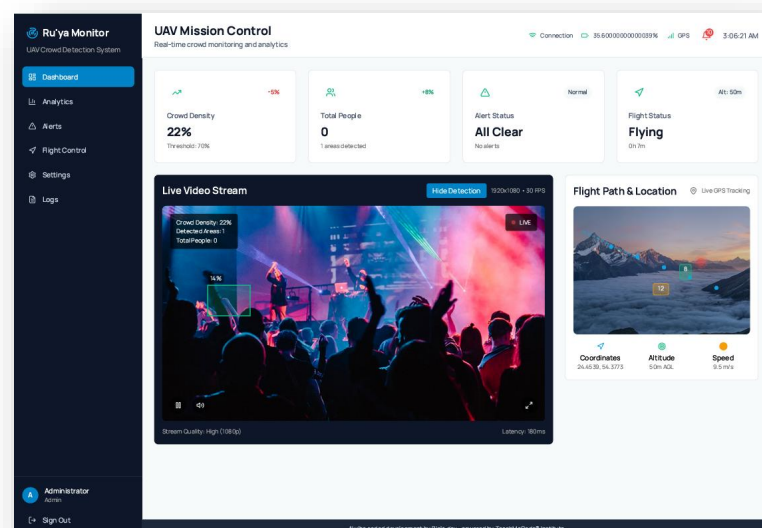


Figure 20. Dashboard Interface

3. Analytics Panel

- ✓ Visualizes historical and real-time data through charts and graphs.
- ✓ Displays Peak Density, Average Density, Total Detections, and Flight Hours statistics.
- ✓ Includes line and bar charts for Crowd Density Over Time and Hourly Detections, plus pie charts for Density Distribution.
- ✓ Supports data export for reporting and research analysis.
- ✓ Lists Recent Mission Summaries with mission ID, date, duration, maximum density, and status.

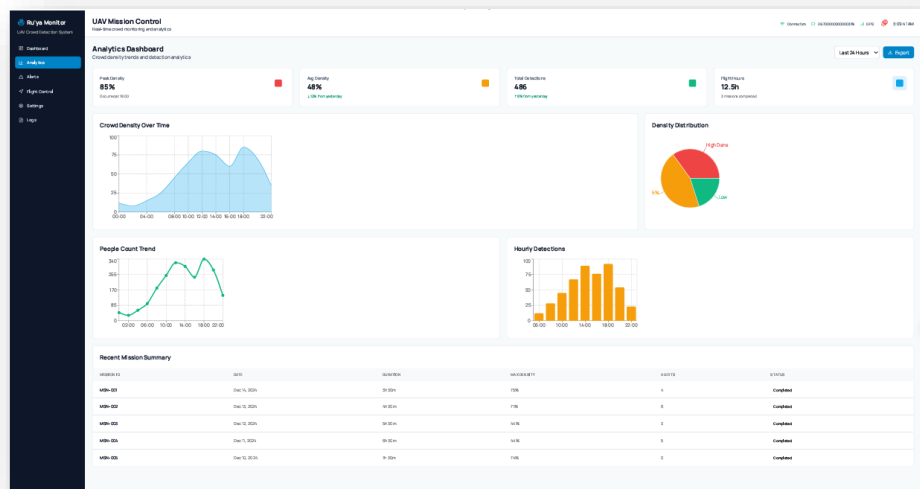


Figure 21. Analytics Panel

4. Alert Management Panel

- ✓ Displays a centralized view of all crowd-density alerts and system warnings.
- ✓ Shows total, unacknowledged, and high-priority alerts with color-coded icons.
- ✓ Allows searching, filtering, and acknowledgment of individual alerts.
- ✓ Provides real-time alert notifications whenever thresholds are exceeded.

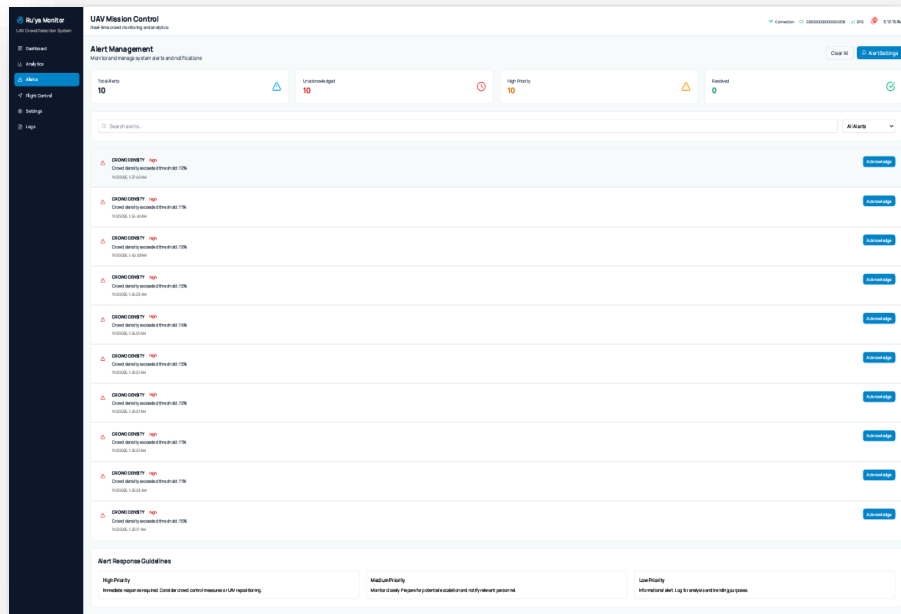


Figure 22. Alert Management Panel

5. Flight Control Center

- ✓ Enables mission monitoring and emergency actions.
- ✓ Displays key telemetry metrics: flight time, altitude, speed, and battery level.
- ✓ Provides control buttons for End Mission and Emergency Land.
- ✓ Integrates a Flight Path Planning map showing waypoints and UAV trajectory.
- ✓ Includes an Emergency Procedures section: Return to Home, Hold Position, and Manual Control.

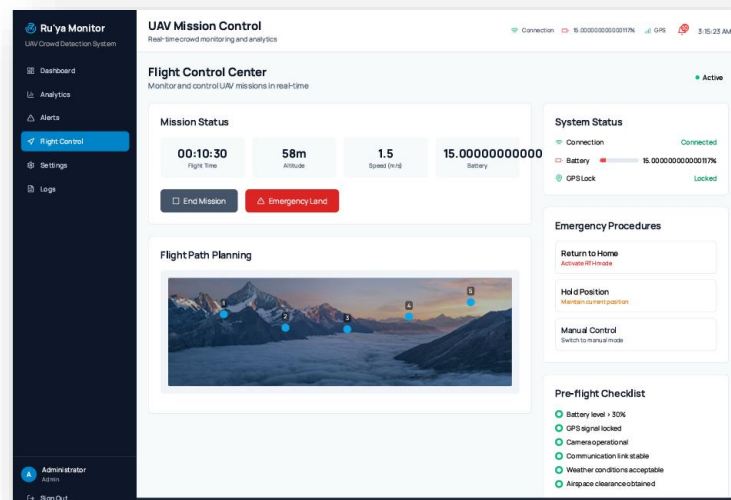


Figure 23. Flight Control Center

6. System Settings Page

- ✓ Provides access to configurable parameters such as Crowd Density Threshold, Video Quality, and Auto-record missions.
- ✓ Includes Alert Settings (frequency, e-mail, and SMS notifications) and Security Settings (data encryption and log retention).
- ✓ Displays System Performance metrics: CPU usage and Memory usage with progress bars.
- ✓ Allow saving configurations securely via the Save Settings button.

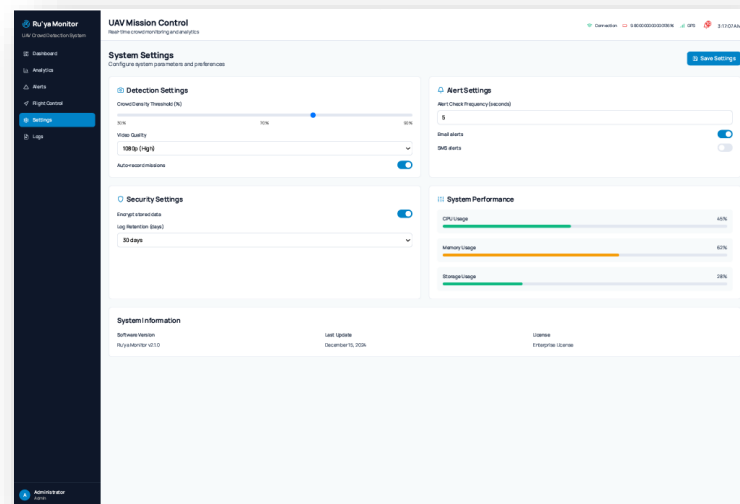


Figure 24. System Settings Page

7. Logs Page

- ✓ Lists all mission and system events, classified as Info, Warning, or Error.
- ✓ Provides filtering and export functionality through Advanced Filters and Export Logs options.
- ✓ Enables administrators to trace historical activities for maintenance and audit purposes.

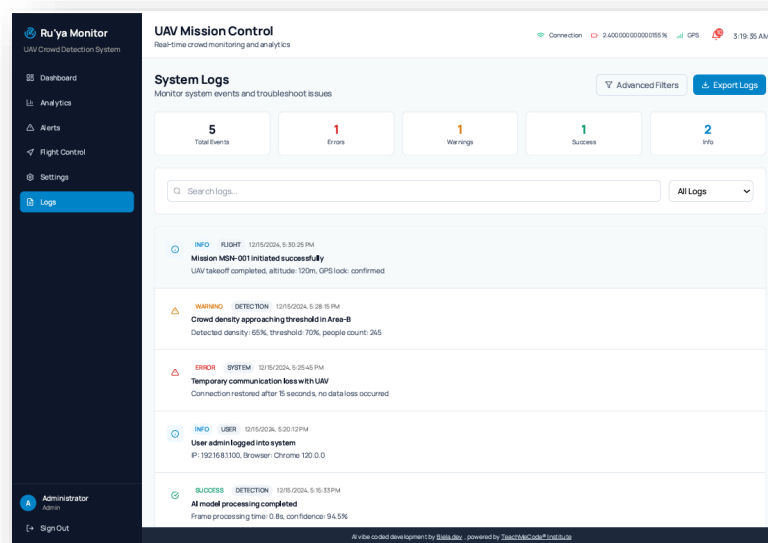


Figure 25. Logs Page

User Interface Requirements

- The GUI must be responsive, maintaining readability and usability on all supported screen sizes.
- New UAV or AI data updates must appear on the dashboard within 5 seconds of receipt.
- Alerts must be displayed in real time and persist until acknowledged by the user.

User Roles

Table 32. User Roles

<i>Role</i>	<i>Description</i>	<i>Accessible Modules</i>
Administrator	Manages users, system configuration, AI thresholds, and performance monitoring.	Full access to all modules.
Operator	Monitors live feed, responds to alerts, and manages flight control operations.	Dashboard, Analytics, Alerts, Flight Control.

Together, these interfaces make the Ru'ya system easy to use and understand, helping users control the UAV, follow real-time updates, and manage mission data effectively.

3.1.2 Hardware Interfaces

The Ru'ya UAV-Based Crowd Detection and Monitoring System integrates several hardware components that enable data collection, flight control, and communication with the monitoring dashboard. These interfaces ensure efficient coordination between the UAV and the ground station.

3.1.2.1 UAV Platform

The system uses a quadcopter UAV equipped with an onboard computer and sensors for navigation and image capture. It communicates wirelessly with the monitoring dashboard through Wi-Fi.

The main components include:

- **Camera Module:** captures aerial video for AI analysis.
- **Flight Controller:** manages flight operations and transmits telemetry data (altitude, GPS, battery).
- **GPS:** provide location and stability data.
- **Battery System:** powers all onboard components and reports status in real time.

Ground Station:

- A laptop or tablet acts as the ground station running the Ru'ya web dashboard.
- Connects to the UAV via a secure wireless link to receive video, telemetry, and alerts.
- Enables operators to control missions and store flight data locally.

Sensors:

- **Camera Sensor:** provides real-time video input for AI detection.
- **Distance and Altitude Sensors:** enhance navigation and obstacle avoidance

3.1.3 Software Interfaces

The Ru'ya system is built on interconnected software components that manage UAV control, AI detection, and data visualization. These modules communicate through defined APIs to ensure smooth coordination between the UAV and the web dashboard.

3.1.3.1 UAV Firmware and Control Software

The UAV operates using firmware that supports autonomous flight, waypoint navigation, and telemetry reporting. For the dashboard, it communicates with the UAV through standard SDKs to send commands such as Launch, Hover, Land, and Return Home.

3.1.3.2 AI Crowd Detection Module

The AI module performs real-time crowd detection and density estimation using aerial images from the UAV camera. It is implemented using lightweight AI frameworks such as TensorFlow Lite or PyTorch Mobile. The module outputs crowd counts, density levels, and detection overlays, which are transmitted to the dashboard via RESTful APIs for visualization.

3.1.3.3 Web Dashboard Software

The web dashboard allows users to monitor live feeds, view analytics, and control UAV missions. It uses a modern web framework for responsive design and secure communication with the UAV and AI modules. User authentication is handled through token-based login, and all mission data and alerts are stored in a cloud database for later review.

3.1.3.4 Operating Environment

The system runs on standard web browsers (Chrome, Edge, Safari). Supported operating systems include Windows 10+, macOS, and Android tablets.

3.1.4 Communication Interfaces

The Ru'ya system relies on reliable, low-latency wireless communication for UAV operation, AI inference updates, and dashboard synchronization.

3.1.4.1 UAV–Dashboard Link

- ✓ Communication between UAV and dashboard uses TCP/IP over Wi-Fi 5 or 4G/5G connection.
- ✓ Telemetry, camera feed, and control signals are exchanged through secure channels using HTTPS and WebSocket protocols.

3.1.4.2 Cloud and Database Synchronization

All mission logs, detection results, and alert data are synced with a cloud database in near real-time.

- ✓ Sync frequency: Every 5 seconds during active missions.
- ✓ Backups occur automatically once every 24 hours.

Security Protocols

- ✓ Communication channels are encrypted with AES-256 standards.
- ✓ Two-factor authentication (2FA) optional for administrative access.
- ✓ Session tokens expire after 30 minutes of inactivity.

4. System Features

4.1 Flight Control

Description and Priority:

Manages UAV flight stability and navigation during monitoring missions. Supports both autonomous and manual control modes.

Priority: High

Stimulus/Response Sequences:

- **Stimulus:** The operator initiates a flight mission from the dashboard.
- **Response:** The UAV activates its flight systems, takes off, and follows a predefined or user-specified path.

Functional Requirements:

- **REQ-1:** The system shall allow authorized users to initiate and terminate UAV missions from the dashboard.
- **REQ-2:** The UAV shall autonomously maintain altitude and stability during flight.
- **REQ-3:** The UAV shall respond to manual control override commands in real time.
- **REQ-4:** The UAV shall record GPS coordinates throughout the flight.

4.2 Aerial Data Acquisition

Description and Priority:

Captures visual data from the UAV camera for further processing by the AI module.

Priority: High

Stimulus/Response Sequences:

- **Stimulus:** The UAV is in flight, and the camera is activated.
- **Response:** The camera continuously captures video frames and forwards them to the AI module.

Functional Requirements:

- **REQ-5:** The system shall capture real-time video at a minimum resolution of 720p.
- **REQ-6:** The camera module shall automatically adjust exposure and focus.
- **REQ-7:** The system shall timestamp each captured frame.

4.3 AI-Based Crowd Detection

Description and Priority:

Performs real-time processing of aerial images to estimate crowd density using onboard deep learning models.

Priority: High

Stimulus/Response Sequences:

- **Stimulus:** The AI module receives frames from the camera.
- **Response:** The module processes the frames and outputs estimated crowd density levels.

Functional Requirements:

- **REQ-8:** The AI module shall process each incoming frame within 1 second.
- **REQ-9:** The system shall identify crowded areas using bounding boxes or heatmaps.
- **REQ-10:** The AI module shall trigger an alert if the estimated density exceeds the configured threshold.

4.4 Data Transmission

Description and Priority:

Handles the wireless transfer of processed results and telemetry data to the monitoring dashboard.

Priority: Medium

Stimulus/Response Sequences:

- **Stimulus:** The AI module generates detection output.
- **Response:** The data transmission unit sends the results and telemetry data to the dashboard.

Functional Requirements:

- **REQ-11:** The system shall transmit processed results with latency not exceeding 2 seconds.
- **REQ-12:** The communication module shall retry transmission if the connection is lost.
- **REQ-13:** The data shall be encrypted during transmission.

4.5 Dashboard Analytics

Description and Priority:

Visualizes incoming UAV data, analytics results, and alerts through a user-friendly interface.

Priority: High

Stimulus/Response Sequences:

- **Stimulus:** The dashboard receives a data stream from the UAV.
- **Response:** Visual elements update in real time to display crowd density, alerts, and the flight path.

Functional Requirements:

- **REQ-14:** The dashboard shall display live video with detection overlays.
- **REQ-15:** The system shall visualize density metrics using graphs and heatmaps.
- **REQ-16:** The dashboard shall issue visual and audible alerts when crowd density exceeds a defined threshold.

4.6 Data Storage

Description and Priority:

Stores mission data, analytics results, and UAV logs for post-analysis and evaluation.

Priority: Medium

Stimulus/Response Sequences:

- **Stimulus:** The mission is completed, or the system triggers a data-saving event.
- **Response:** Results are archived within the data management subsystem.

Functional Requirements:

- **REQ-17:** The system shall store detection results, timestamps, and UAV telemetry after each mission.
- **REQ-18:** Users shall be able to retrieve and review past mission data through the dashboard.
- **REQ-19:** Stored data shall be protected against unauthorized access or modification.

5.Non-Functional Requirements

➤ Performance Requirements

- The system shall process each video frame within 1 second to support real-time crowd detection.
- The dashboard shall display updated analytics and alerts within 5 seconds of data reception.
- The system shall support continuous UAV operation for a full mission without performance degradation.

➤ Reliability and Availability

- The system shall maintain operational availability during active UAV missions.
- The UAV shall automatically trigger a safe fallback action in case of communication failure.
- All system errors and mission events shall be logged for post-mission analysis.

➤ **Security Requirements**

- The system shall encrypt all transmitted data using AES-256 encryption.
- Only authenticated users shall be allowed to access the dashboard.
- Administrative users shall be able to enable two-factor authentication.
- User sessions shall expire after 30 minutes of inactivity.

➤ **Usability Requirements**

- The dashboard shall be web-based and accessible through modern browsers.
- The user interface shall display alerts using clear visual indicators.
- The system shall be operable by trained users with minimal instruction.

➤ **Scalability Requirements**

- The system shall support future integration of multiple UAVs.
- The AI module shall allow retraining or replacement without major system redesign.
- Stored mission data shall scale to support long-term analysis.

➤ **Maintainability**

- The system shall follow a modular architecture separating UAV control, AI processing, and visualization.
- Software updates shall not affect previously stored mission data.
- System logs shall support debugging and maintenance activities.

➤ **Portability**

- The dashboard shall be compatible with Chrome, Edge, and Safari browsers.
- The system shall operate on Windows, macOS, and Android tablets.
- The AI module shall use lightweight frameworks suitable for edge devices.

➤ **Safety and Compliance**

- The UAV shall continuously monitor battery level and GPS status.
- Emergency control actions shall be available at all times.
- The system shall comply with GACA regulations and data protection guidelines.

Chapter 5: Software Design Specifications

1. Introduction

The Ru'ya system is an AI-driven UAV-based crowd detection and tracking system that will facilitate safety management of high-density events with large crowds like Hajj, concerts, and sports events. The system allows real-time crowd density estimation, alerts, and live visualization via an interactive dashboard by integrating drones with lightweight AI models.

1.1 Purpose

This document outlines the software design of the Ru'ya Lightweight AI Model for a UAV-based Crowd Detection and Monitoring System. It explains the design of all major system components, including UAV operation, AI-based crowd detection, data transmission, and the web dashboard, and how they interact to deliver real-time monitoring and analytics. This document is a guideline to developers, testers, and assessors to make sure that the system is developed and constructed based on the requirements as outlined in the SRS.

1.2 Scope

The Ru'ya system is a web-based monitoring solution integrated with a UAV platform that uses AI-based crowd detection to provide real-time monitoring and analytics.

The scope of this document includes:

- The detailed design of the UAV control and communication modules.
- The AI detection module for real-time processing and analysis of aerial data.
- The web-based dashboard for live visualization, analytics, and system management.
- The data storage subsystem for mission archiving and retrieval.

This SDS will be used by developers to implement the system modules and maintain design consistency, by testers to create and execute test cases based on the design specifications, and by project supervisors and evaluators to verify that the software design aligns with the objectives and requirements defined in the SRS.

1.3 Definitions, Acronyms, and Abbreviations

This section explains the main terms and abbreviations used in this document. It helps readers understand the technical words that appear throughout the Ru'ya system description.

Table 33. Definitions, Acronyms, and Abbreviations

<i>Term / Acronym</i>	<i>Definition</i>
<i>UAV</i>	Unmanned Aerial Vehicle used for aerial imaging and data collection.
<i>AI</i>	Artificial Intelligence; algorithms used for real-time crowd detection and analysis.
<i>YOLO</i>	“You Only Look Once,” a real-time object detection model used for identifying people in video frames.
<i>GPS</i>	Global Positioning System; provides UAV location, altitude, and path data.
<i>DFD</i>	Data Flow Diagram representing system processes and data movement.
<i>SDK</i>	Software Development Kit enabling communication with UAV hardware.
<i>REST API</i>	Representational State Transfer API; used for backend communication between UAV, server, and dashboard.
<i>FPS</i>	Frames Per Second; rate at which the UAV camera captures video.
<i>CPU</i>	Central Processing Unit; processes system tasks for backend and dashboard.
<i>GPU</i>	Graphics Processing Unit; used for accelerated AI model inference.
<i>JSON</i>	JavaScript Object Notation; format used to transmit detection and telemetry data.
<i>Telemetry</i>	Real-time UAV data such as altitude, battery level, GPS coordinates, and speed.
<i>Crowd Density</i>	Percentage estimate indicating how crowded a monitored area is.
<i>Inference</i>	The process of the AI model generating predictions (detections, counts, density) from input frames.
<i>WebSocket</i>	Communication protocol used for real-time dashboard updates.
<i>Local Database</i>	Storage system that saves mission logs, detection outputs, timestamps, and analytics.
<i>Threshold</i>	Predefined limit for crowd density that triggers alerts.
<i>Operator</i>	User responsible for flight monitoring and live mission operation.
<i>Administrator</i>	User with elevated privileges to change settings, manage users, and access full logs.
<i>Mission Log</i>	Stored data containing detections, flight path, timestamps, and alerts for each UAV mission.

2. System overview

2.1 General description

Ru'ya UAV Crowd Detection and Monitoring System is a smart, AI-driven platform that helps authorities and event organizers handle crowds by providing real-time crowd density estimation, alert management, and flight control through an integrated UAV and dashboard system.

This system combines UAV technology with computer vision models to detect and estimate crowd density from aerial video streams. The processed information is visually presented in a web-based dashboard that displays analytics, flight data, and system alerts to authorized users.

Ru'ya was developed to enhance situational awareness, public safety, and decision-making efficiency for any high-density event like the Hajj, concerts, or sports events. It also aligns with the Saudi Vision 2030 goals for innovation and smart city infrastructure.

2.2 System Functionality

The Ru'ya system provides the following key functionalities at a high level:

- 1. Aerial Data Capture:**

A camera mounted on the UAV captures live video of the area it is monitoring.

- 2. AI-Based Crowd Detection:**

Real-time estimation of crowd size, density, and distribution is done through the onboard AI model; the AI uses lightweight deep learning algorithms for real-time video analysis.

- 3. Data Transmission:**

The processed results are securely transmitted to the dashboard via encrypted wireless communication protocols.

- 4. Dashboard Visualization:**

The web dashboard provides mission analytics, live UAV telemetry, and real-time alerts through an intuitive graphical interface.

- 5. Alert Management:**

Alerts on crowd density beyond pre-set thresholds and UAV system anomalies are generated automatically by the system.

- 6. Flight Control:**

It enables operators to monitor flight status, track UAV position, and execute emergency actions like Return to Home or Emergency Land.

- 7. System Configuration and Logs:**

Administrators can set thresholds, manage user access, and view mission logs for the purpose of performance assessment and auditing.

2.3 Design Approach

The Ru'ya UAV Crowd Detection and Monitoring System is designed using a modular approach that divides the system into three main subsystems: Hardware, Processing, and Visualization.

Each part has a specific role that helps the system work efficiently and makes it easier to maintain and improve in the future.

- **Hardware Subsystem:** Handles the UAV operations, flight control, and camera for capturing aerial images and sensor data.
- **Processing Subsystem:** Uses AI algorithms to analyze the captured images, estimate crowd density, and detect crowd areas in real time.
- **Visualization Subsystem:** Displays the processed results in the dashboard, including live video, analytics, alerts, and flight information for the operator or admin.

This design keeps the system organized and ensures smooth communication between components, from data collection in the UAV to real-time visualization on the dashboard.

3. Design Considerations

This section outlines the main assumptions, dependencies, and constraints that guide the design of the Ru'ya UAV Crowd Detection and Monitoring System. These factors define the system boundary, determine expected performance, and allow for a feasible system design.

3.1 Assumptions and Dependencies

The following assumptions and dependencies have been made during the design phase:

1. Hardware Dependencies

- The UAV platform is equipped with:
 - A high-resolution camera capable of streaming 1080p video.
 - GPS, gyroscope, and altitude sensors.
 - Sufficient onboard computational power (e.g., NVIDIA Jetson Nano or Raspberry Pi 4) to run the AI detection model.
- The dashboard operates on standard computing devices (laptop or tablet) with a stable internet connection.

2. Software Dependencies

- The UAV firmware supports SDK-based communication (e.g., DJI SDK or DroneKit API).
- The AI module uses frameworks such as TensorFlow Lite or PyTorch Mobile for lightweight, real-time processing.
- The dashboard backend uses a RESTful API to communicate with the UAV and database.
- The frontend is implemented using a web framework (e.g., React.js or Flask) for responsive design.

3. Operating System Dependencies

- Ground control devices run on Windows 10+, macOS, or Android tablets.
- The onboard system runs a Linux-based OS compatible with the AI model runtime (e.g., Ubuntu, JetPack).

4. End-User Assumptions

- Authorized users (administrators and operators) have basic technical knowledge of UAV operation and network usage.
- Only verified personnel can access the dashboard via secure authentication.
- Internet connectivity is available and stable during missions for real-time data transmission.

5. Probable Future Changes

- The system design anticipates integration with multiple UAVs in the future.
- The AI module may be retrained or updated as new datasets become available.

3.2 General Constraints

The following constraints affect or guide the design of the Ru'ya system:

1. Hardware and Performance Constraints

- The UAV's onboard processor has limited computational capacity, requiring optimized AI models and low-latency inference.
- The video transmission bandwidth limits the system to a maximum of 1080p at 30 FPS.
- The UAV battery life (approx. 25–30 minutes) restricts mission duration and requires efficient power management.

2. Network and Communication Constraints

- Communication between UAV and dashboard relies on Wi-Fi 5 or 4G/5G connectivity.
- Latency must be kept below 200 ms for effective real-time visualization.
- Network interruptions may temporarily halt video transmission but should not corrupt data.

3. Security Constraints

- All communications use AES-256 encryption and HTTPS protocols to protect data integrity.
- Only authenticated users can access the system.
- Session tokens expire automatically after 30 minutes of inactivity.

4. Interoperability and Standards

- The design complies with IEEE 830 for requirements specification and IEEE 1016 for design documentation.
- Follows standard UAV SDK protocols to ensure compatibility with future UAV models.

5. Data Management Constraints

- The local or cloud database must store mission data, alerts, and logs with backup capability every 24 hours.
- Storage is limited by available disk space or cloud quota; therefore, old logs may be archived after a set period.

6. Environmental Constraints

- UAV operation is affected by weather conditions (e.g., wind speed, rain).
- The system is designed for outdoor daylight missions; nighttime detection may require infrared camera support in future upgrades.

7. Verification and Validation Constraints

- All software modules must undergo component testing, integration testing, and acceptance testing before deployment.
- AI model performance must be validated against pre-labeled datasets for accuracy and consistency.

4. User Interface Design

4.1 Overview of User Interface

The **Ru'ya UAV-Based Crowd Detection and Monitoring System** provides a responsive **web-based system** designed for two main user roles: **Administrator** and **Operator**. Both users interact with the same unified interface but have different access privileges and control options based on their responsibilities.

4.1.1 Administrator Interface

The **administrator interface** contains the following main sections:

- **Dashboard:** Displays a live overview of current UAV missions, including crowd density, total detected people, alert status, and flight status. The center panel shows a live video stream with AI detection overlays, while a panel presents the flight path and GPS tracking map.
- **Analytics:** Provides access to historical data visualization, including density graphs, heatmaps, and performance metrics from past missions.
- **Alerts:** Lists all active and resolved alerts generated by the AI module, with color-coded severity levels and timestamps.
- **Flight Control:** Enables mission management, such as launching, hovering, or landing the UAV, and viewing telemetry details like altitude, GPS coordinates, and battery levels.
- **Settings:** Allows configuration of AI thresholds, notification methods, and user management controls.
- **Logs:** Stores flight histories, system actions, and maintenance records for administrative review.
- **User Management:** Add or remove operators.

4.1.2 Operator Interface

The **operator interface** is optimized for live UAV mission control and real-time monitoring. It includes the following sections:

- **Dashboard:** Serves as the operator's main control view. It displays key mission indicators such as crowd density, total detected people, alert status, and flight status. The main area features a Live Video Stream with real-time AI detection overlays highlighting detected people and density percentages.
- **Analytics:** Allows the operator to review mission summaries, compare detection results, and view visual analytics such as charts and heatmaps.
- **Alerts:** Displays real-time notifications when the system detects unusual crowd density levels. Alerts are color-coded and remain active until acknowledged by the operator.
- **Flight Control:** Provides direct access to mission commands, including Launch, Hover, Land, and Return Home. Telemetry data such as altitude, GPS coordinates, and battery levels are also visible during operation.

4.2 Interface Design Rules

The **System** interface follows simple and consistent design rules to ensure clarity, usability, and reliability for both administrators and operators.

- **Consistency:**
All pages share a fixed left navigation bar and top status bar. Icons, colors, and button styles remain uniform across screens.
- **Simplicity:**
Each role sees only relevant tools; the operator focuses on flight and monitoring, while the admin accesses settings and logs.
- **Responsiveness:**
The interface automatically adjusts to different screen sizes (desktop and tablet) for flexible field use.
- **Feedback:**
Each user action produces immediate feedback through button highlights, loading indicators, or confirmation pop-ups.
- **Error Handling:**
The system validates all user inputs (e.g., login, mission parameters) and provides descriptive error messages. Connection issues are displayed as status indicators in the top bar.
- **Accessibility:**
Uses readable fonts, high contrast, and text-labeled alerts (e.g., green for safety, red for critical).
- **Security:**
Role-based access separates admin and operator functions to prevent unauthorized actions.

4.3 Screen Images

This section presents the main user interfaces of the Ru'ya UAV Crowd Detection and Monitoring System. Each screen image shows the primary layout, core features, and visual flow from the perspective of an Administrator or Operator.

4.3.1 Login Page

Purpose:

Allows secure authentication for both Administrators and Operators.

Design Description:

The page contains two main input fields (username and password) and a “Login” button. When users submit valid credentials, the system redirects them to the Main Dashboard. Invalid login attempts show an error message.

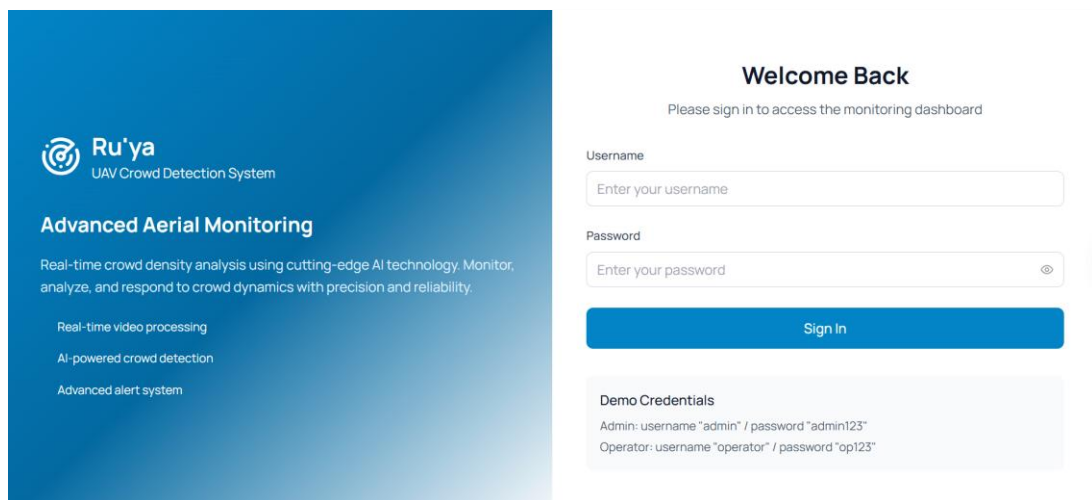


Figure 26. Login Page

4.3.2 Main Dashboard (Home Page)

Purpose:

Shows live UAV mission data and crowd detection results.

Design Description:

The screen is divided into sections:

- **Top bar:** Mission cards showing crowd density, total people, alert status, and flight information.
- **Left panel:** Live UAV video stream (1080p, 30 FPS).
- **Right panel:** Real-time UAV map with GPS coordinates and flight path tracking.

All data are updated in real time using WebSocket communication.

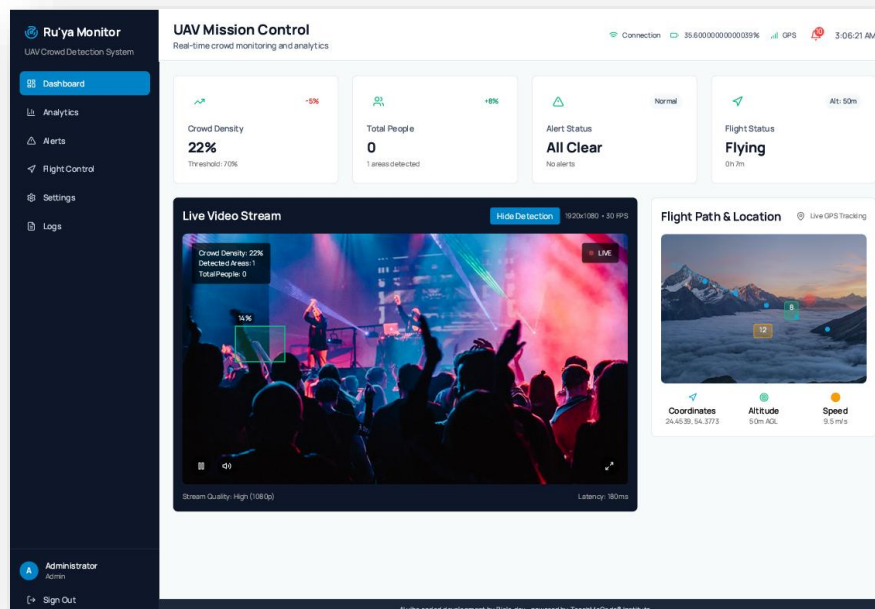


Figure 27. Main Dashboard (Home Page)

4.3.3 Analytics Panel

Purpose:

Displays statistical analysis and trends of mission data.

Design Description:

Includes charts for:

- Crowd density over time
- Number of detected people
- UAV flight altitude and speed trends

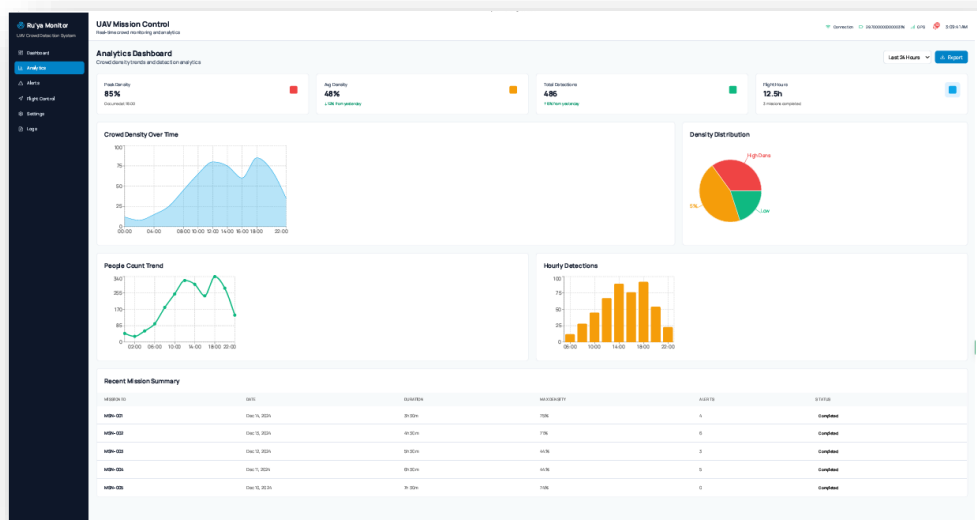


Figure 28. Analytics Panel

4.3.4 Alert management panel

Purpose:

Allows monitoring and control of system alerts.

Design Description:

Alerts are shown in a table with columns for alert type, priority (Low/Medium/High), and timestamp.

Admins can acknowledge or dismiss alerts.

The interface highlights critical alerts in red and plays a short sound when a new high-priority alert appears.

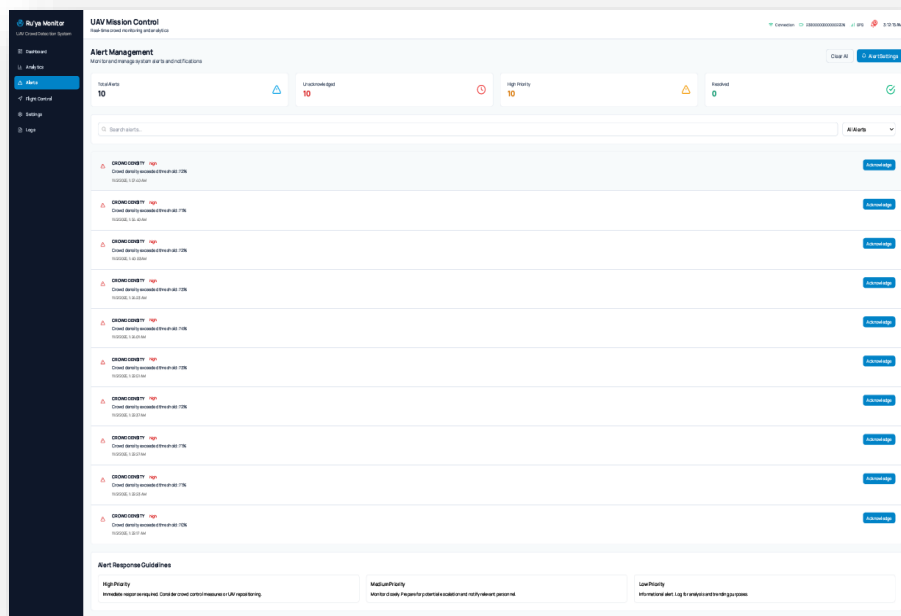


Figure 29. Alert management panel

4.3.5 Flight control center

Purpose:

Provides manual and automated flight control functions.

Design Description:

Displays UAV flight status, altitude, and battery level.

Buttons include *Return to Home*, *Pause Mission*, and *Emergency Land*.

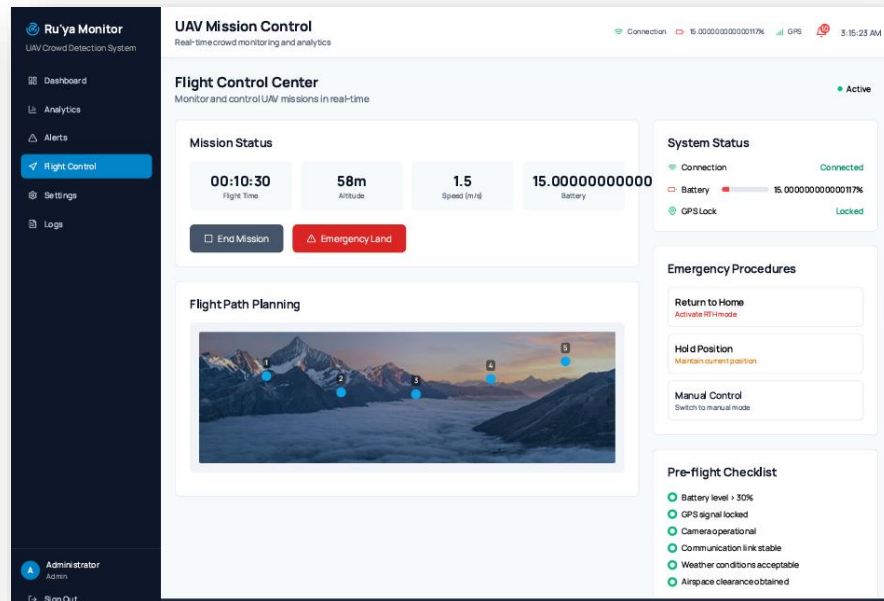


Figure 30. Flight control center

4.3.6 System settings page

Purpose:

Allows administrators to configure system parameters.

Design Description:

Includes fields to update:

- Alert threshold (e.g., 70% crowd density)
- User management (add/remove)
- System calibration options

Changes require admin confirmation and are saved to the central database.

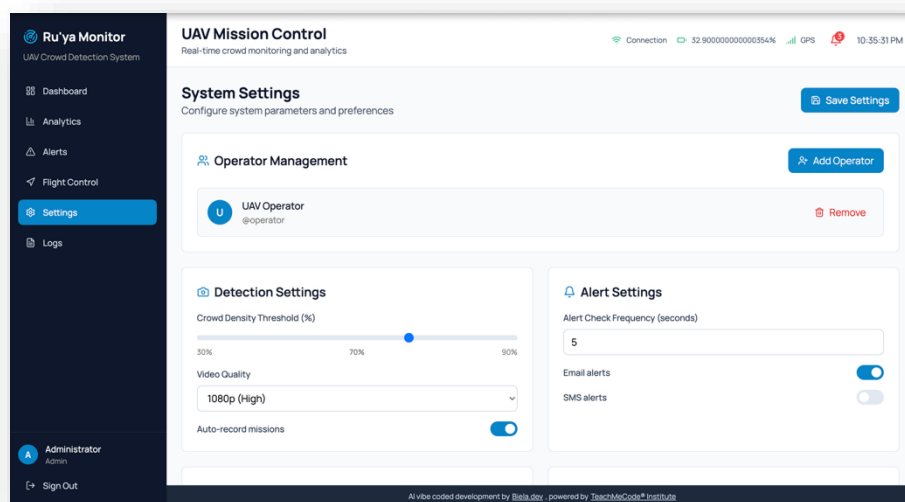


Figure 31 System settings page

4.3.7 Logs page

Purpose:

Provides access to past mission and alert logs.

Design Description:

Shows a searchable table of records including mission ID, date, duration, and alert count. Admins can filter by date or export logs as CSV.

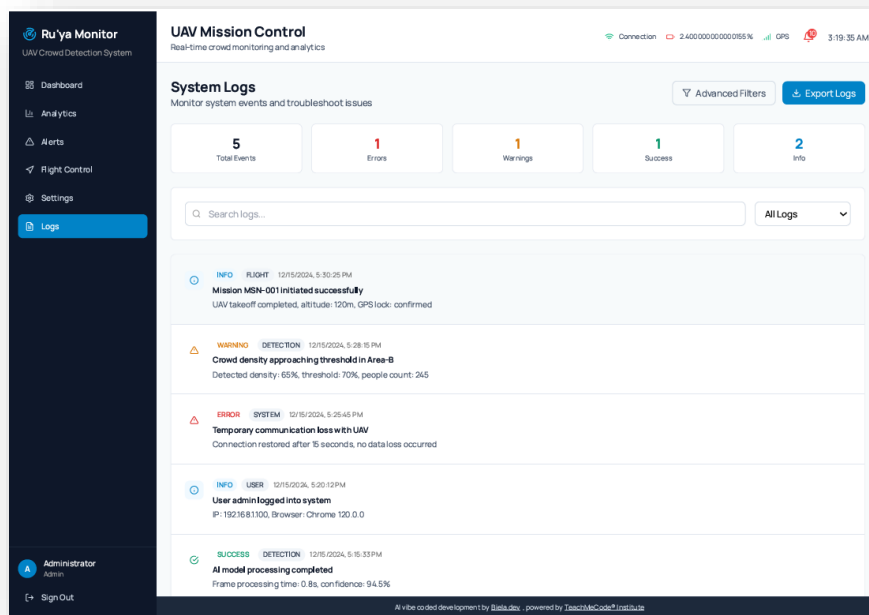


Figure 32: Logs page

4.2 Screen Objects and Actions

This section describes the interactive objects present on each screen of the Ru'ya dashboard and the user actions associated with those objects. The design follows the Eight Golden Rules of Interface Design, ensuring consistency, feedback, and error prevention.

Table 34. Screen Objects and Actions

Login Page		
Screen Object	Description	User Action / Behavior
Username Field	Input box for user's ID or email.	User types their username.
Password Field	Secured text input field.	User enters password; hidden by default.
Show Password Toggle	Eye-icon button.	Reveals or hides password characters.
Login Button	Triggers authentication request.	Validates credentials and redirects to Dashboard.
Main Dashboard		
Mission Summary Cards	Display real-time mission data (Crowd Density, Total People, Alert Status, Flight Status).	Automatically updated; clicking opens detailed analytics.
Live Video Feed	Streaming area for UAV camera.	User can toggle <i>Show/Hide Detection</i> overlay.
Connection Indicator	Displays signal strength and data link status.	Hover shows latency and uptime details.
Flight Status Label	Shows whether UAV is flying, idle, or disconnected.	Color changes dynamically.
Alert Bar	Visual banner for critical warnings.	Clicking opens Alert Management Panel.
Analytics Panel		
Charts (Line, Bar, Pie)	Visual representation of crowd and mission data.	Hover to view exact values; filter by mission/date.
Mission Summary Table	Displays ID, duration, max density, status.	Click row to open detailed mission report.
Export Button	Generates downloadable report (CSV/PDF).	User selects export format and destination.
Flight Control Center		
Map Panel	Displays live UAV path and waypoints.	Zoom, pan, or select waypoint for info.
Control Buttons	<i>End Mission, Return Home, Hold Position, Emergency Land.</i>	Send control commands to UAV firmware.
Telemetry Display	Shows altitude, speed, battery, time.	Updates automatically in real-time.
Emergency Actions Panel	Contains critical safety controls.	Requires confirmation before execution.
System Settings Page		
Threshold Input Field	Numeric box for alert density level.	Admin enters new value (e.g., 70%).

Notification Settings	Toggles for email/SMS alerts.	Admin enables/disables as needed.
Save Settings Button	Confirms and saves configuration.	Applies new settings to system runtime.
CPU/Memory Bars	Visual indicators of system load.	Read-only; refresh every 5 seconds.
Logs Page		
Log Table	Displays mission and system events.	User scrolls, sorts, or filters logs.
Export Logs Button	Downloads logs in CSV format.	Initiates export process.
Advanced Filters	Allows filtering by date, event type, or module.	User sets criteria and applies.

5. System Architecture

The Ru'ya system is composed of several components that divide responsibilities such as sensing, acquisition, processing, and visualization. This decomposition approach provides scalability, tractability, and maintainability, enabling the system to operate independently and supporting future integrations. The higher-level components consist of the following primary roles, shown in Table 34:

Table 35. Major Identified Roles Of The System.

Role	Description
Flight control and Data Acquisition	✓ Enables UAV operations. ✓ Crowd sensing and capturing visual data.
Intelligent Processing	✓ Preprocesses and analyses the captured aerial data. ✓ provide a real-time estimate of the crowd gathering.
Visualization and Data Storage	✓ Displaying crowd estimations and analytics. ✓ Archives previous results

Each component fulfils its role while working closely with the other components through communication links. Hence, collaborating to achieve the required results. This design ensures a modular and fault-tolerant system that works well as a standalone project and in future distributed deployments.

Other system decompositions were explored and discussed, such as combining the AI processing and the visualization subsystems into a single unit; however, this decomposition was rejected due to hardware limitations and potential computational overhead, while the chosen approach ensures smooth processing without affecting other components in the system. Thus, our proposed decomposition proves itself in many discussed aspects, providing a flexible and efficient workflow.

5.1 Architectural Design Approach

Our architectural design approach employed a layered modular architecture inspired by the Model–View–Controller (MVC) design pattern.

1. **Model (Hardware Subsystem):** denotes the UAV hardware and sensor data.
2. **Controller (Processing Subsystem):** processes the visual data captured by the UAV.
3. **View (Visualization Subsystem):** display the processed results in the dashboard.

Each subsystem has a clear boundary and communicates through simple communication links to ensure reliable and consistent results.

5.2 Architectural Design

Based on the previous analysis of roles decomposition, the architectural design of the Ru'ya system is organized into three primary subsystems.

1. The Hardware Subsystem

serves as the baseline of the system responsible for the flight control and aerial data acquisition. It consists of the UAV platform, flight controller, and camera module, which operate collaboratively to collect visual data of crowd gatherings and then transmit it to the processing subsystem for AI-based analysis.

2. The Processing Subsystem

operates as the “brain” of the system, it receives the raw captured aerial data from the hardware subsystem, preprocesses and analyses the data to provide a real-time analysis and estimate of the crowd gathering, then it transmits these results to the visualization subsystem.

3. The Visualization Subsystem

acts as the user interface; it receives the results from the AI module and displays them in a user-friendly dashboard. It will display visualization and analytics of the crowd estimate and provide alerts and warnings if a certain crowd threshold is exceeded. It furthermore archives previous results and flight logs for review and system evaluation. Figure 33 depicts the overall system architecture.

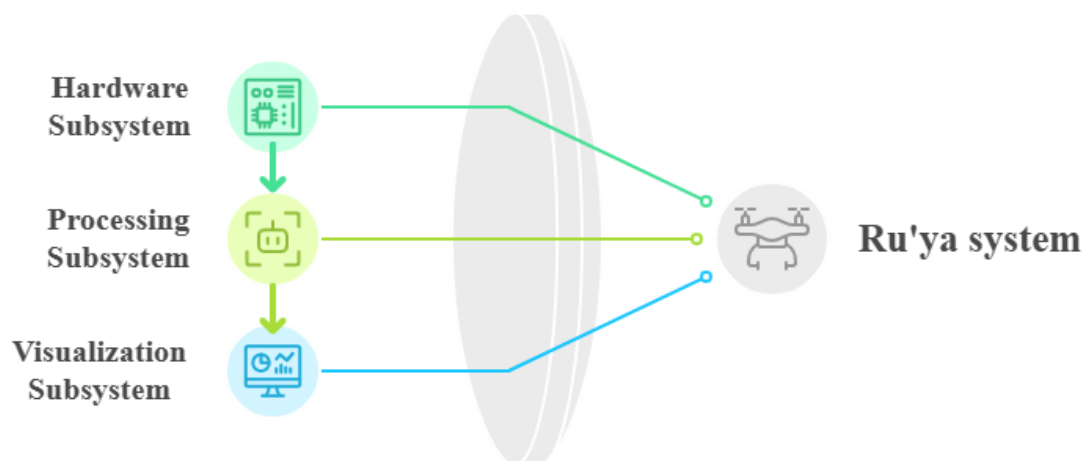


Figure 33. The Architectural Design of Ru'ya

As shown in Figure 33, the data flow sequentially starts from hardware to processing to visualization, with lightweight communication links such as Wi-Fi. Ensuring clear separation between the roles discussed previously, which provides ease of use, while allowing consistent communication between subsystems.

5.3 Subsystem Architecture

This section will provide a decomposition of the subsystems in the architectural design, using the functional description approach, which will demonstrate what the system does in terms of the data flow and control between activities.

Figure 34 shows the overall system's flow of data with three main processes transferring data between and with other external entities; it demonstrates the whole data cycle, the hardware subsystem collects data, the AI module processes it, and the dashboard displays live analytics. Finally, the operator can view results and monitor crowds smoothly. This data flow diagram summarizes all previously discussed components and shows the actual operations happening between these subsystems to provide the expected results.

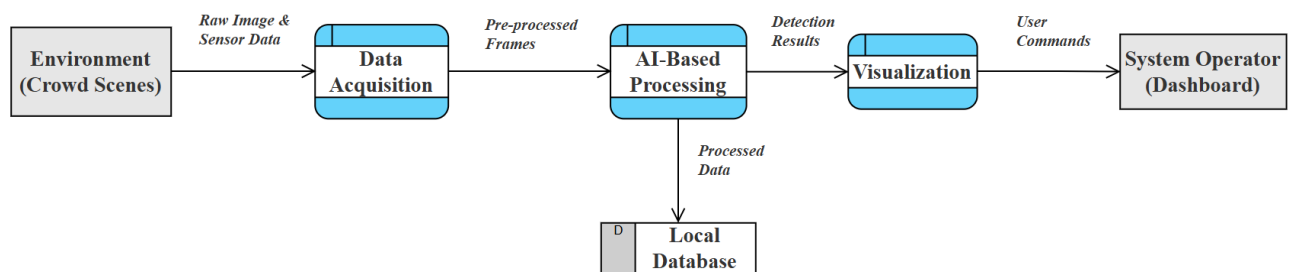


Figure 34. Top-Level Data Flow Diagram (DFD Level 0)

1. The Hardware Subsystem

The hardware subsystem handles flight control and data acquisition through several steps. The main processes are depicted in Figure 35. The UAV captures the image frames using the camera module from the external environment, then collects sensor data such as GPS and movement data then combines each frame with its matching sensor data (same timestamp). Finally, the captured aerial data are ready to be transmitted to the processing subsystem.

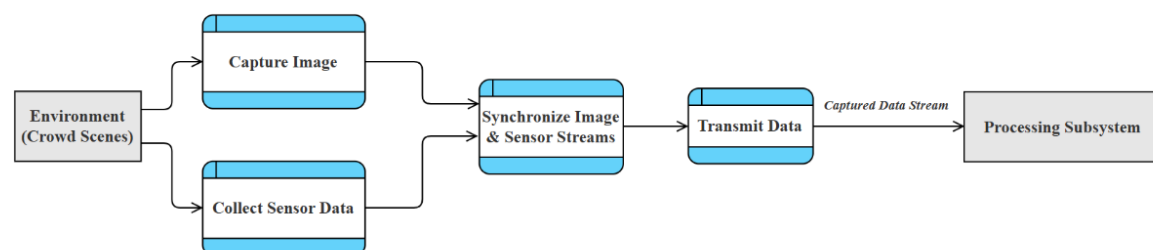


Figure 35. The Hardware Subsystem (DFD Level 1)

2. The Processing Subsystem

The processing subsystem serves as the “brain” of the entire system. The main processes are demonstrated in Figure 36 It receives data from the hardware, cleans it and prepares it, which usually includes filtering and resizing of images, then processes it by running the AI model to estimate crowd density, and finally broadcasts the findings for visualization.

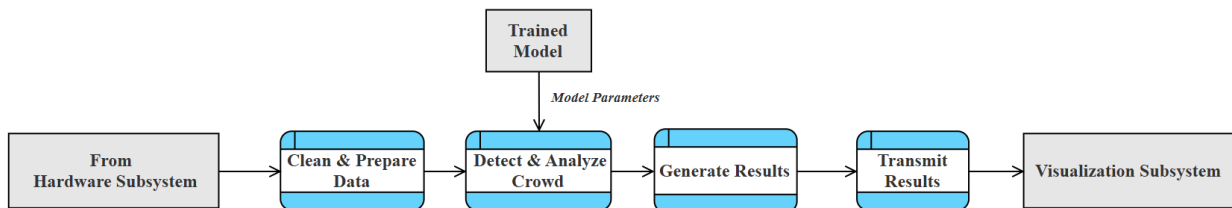


Figure 36. The Processing Subsystem (DFD Level 1)

3. The Visualization Subsystem

The Visualization Subsystem is considered the user-interface side of Ru’ya. It receives the crowd estimation results from the AI module, handles the system operator inputs, displays results in the dashboard, and stores processed data in a local database.

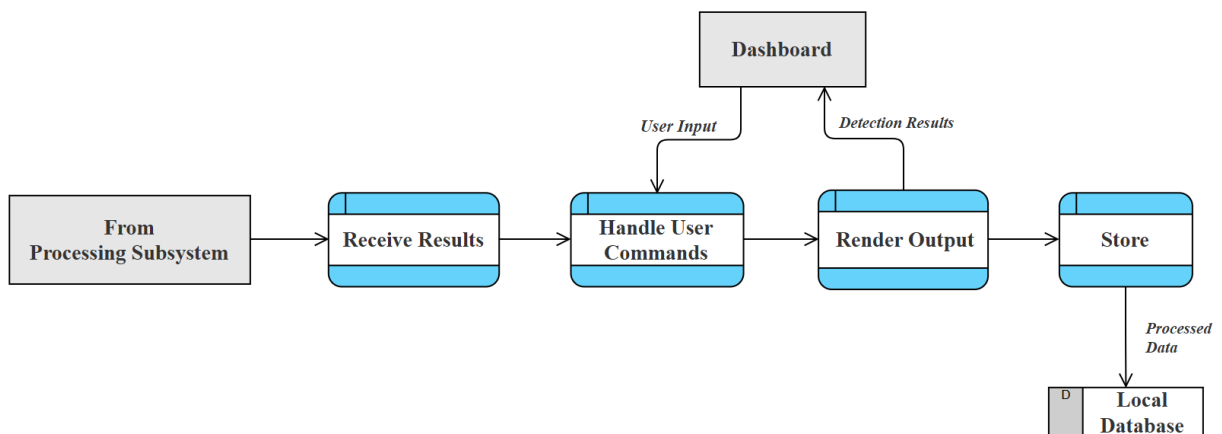


Figure 37. The Visualization Subsystem (DFD Level 1)

5.3.4 The Structural Decomposition

In this section, we will illustrate the hierarchical organization of the Ru'ya System shown in Figure 38. It represents the system's internal structure as a set of three primary subsystems, categorized by their functional roles. This decomposition emphasizes the logical flow of the task, starting from aerial operation and data acquisition, through AI-based processing and analysis, to user-friendly visualization. Each component functions independently but also preserves smooth communication with the other components, which enables the system to operate in a modular, scalable, and flexible manner. This structure also assists future additions such as UAV upgrades, AI model modification, or dashboard enhancements without changing the entire system.

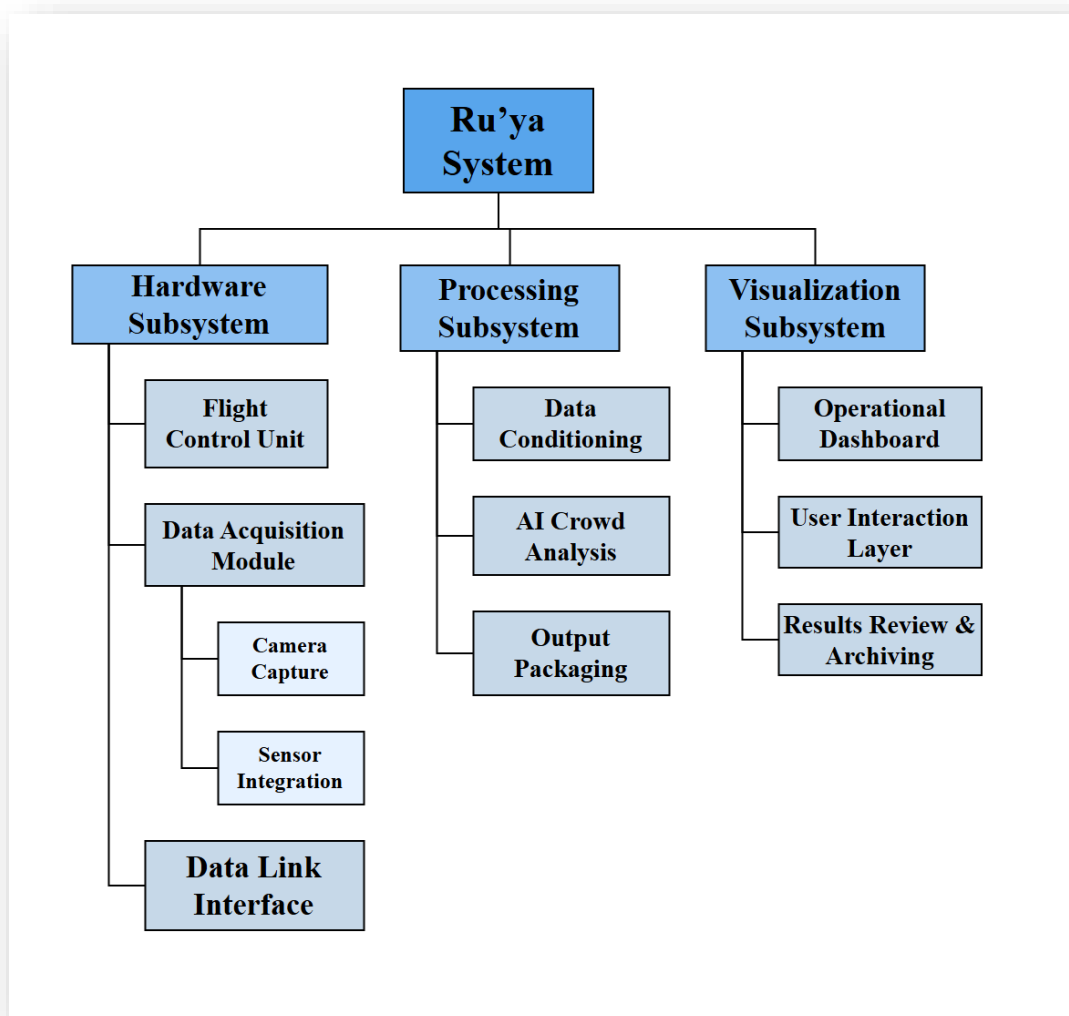


Figure 38. Structural Decomposition of the Ru'ya System

6. Data Design

This section includes a description of the Ru'ya system's data, data types, and a list of all the functions and function parameters of the application's database.

6.1 Data Description

In this Sub-section, we will describe how our system's information domain is transformed into data structures, how the major system entities are stored, processed, and organized.

The Ru'ya system handles various types of data acquired, processed, visualized, and stored throughout the system's operations and procedures. The information domain primarily consists of aerial data collected by the UAV's camera module, processed results by the AI module, and metadata of mission status and archived flight logs. Table 36 clearly demonstrates how the information domain is transformed into data structures during the Ru'ya system's operations.

Table 36. Information Domain Process

Stage	Data Source	Data structure	Transmitted To
Data Acquisition	UAV camera module captures aerial data.	Raw image frames (H×W×D).	RAM or temporary buffer
Processing	The AI module processes the captured frames and detects crowd density.	Structured detection results (JSON/dict).	Dashboard and Local database.
Communication	Processed data packaged via the communication links	Metadata packets.	Dashboard
Storage	Results and mission data are stored.	Mission logs, timestamps, counts.	Local database

The system contains all the required data to have complete functionality. These identified data structures can be categorized into the following:

- 1. Temporary Data:** Exists only during flight and is cleared once it is transmitted.
 - ✓ Raw image frames captured by the UAV camera during flight.
- 2. Permanent Data:** Stored for later analysis, reporting, and performance evaluation.
 - ✓ Detection results (crowd counts, bounding boxes, GPS, timestamps).
 - ✓ Mission logs containing summarized information and performance metrics.

In summary, the system saves temporary data in memory during flight and stores permanent data in a local lightweight database.

6.2 Data Dictionary

The Ru'ya dashboard (The visualization subsystem) has two user roles:

- **Administrator:** Full access, including access to settings and mission logs.
- **Operator:** Limited access can only monitor crowd analytics, alerts, and flight.

Table 37 shows the data dictionary of the Ru'ya system containing a list of all the functions, function parameters, Accessibility to whom, and a description of the function.

Table 37. Data Dictionary

Function / Data Name	Type	Parameters	Accessible By	Description
<i>AcknowledgeAlert()</i>	Function	alert_id	✓ Admin ✓ Operator	Marks an alert as acknowledged.
<i>AlertStatus</i>	String	N/A	✓ Admin ✓ Operator	Alert state (Active, High, Resolved).
<i>avg_density</i>	Float	N/A	✓ Admin ✓ Operator	Average crowd density over a time range.
<i>BatteryLevel</i>	Float %	N/A	✓ Admin ✓ Operator	UAV battery percentage during flight.
<i>CaptureFrames()</i>	Function	frame_rate, resolution	✓ Admin ✓ Operator	Captures image frames from the UAV camera.
<i>CrowdDensity</i>	Float %	N/A	✓ Admin ✓ Operator	Real-time crowd density from the AI module crowd detection.
<i>DisplayAnalytics()</i>	Function	time_range, mission_id	✓ Admin ✓ Operator	Loads analytics view and shows summary charts.
<i>DisplayDashboard()</i>	Function	user_role	✓ Admin ✓ Operator	Loads the dashboard based on the user's role.
<i>DisplayFlightStatus()</i>	Function	altitude, speed, battery_level, gps_lock	✓ Admin ✓ Operator	Shows UAV flight status.
<i>DisplayLiveStream()</i>	Function	video_source, overlay_data	✓ Admin ✓ Operator	Streams UAV video feed with AI detection.
<i>DisplayLogs()</i>	Function	log_type, mission_id	✓ Admin	Retrieves mission logs from storage.
<i>EmergencyLand()</i>	Function	N/A	✓ Admin ✓ Operator	Executes immediate UAV landing.
<i>EndMission()</i>	Function	mission_id	✓ Admin ✓ Operator	Ends mission.
<i>ExportReport()</i>	Function	report_type, mission_id	✓ Admin	Exports mission logs or analytics summary.
<i>FlightHours</i>	(Float hours)	N/A	✓ Admin ✓ Operator	Flight time.
<i>GenerateCharts()</i>	Function	time_range, data_points	✓ Admin ✓ Operator	Produces charts for density and detections.
<i>GenerateHourlyDetections()</i>	Function	mission_id	✓ Admin ✓ Operator	Generates an hourly detection distribution plot.
<i>GeneratePeopleCountTrend()</i>	Function	time_range	✓ Admin ✓ Operator	Creates a trend graph of people count over time.

<i>gps_coordinates</i>	(Float, Float)	N/A	✓ Admin ✓ Operator	UAV latitude and longitude during operation.
<i>InitializeMission()</i>	Function	mission_id, flight_path	✓ Admin ✓ Operator	Sets flight path for mission.
<i>LoadModel()</i>	Function	model_path	✓ Admin	Loads trained AI model into inference module.
<i>LogMissionData()</i>	Function	mission_id, crowd_count, timestamp	✓ Admin	Records detections and status in the mission log.
<i>MissionSummary</i>	Data Structure (JSON)	N/A	✓ Admin ✓ Operator	Mission data (summary of detections, density, alerts).
<i>peak_density</i>	Float %	N/A	✓ Admin ✓ Operator	Maximum crowd density recorded per mission.
<i>ProcessFrames()</i>	Function	frame, model_weights	✓ Admin ✓ Operator	Runs the AI module to detect and count people in frames.
<i>RecentMissionSummary</i>	Data Structure (JSON)	N/A	✓ Admin ✓ Operator	Summary of recent missions in the analytics panel.
<i>SignalStatus()</i>	Function	connection_status	✓ Admin ✓ Operator	Monitors the communication link.
<i>SystemMetrics</i>	Data Structure (System Stats)	N/A	✓ Admin	Tracks resources usage.
<i>total_detections</i>	Integer	N/A	✓ Admin ✓ Operator	Total number of people detected in a mission.
<i>TransmitResults()</i>	Function	detection_data, gps_coordinates	✓ Admin ✓ Operator	Sends processed results to the dashboard.
<i>TriggerAlert()</i>	Function	density_threshold, current_density	✓ Admin ✓ Operator	Trigger alert when density exceeds threshold.
<i>UpdateAnalytics()</i>	Function	mission_id, density_values	✓ Admin ✓ Operator	Retrieve analytics with the updated detection results.
<i>UpdateSettings()</i>	Function	threshold, video_quality, alert_frequency	✓ Admin	Changes crowd detection threshold and alert configurations.

6.3 Database Description

The Ru'ya system utilizes a local lightweight database, consisting of four primary entities: User, Mission, Detection, Alert, and Analytics Summary. Figure 39 demonstrates the conceptual entity relationship diagram of our database, showing how the high-level entities interact and relate; hence, describing real-world logic.

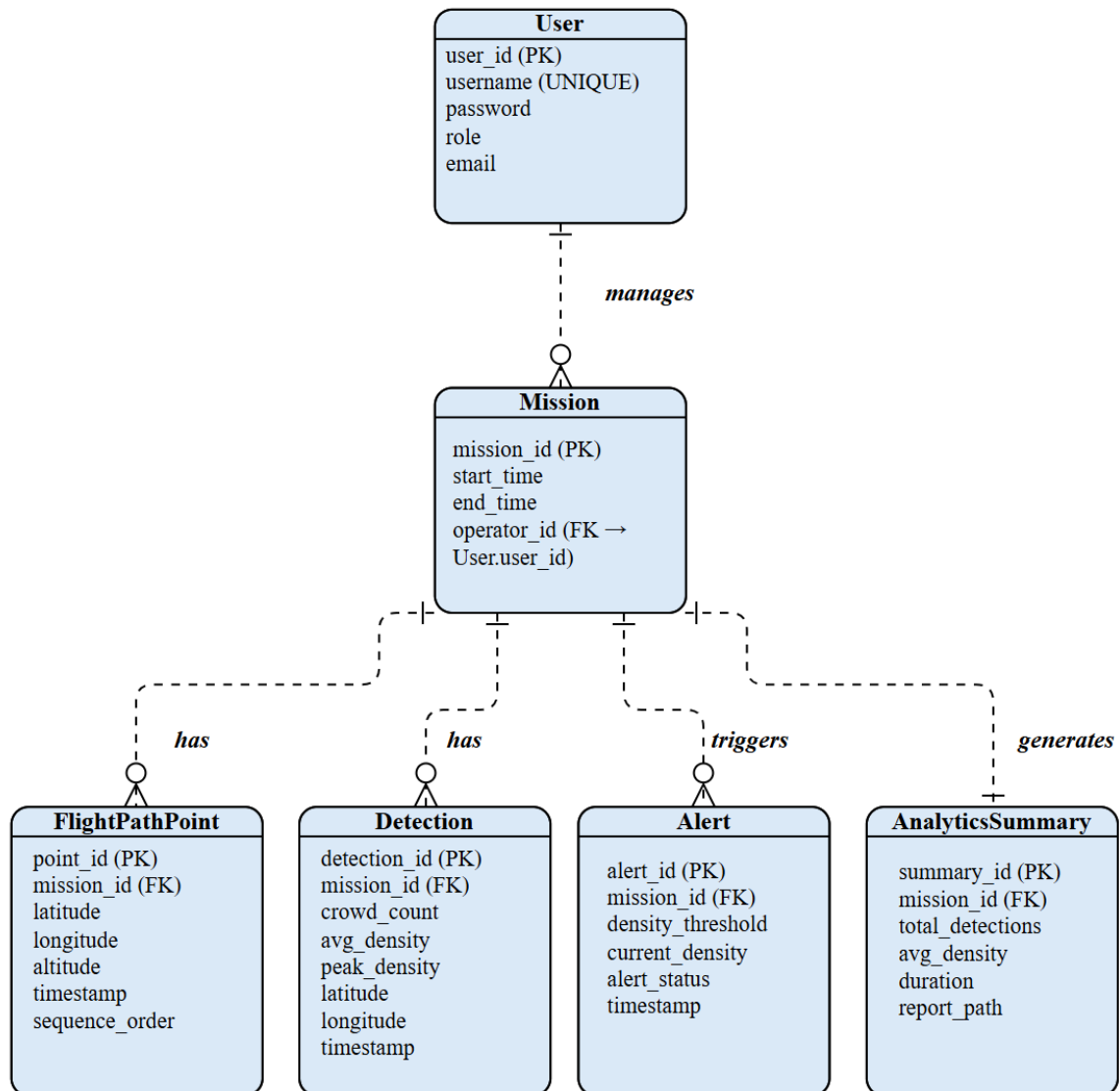


Figure 39. Conceptual Entity Relationship Diagram

For additional clarification of the actual implementation of the database, the tables with their primary key (PK) and foreign key (FK) structure, and how the data will be stored, a relational entity relationship diagram is illustrated in Figure 40.

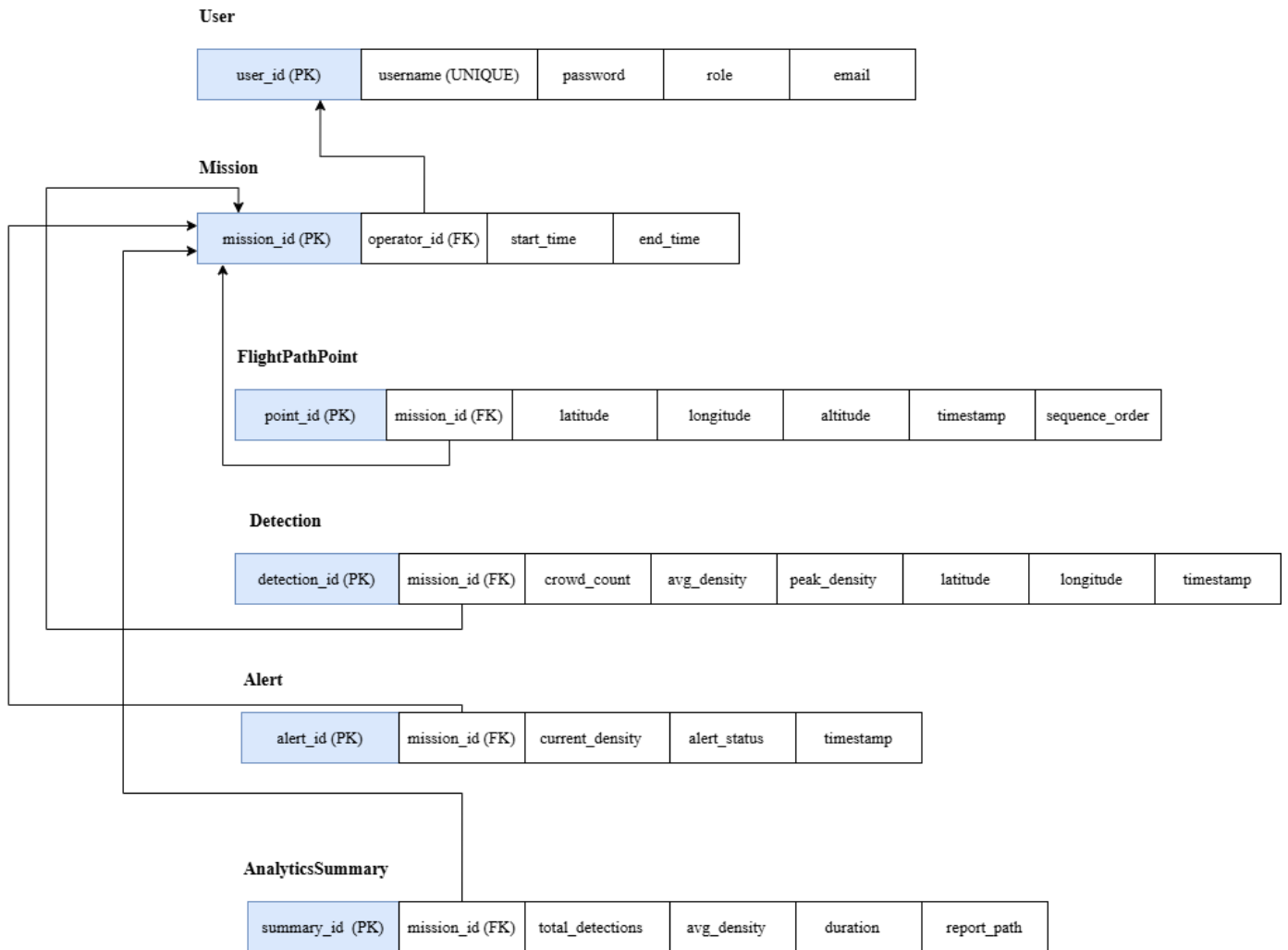


Figure 40. Relational Entity Relationship Diagram

7. Component Design

This section presents the pseudocode for the main functional components of the Ru'ya UAV Crowd Detection and Monitoring System. The pseudocode follows informal English-based logic to help developers understand the behavior and flow of each major function.

7.1 Login component

```
If username OR password is empty:
    Display error message "Please fill out this field"
Else:
    Retrieve user record from database

    If user does not exist:
        Display error message "Invalid credentials"
    Else:
        Redirect user to Dashboard
```

Figure 41. Login component Pseudocode

7.2 Dashboard Component

```
Load real-time UAV data
Load video stream
Show crowd density and detection results
Show UAV telemetry (GPS, altitude, speed)
Update all values continuously
```

Figure 42. Dashboard Component Pseudocode

7.3 Analytics Component

```
Fetch historical mission data
Generate charts and trends
Display analytics to user

If export requested:
    Produce downloadable report
```

Figure 43. Analytics Component Pseudocode

7.4 Alerts Component

```

Check crowd density against threshold

If exceeded:
    Create alert and show it
Else:
    Show system as clear
Update alert list regularly
    
```

Figure 44. Alerts Component Pseudocode

7.5 Flight Control Component

```

Show available drone commands

If user selects a command:
    Send it to UAV
Receive live telemetry
Update flight status on screen
    
```

Figure 45. Flight Control Component Pseudocode

7.6 Settings Component

```

Load current configuration
Show editable settings

If admin saves changes:
    Validate and store new values
    Apply updated configuration
    
```

Figure 46. Settings Component Pseudocode

8. Detailed System Design

This section breaks down the main software components of the Ru'ya UAV Monitoring System. For each component, we outline what it is, what it does, any limitations it has, and how it's structured internally. The goal is to give a straightforward, detailed view of how each part of the system works and how they fit together.

8.1 Classification, Definition, Responsibilities

Table 38. Component, Classification, Definition and Responsibility

Component	Classification	Definition & Responsibilities
Login	User Interface Module	Verifies the user's credentials and opens a secure session. Successful logins go to the Dashboard, invalid attempts are rejected with an error.
Dashboard	Primary Interface Subsystem	Provides a real-time view of the UAV mission. It shows the live video feed, detection overlays, alerts, and key flight metrics so operators can monitor everything in one place.
Analytics	Data Processing Module	Processes stored mission data to generate trends, charts, and summaries. Helps operators review crowd behavior, detections, and UAV performance over time.
Alerts	Event Monitoring Module	Watches live detection data for unsafe conditions. When density or UAV status crosses a set threshold, it triggers an alert and surfaces it on the Dashboard.
Flight Control	Command and Telemetry Module	Lets the operator manage the UAV directly. They can start or stop missions while the system streams live GPS, altitude, battery, and movement data from the UAV.
Settings	Configuration Management Module	Manages system configuration, including detection thresholds, quality parameters, and alert rules. Admin updates are saved and applied across the system.

8.2 Constraints

Table 39. Components Constrains

Component	Constraints
Login	• Credentials must be validated securely. • Session must remain consistent after login. • Accounts lock after repeated failed attempts.
Dashboard	• Requires stable real-time data from UAV. • stream latency must stay low. • Must handle rapid updates without slowing down.
Analytics	• Only authorized users can access historical data. • Large datasets must load efficiently. • Data must stay synced with backend storage.
Alerts	• Alert frequency must respect system limits. • Depends on correct thresholds from Settings. • Every alert must be logged for traceability.
Flight Control	• Requires stable UAV connection for sending commands. • Only authenticated users can send commands. • Telemetry must refresh with minimal delay.
Settings	• Only admins can change system configuration. • Updates must be validated before applying. • System modules must reload cleanly when settings change.

8.3 Composition

Table 40: Components Composition

Component	Composition (Subcomponents & Their Roles)
Login	• Credential input form (username & password) • Authentication check handled by backend session validation
Dashboard	• Live video feed with detection overlays • Status cards (crowd density, alerts, total people, UAV state) • Telemetry panel (GPS, altitude, speed)
Analytics	• Historical mission data retrieval • Trend and summary report generator (charts, statistics)
Alerts	• Threshold comparison logic for density and UAV status • Alert display panel on the Dashboard
Flight Control	• Operator command interface (start, stop, emergency actions) • Real-time telemetry stream (GPS, battery, altitude, speed)
Settings	• Configuration form (thresholds, quality settings, retention options) • Backend update handler that applies new system parameters

8.4 Uses/Interactions

The Ru'ya Drone System is composed of multiple modules that interact to achieve real-time crowd monitoring and analysis. Major interactions occur among the Drone Module, AI Processing Unit, and User Dashboard, supported by communication and data management subsystems.

1. Drone → AI Processing Unit

- The drone captures live aerial footage and send it to the AI unit via secure Real-Time Streaming Protocol (RTSP) to process the raw data.
- For each frame metadata such as GPS coordinates, altitude, and timestamp are attached.

2. AI Processing Unit → Dashboard

- The AI module performs preprocessing on the captured live aerial footage, object detection and density estimation.
- The processed data (crowd size, movement patterns, and heatmaps) are sent to the dashboard through RESTful APIs.

3. Dashboard ↔ Drone Operator

The dashboard acts as a central control and monitoring interface enabling the operator to:

- View live processed data and real-time analytics (crowd density, movement heatmaps).
- The operator can generate commands to the drone, such as zoom control, camera repositioning and altitude adjustments.
- Access mission statistics and export analytical reports for documentation and review.

All communications between the dashboard and the drone real time data are handled through the backend API, while maintaining secure transmission and consistent synchronization of telemetry data such as battery, GPS and altitude.

In the following section, we will discuss the main dashboard functionalities through using use case diagrams, explaining how the sub-system components and the user interact with each other to perform the essential operations within the Ru'ya monitoring system.

8.4.1 Login

Figure 47 shows the “Login” sequence diagram.

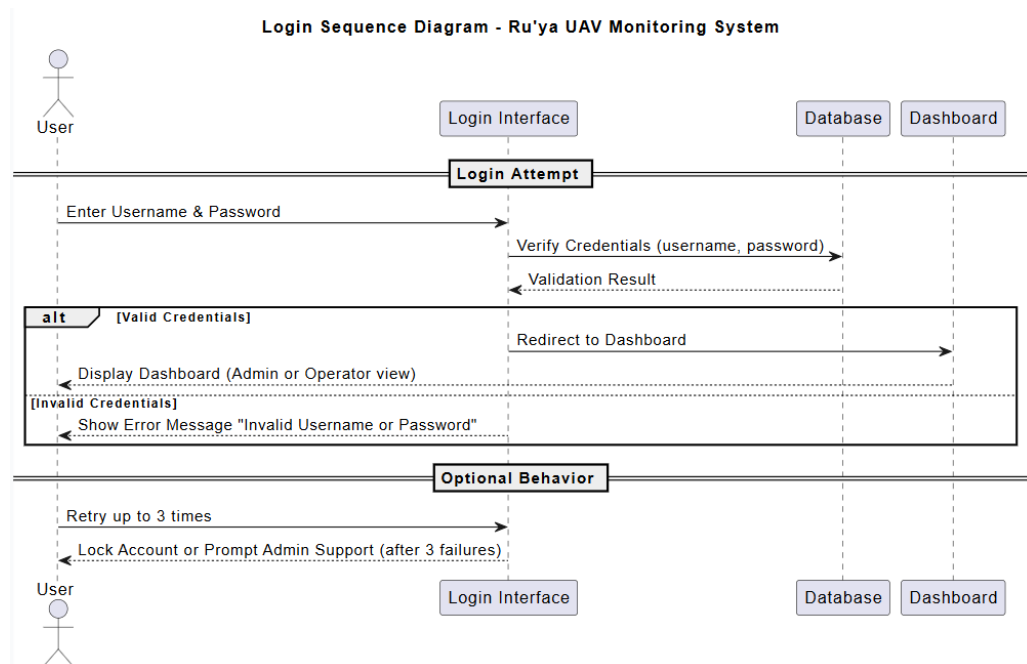


Figure 47. The “Login” sequence diagram

8.4.2 Detection and Alert interaction

Figure 48 shows the “Detection and Alert interaction” sequence diagram.

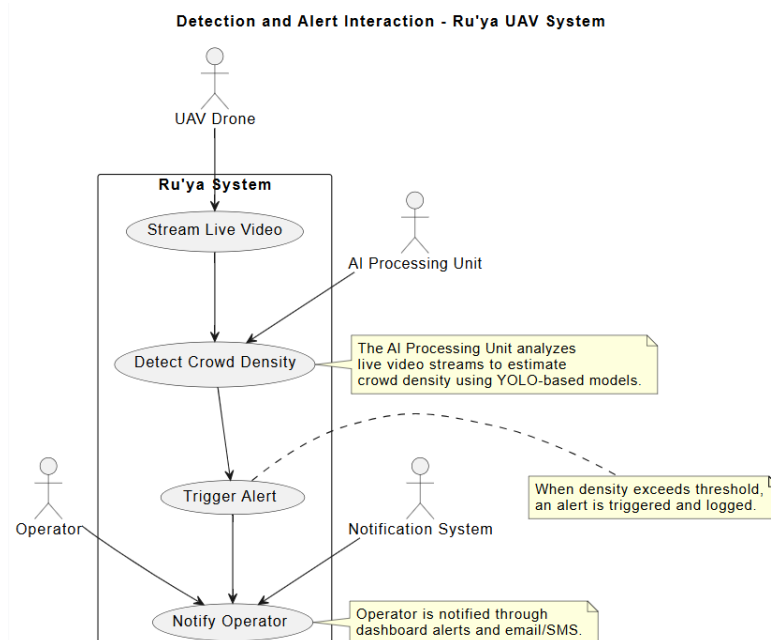


Figure 48. The “Alert & Detection” sequence diagram

8.4.3 Analytics

Figure 49 shows the “Analytics” sequence diagram

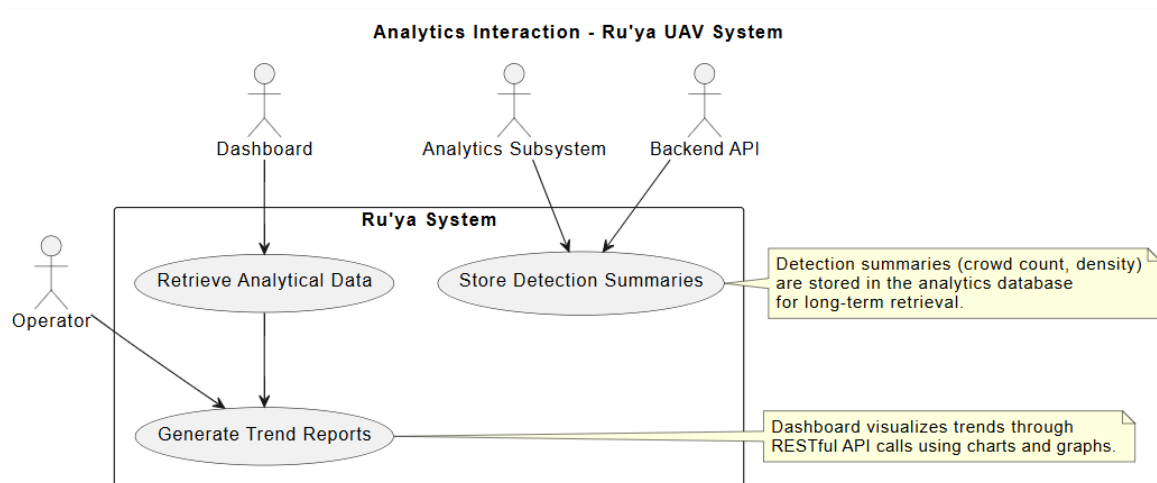


Figure 49. The “Analytics” sequence diagram

8.4.4 Flight Control

Figure 50 shows the “Flight Control” sequence diagram.

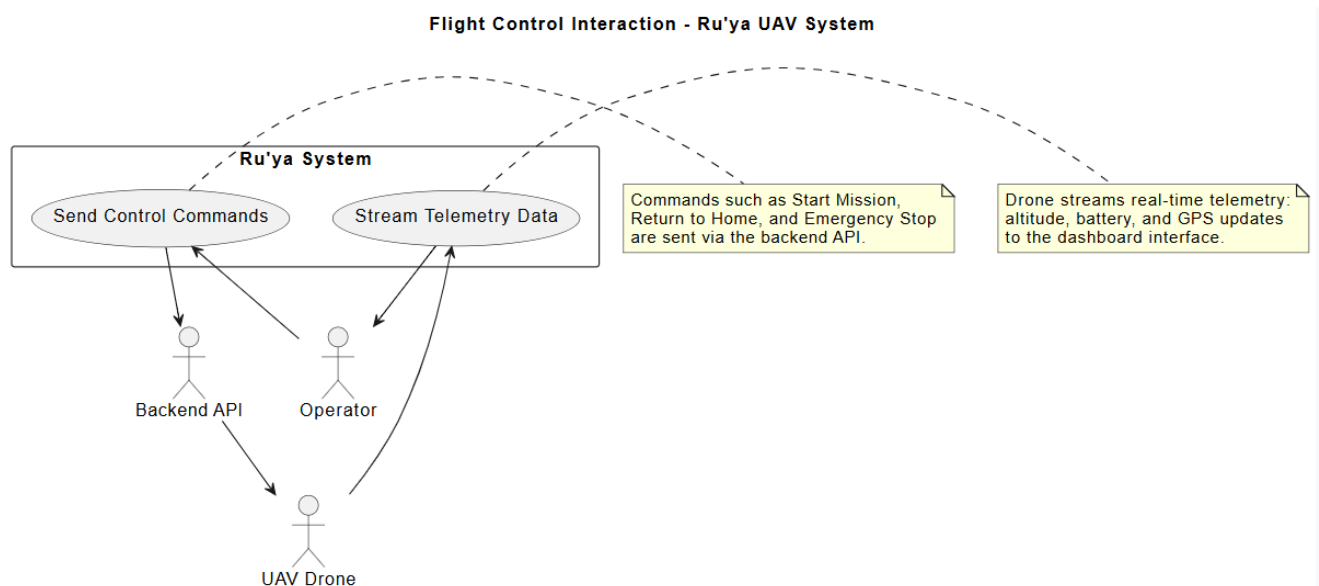


Figure 50. The "Flight Control" sequence diagram

8.4.5 Setting and Admin Interaction

Figure 51 shows the “Setting and Admin Interaction” sequence diagram.

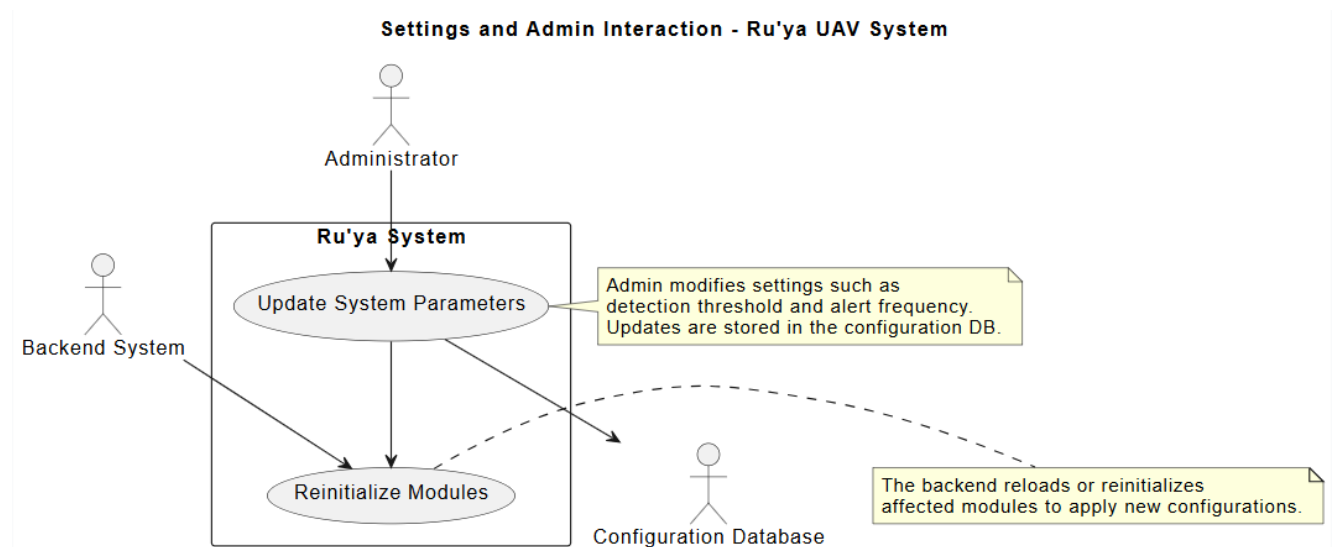


Figure 51. The "Setting and Admin" sequence diagram

8.4.6 System Logs Use Case

Figure 52 shows the “System Logs” sequence diagram.

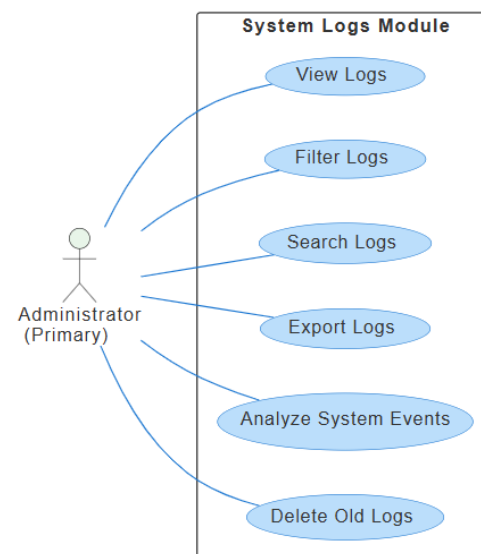


Figure 52. The "System Log" sequence diagram

8 Admin ↔ AI Processing Unit

- The administrator manages datasets, updates model checkpoints, and monitors inference performance

8.5Resources

8.5.5 Resource Overview

The foundation of Ru'ya drone system is a combination of hardware, software and data resources to enable real-time crowd monitoring and detection through AI driven analytics. The following table 41 illustrates and summarizes the key resources our project utilized while maintaining proper management to achieve high performance and scalability, and fault tolerance across distributed modules.

Table 41. Summary of hardware, software, and data resources utilized by the Ru'ya Drone System

Category	Resources	Description
Hardware	Drone Frame	Holds all drone components and ensures flight stability.
	Flight Controller (Pixhawk)	Manages drone stabilization, navigation, and sensor data. Integrates with GPS and IMU to maintain flight control and respond to AI-driven commands.
	Onboard Compute Module (Raspberry Pi 4 / Jetson Nano)	Compact single-board computers used for onboard processing. They handle real-time image capture, preprocessing, and communication with the ground station, enabling efficient edge computing and reduced latency.
	Camera Module (HD / 4K)	Captures aerial images for analysis.
	Power System	Supplies power to all drone components.
	Ground Station / Server	High-performance workstation or cloud instance for AI inference and analytics.
	Network Interface	Wi-Fi or 5G connectivity supporting low-latency video transmission.
Software	Backend	Python backend is responsible for API management and data exchange.
	AI Libraries	TensorFlow, PyTorch, OpenCV, and NumPy are used for deep learning inference, object detection, and image preprocessing.
	Frontend	HTML/CSS and JavaScript for dashboard visualization and user interaction.
	Database	PostgreSQL or Firebase for storing analytics, logs, and metadata.
	Containerization	Docker ensures environmental consistency and reproducible deployments.
Dataset	Datasets	Real-world crowd datasets used for model training and validation.
	Analytics Logs / Video Frames	Historical outputs and recorded sessions used for performance analysis.

8.7.2 Concurrency and Dependency Considerations

Ru'ya project relies on distributed architecture involves multiple modules operating simultaneously which requires careful synchronization to avoid race conditions, deadlocks, or resource contention situations. In the following table 42 outlines the Potential issues arise because of concurrency and parallelism and the implemented strategies to avoid it through the design process.

Table 42. Concurrency and Dependency Management Strategies in the Ru'ya Drone System

Potential Issue	Cause	Resolution Strategy
Frame buffer race condition	Simultaneous access by image capture and transmission threads	Thread-safe buffer access with mutex locking
Database contention	Concurrent write requests from AI and Dashboard modules	Use transaction control and write queueing
Network congestion	High-resolution video transmission under weak signal	Adaptive bitrate encoding and data compression
Deadlock between modules	Circular dependency between GPU and DB write threads	Use fixed lock acquisition order and timeout-based rollback

8.6 Processing

The Ru'ya Drone System performs real-time crowd detection and analytics through an integrated pipeline built around the YOLO (You Only Look Once) deep learning architecture. The system follows a modular architecture, where the drone, AI inference engine, and dashboard operate concurrently through synchronized communication.

1. Data Acquisition:

The UAV drone continuously captures aerial video using a high-resolution onboard camera. Frames are transmitted to the AI Processing Unit via Wi-Fi or 5G. Buffered streaming ensures stable frame rates even under fluctuating network conditions.



Figure 53. Example of a raw drone-captured crowd image illustrating the initial visual input used by the Ru'ya Drone System for real-time monitoring and analysis.

2. Preprocessing:

Incoming frames are resized and normalized using OpenCV to match YOLO's input specifications. Techniques such as Gaussian blurring and color correction are applied to enhance detection accuracy under varying lighting or crowd density conditions.

Drone-Based Crowd Monitoring Preprocessing Pipeline

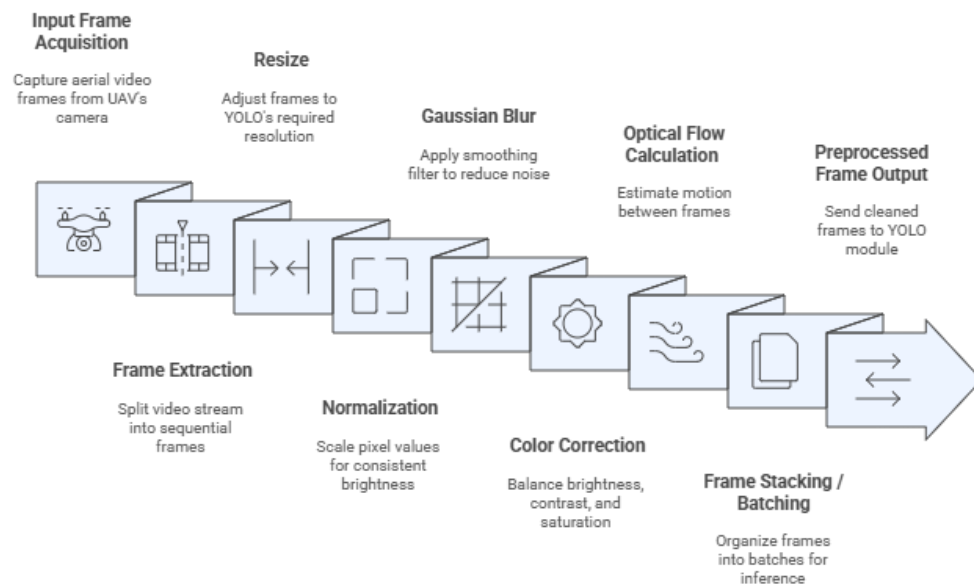


Figure 54. Preprocessing Pipeline for the Ru'ya Drone-Based Crowd Monitoring System

3. AI Inference (YOLO-based):

The YOLO model performs real-time object detection to identify individuals within each frame. Bounding boxes and confidence scores are generated, and crowd density is estimated by aggregating detections over a grid-based spatial map.



4. Postprocessing and Data Handling:

Detection outputs (bounding boxes, counts, and density metrics) are formatted into

Figure 55. YOLO-based detection output

structured JSON objects and sent to the backend for aggregation. Data is logged in database, supporting both historical analysis and real-time updates.

5. **Visualization and Alerts:**

The dashboard module visualizes heatmaps, charts, and real-time statistics using HTML/CSS and JavaScript. Threshold-based alerts are triggered if crowd size or density surpass safety limits, aiding immediate decision-making by operators.

6. **Concurrency and State Management:**

- Multithreaded architecture allows video capture, model inference, and dashboard updates to run in parallel.
- Mutex locks prevent race conditions in shared memory buffers.
- State transitions follow: *Idle* → *Capturing* → *Processing* → *Alerting* → *Logging*.

7. **Initialization and Cleanup:**

At startup, YOLO model weights are loaded, GPU resources are allocated, and network connections are verified. On termination, buffers are flushed, logs are saved, and GPU memory is released to ensure graceful shutdown.

8. **Exception Handling:**

The system includes mechanisms for managing dropped frames, model load errors, and connectivity interruptions. A retry queue and timeout-based recovery prevent critical failures during live operations.

8.7Interface/Exports

8.7.1 Login: Table 43 presents the description, inputs, outputs, and constraints of the “Login” component.

Table 43. Description of “Login” Component

Description	Allows users (operators or administrators) to securely access the Ru’ya Monitor system.
Input	Username and Password
Output	Access token / session initialized, user redirected to dashboard
Constraints	Password must meet security requirements; account locked after multiple failed attempts

8.7.2 Dashboard: Table 44 presents the description, inputs, outputs, and constraints of the “Dashboard” component.

Table 44. Description of “Dashboard” Component

Description	○ Displays real-time UAV video feed with detection overlays. ○ Shows live mission statistics such as crowd density, detected people, and UAV status. ○ Provides immediate visual alerts for density threshold breaches
Input	Real-time data from backend APIs
Output	Processed frames with bounding boxes and updated numerical indicators
Constraints	Stream latency must remain under 3 seconds and requires active UAV connection

8.7.3 Analytics: Table 45 presents the description, inputs, outputs, and constraints of the “Analytics” component.

Table 45. Description of “Analytics” Component

Description	○ Provides detailed analytics of crowd behavior and UAV missions. ○ Displays density trends, detection counts, and flight statistics over time. ○ Allows exporting charts and reports.
Input	Historical mission data from PostgreSQL/Firebase
Output	Graphs, charts, and summary metrics (peak density, average density, total detections)
Constraints	Data must be synchronized from backend before visualization; limited to authorized users

8.7.4 Alerts: Table 46 presents the description, inputs, outputs, and constraints of the “Alerts” component.

Table 46. Description of “Alerts” Component

Description	○ Monitors real-time data for crowd density thresholds and UAV status. ○ Triggers visual notifications when unsafe levels are detected.
Input	Detection results and defined threshold values
Output	Alert notifications displayed on dashboard
Constraints	Alert frequency adjustable (minimum 5 seconds)

8.7.5 Flight Control: Table 47 presents the description, inputs, outputs, and constraints of the “Flight Control” component.

Table 47. Description of “Flight Control” Component

Description	○ Enables the operator to manage UAV missions (launch, land, stop). ○ Displays real-time UAV telemetry including altitude, GPS, and battery level. ○ Supports emergency command execution.
Input	UAV command requests (start, stop, emergency landing)
Output	Mission status confirmation and live telemetry updates
Constraints	Requires stable communication with UAV and operator authentication required for commands

8.7.6 Settings: Table 48 presents the description, inputs, outputs, and constraints of the “Settings” component.

Table 48. Description of “Settings” Component

Description	○ Allows configuring detection parameters and alert frequencies. ○ Provides encryption and log retention options for data security. ○ Displays CPU, memory, and storage performance stats.
Input	User selections for thresholds, quality settings, retention duration
Output	Updated configuration saved to backend
Constraints	Only admin-level users can modify global configurations and data must be validated before submission

8.7.7 Logs: Table 49 presents the description, inputs, outputs, and constraints of the “Logs” component.

Table 49. Description of “Logs” Component

Description	○ Displays system and mission logs for monitoring and troubleshooting. ○ Allows filtering by log type (Error, Warning, Info, Flight, etc.). ○ Supports exporting logs for offline analysis.
Input	Filter type (e.g., Error, System, Flight)
Output	○ Log entries displayed in chronological order ○ export file in CSV or JSON
Constraints	Logs stored securely for limited retention period

8.7.8 Admin Control: Table 50 presents the description, inputs, outputs, and constraints of the “Admin Control” component.

Table 50. Description of “Admin Control” Component

Description	○ Provides administrative access for managing users, licenses, and system updates. ○ Allows viewing and modifying operator roles and privileges. ○ Displays software version and system health overview.
Input	Admin credentials and configuration commands
Output	Confirmation of applied updates or role modifications
Constraints	Restricted to admin accounts, requires secure connection and authentication

9. Other Design Features

This section describes additional and prospective design features that were not fully covered in the previous sections. These features reflect future improvements and ethical considerations that will enhance the Ru'ya UAV Monitoring System's performance, security, and societal impact.

1. Data Security and Encryption

Future iterations of the system will integrate strong encryption techniques (e.g., AES-256 or RSA) to secure data transmission between UAVs, servers, and dashboards. End-to-end encryption will ensure that sensitive telemetry, video streams, and user credentials are protected from interception or tampering. Additionally, authentication tokens will be rotated periodically to prevent replay attacks and unauthorized access.

2. Ethical and Privacy Protection

To comply with privacy and ethical AI standards, future updates will include an automatic face-blurring module to anonymize individuals captured in drone footage. This will safeguard personal identities and align with data protection policies such as Saudi Data & AI Authority

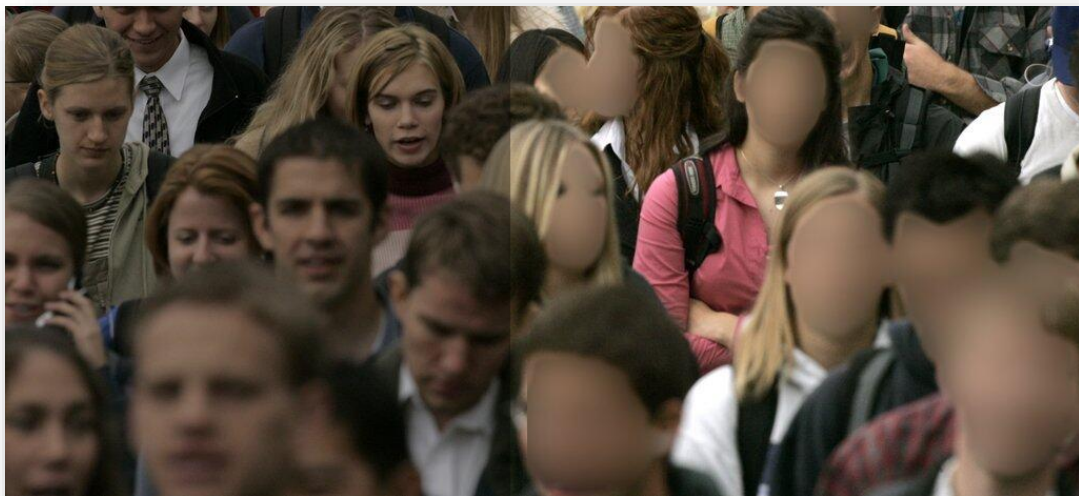


Figure 56. Example of automatic face-blurring for privacy protection in crowd monitoring systems Reproduced from [50]

(SDAIA) ethical guidelines.

3. Thermal and Night Vision Integration

The system could be expanded to support thermal and low-light imaging for nighttime monitoring or emergency scenarios (e.g., crowd evacuation or rescue). This enhancement would extend usability and safety in diverse operational environments.

4. **Modular Architecture**

The system is designed so models, sensors, and analytic modules can be replaced or upgraded independently without affecting the entire pipeline.

5. **Battery Health Monitoring**

Automated checks on drone battery temperature, discharge rate, and flight time prediction to avoid mid-air power failures.

6. **Real-Time Alert System**

Integration of an intelligent event-triggering engine that sends alerts (SMS, email, dashboard notifications) when anomalies or unsafe crowd densities are detected.

7. **Customizable User Interface**

The operator dashboard supports customizable widgets and themes support to adapt to different users and operational sectors.

8. **Event Logging & Auditing**

Complete logging of flight paths, user actions, detections, and system decisions for audits, debugging, and safety validation.

9 Requirements Traceability Matrix

The Requirements Traceability Matrix (RTM) provides a mapping between the functional requirements defined in the System Requirements Specification (SRS) and the corresponding design elements described in this System Design Description (SDD). This ensures that every requirement is fully addressed by one or more components of the system architecture. A summary of the traceability is provided on table 51.

Table 51. Traceability Matrix

SRS Requirement (FR#)	Requirement Description	Design Component(s) in SDS	Verification Method
FR1 – Flight Control	UAV must support autonomous/semi-autonomous flight operations.	• UAV Firmware & Flight Controller • Flight Control Center (Dashboard) • Communication Interface (Wifi / TCP/IP)	Simulation Testing, System Testing
FR2 – Aerial Data Acquisition	UAV must capture live video/images.	• Camera Module • Data Acquisition Layer • UAV–Dashboard Link	Functional Testing
FR3 – AI-Based Crowd Detection	System must detect people and estimate density in real time.	• AI Crowd Detection Module (YOLO)	Unit Testing, Model Validation

		Onboard AI Preprocessing Layer	
FR4 – Data Transmission	AI results and video must be sent to dashboard with low latency.	- REST API Service - WebSocket Streaming - Communication Interface (AES-256 encryption)	Integration Testing
FR5 – Dashboard Analytics	System must display density, alerts, charts, mission logs.	- Dashboard Front-End (Analytics Panel) - Backend Database (Mission & Metrics) - Visualization Engine	UI Testing, System Testing
FR6 – Alert System	System must alert when crowd threshold is exceeded.	- Alert Management Module - Threshold Engine - Notification Service	Functional Testing
FR7 – Data Storage	Store logs, mission data, historical density metrics.	- Cloud/Local Database - Log Storage Module - Sync Service	Inspection, Database Testing
FR8 – User Authentication	Only authorized users access the system.	- Login & Authentication Module (Token-based) - Security Layer (2FA optional)	Security Testing
FR9 – UAV Telemetry Display	Display UAV GPS, altitude, speed, battery, connection.	- UAV Telemetry Parser - Flight Control Center (UI) - Communication Layer	System Testing
FR10 – Emergency Controls	User can trigger RTH, Hover, Emergency Land.	- Flight Safety Module - Manual Control Handler - UAV Firmware Commands	Simulation Testing
FR11 – System Settings Management	User can configure thresholds, video quality, alerts.	- Settings Module - Configuration Database - Admin Privilege Manager	Functional Testing
FR12 – Logs and Audit Trails	System must store all events, warnings, errors.	- Logs Module - Cloud Sync - Admin Dashboard	Inspection & Review
FR13 – Multi-Role Access	Admin and Operator roles with different permissions.	- RBAC (Role-Based Access Control) - Authorization Middleware	Security & Role Testing
FR14 – Real-Time UI Updates	Dashboard must update within 5 seconds.	- WebSocket Real-Time Engine - Front-End Event Listener	Load Testing
FR15 – UAV Path Tracking	Show real-time UAV path on a map.	- GPS Telemetry Processor - Map Component (Leaflet/Mapbox) - Dashboard UI	Simulation Testing

Chapter 6: Implementation Phase

1. Introduction

The Ru'ya project focuses on developing a lightweight artificial intelligence (AI) system integrated with unmanned aerial vehicles (UAVs) for real-time crowd detection and monitoring. This project is designed to address the growing need for efficient crowd management solutions in large-scale events, especially in the context of Saudi Arabia's Vision 2030, which focuses on smart technologies and public safety enhancements.

With the increasing number of mass gatherings, such as upcoming international events like the FIFA World Cup 2034, traditional monitoring methods such as CCTV and manual surveillance have become insufficient due to limited coverage, blind spots, delayed response times, and a lack of adaptability, as illustrated in figure 57. These limitations highlight the importance of an intelligent system capable of providing real-time insights and more situational awareness.



Figure 57. Comparison between CCTV (left) and UAV drone (right) views for crowd monitoring reproduced from [51]

In response to the necessity for advanced intelligent systems, the Ru'ya system introduces a UAV-based solution that leverages computer vision and deep learning techniques to detect and analyze crowds dynamically. The system captures aerial data through onboard cameras, processes it using a lightweight AI model, and delivers real-time monitoring outputs to support decision-making. The main focus of Ru'ya implementation is ensuring that the AI model is efficient enough to run on embedded hardware, enabling on-device processing without reliance on high computational resources.

This mid-term implementation report presents the progress achieved so far in developing the Ru'ya system. It outlines the system architecture, datasets used, model development, and preliminary implementation results. Additionally, it highlights the challenges encountered and the strategies adopted to address them, including considerations for performance optimization, real-time processing, and privacy preservation.

2. Summary of the proposal

The Ru'ya project proposes the development of a lightweight AI-powered UAV by maintaining a smaller number of parameters and using quantization techniques for real-time crowd detection and monitoring. The main goal is to enhance public safety and improve crowd management during large-scale events by providing accurate real-time insights.

The proposed system integrates unmanned aerial vehicles (UAVs) with computer vision and deep learning techniques to detect people and estimate crowd density. Unlike traditional surveillance systems, which rely on fixed cameras and manual monitoring, this solution offers dynamic aerial coverage, faster response times, and improved situational awareness.

A key aspect of the proposal is the use of a lightweight AI model that can operate efficiently on embedded hardware within the drone. This reduces the dependency on high-performance external computing resources and enables real-time processing directly onboard the UAV. Our Ru'ya system is designed to function across different environments and conditions, to ensure robustness.

In addition, the proposal considers ethical and regulatory aspects, including privacy preservation and compliance with UAV operation regulations in Saudi Arabia. Techniques such as data minimization and anonymization are incorporated to ensure responsible use of collected data.

Overall, the Ru'ya proposal aims to deliver an intelligent, reliable and efficient crowd monitoring solution that aligns with Saudi Vision 2030 by supporting smart cities initiatives and enhancing safety during mass gatherings.

3. Changes from the proposal phase and justifications

During the implementation phase of the Ru'ya project, several adjustments were made to better align the system to real-world UAV deployment making it more practical in real scenarios. Initially, the proposal phase mainly focused on the detection accuracy; However, we found that real-time performance is equally critical specifically in edge devices. Hence, inference time and computational efficiency metrics are incorporated in this phase to evaluate the model's capabilities. Another significant change involved the dataset selection; the final implementation focused primarily on the VisDrone dataset. This choice was made after exploring and testing multiple different datasets in the proposal phase. The VisDrone dataset provides real-world UAV imagery with diverse conditions and is well-suited for crowd detection tasks. Using a single, dataset also allowed us for more controlled experiments and clearer analysis of model performance. Additionally, the preliminary experiments showed that the performance of the model is highly dependent on the UAV flight altitude since altitude directly affects the crowd scale and the general details of the video frame, which impacts the model's ability to detect crowds. These observations weren't discussed in the proposal phase, so we examined how detection performance varies with UAV altitude to better read and interpret the results. Finally, the model selection process was refined. Instead of broadly considering multiple models, the final system uses YOLOv8s as the primary model due to its lightweight architecture and suitability for real-time inference on edge devices. Heavier variants, such as YOLOv8m, was explored and tested in the proposal phase but were not selected in the implementation due to their higher computational cost and reduced efficiency in real-time scenarios.

Overall, these changes reflect a shift from a broader, accuracy-focused proposal to a more focused and deployment-oriented system that prioritizes efficiency, consistency, and real-world applicability.

4. Implementation Progress

This section describes the implementation of the proposed UAV-based crowd monitoring system, including data preparation, model development, crowd density estimation, and real-time deployment pipeline.

4.1 System Overview

The implemented system processes aerial video frames captured from a UAV and performs real-time crowd analysis. The pipeline consists of person detection using a deep learning model, followed by spatial partitioning, density estimation, and risk classification. The overall system pipeline is illustrated in Figure 58, which summarizes the main stages involved in processing UAV imagery for crowd monitoring and risk assessment.

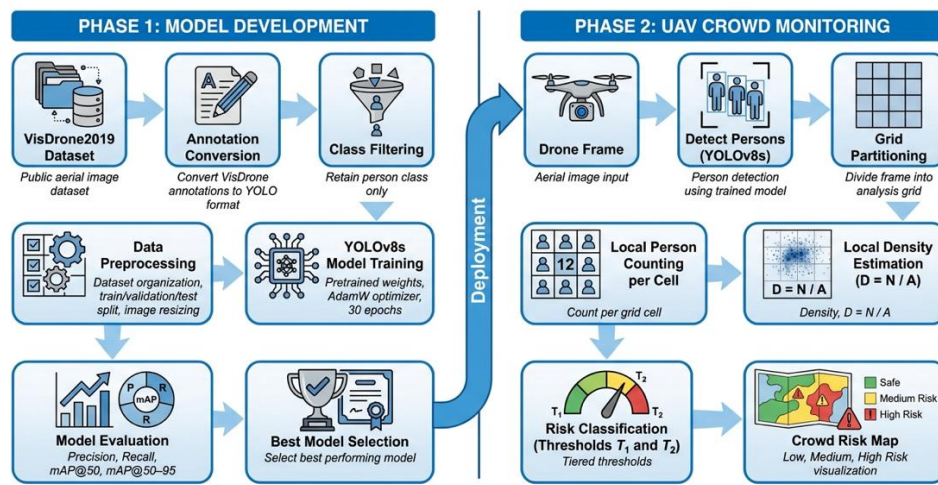


Figure 58: Overall system pipeline for UAV-based crowd detection and risk assessment

4.2 Dataset and Preprocessing

The performance of the proposed UAV-based crowd monitoring system depends heavily on the quality and suitability of the dataset used for training the detection model. Therefore, careful selection and preprocessing of the dataset were essential steps in the implementation.

4.2.1 Dataset Selection

The VisDrone dataset was utilized for training and evaluating the proposed UAV-based crowd monitoring system. The dataset was obtained from Kaggle and is based on the VisDrone2019-DET benchmark [53], which contains aerial images captured from unmanned aerial vehicles (UAVs) under real-world conditions. The dataset consists of 8,629 images.

This dataset is particularly suitable for the task due to:

- The top-view aerial perspective consistent with drone-based monitoring
- The presence of small-scale human instances
- Real-world variations in lighting, altitude, and scene complexity

Although the dataset includes multiple object categories, only the person class was used in this project to focus specifically on crowd detection and density estimation.

4.2.2 Data Preprocessing

Several preprocessing steps were applied to prepare the dataset for training and ensure compatibility with the YOLOv8 model.

- **Image Resizing:**

Images were processed at their original resolution during preprocessing. During training, all images were resized to 1280×1280 pixels, as specified in the YOLOv8 configuration.

- **Annotation Conversion and Class Filtering:**

The VisDrone dataset annotations were originally provided in a custom format (x, y, width, height, score, class_id, truncation, occlusion).

In contrast, YOLOv8 requires annotations in the format (class_id, x_center, y_center, width, height) with normalized coordinates. Therefore, a preprocessing script was used to convert the original annotations into the YOLO format and retain only the person class for crowd detection.

The conversion process included:

- Transforming bounding box coordinates from (x, y, w, h) into the normalized YOLO format $(x_{center}, y_{center}, width, height)$

Computing the center coordinates and normalized dimensions using:

$$x_{center} = (x + w/2) / W$$

$$y_{center} = (y + h/2) / H$$

$$width = w / W$$

$$height = h / H$$

where W and H denote the image width and height, respectively. This normalization converts pixel-based coordinates into values between 0 and 1, which are required by the YOLOv8 framework.

- Normalizing all coordinates based on the corresponding image dimensions
- Filtering annotations to retain only the person class
- Discarding all other object categories to simplify the detection task and improve model focus

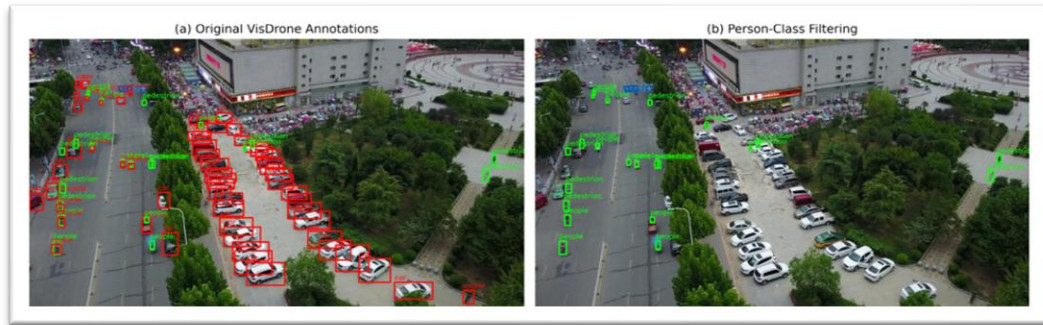


Figure 59: Example of class filtering applied to the VisDrone dataset.

Figure 59 shows the class filtering process applied to the VisDrone dataset. While the original annotations contain multiple object categories, only person-related classes were retained for training the YOLOv8s crowd detection model.

- **Dataset Splitting:**

The dataset was used following the standard VisDrone split:

- The training set: 6,471 training images was used for model learning
- The validation set: 548 validation images was used to monitor performance during training
- The testing set: 1,610 testing images was reserved for evaluation

This predefined split ensures fair and consistent performance assessment.

- **Data Augmentation:**

Data augmentation was explicitly configured during training using the YOLOv8 framework to improve model generalization. The following techniques were applied:

- Mosaic augmentation (probability = 1.0)
- MixUp augmentation (probability = 0.1)
- Random horizontal flipping (probability = 0.5)
- Scaling ($\pm 50\%$)
- Rotation (± 5 degrees)
- HSV color augmentation:
 - Hue = 0.015
 - Saturation = 0.7
 - Value = 0.4

These augmentations enhance the model's robustness to variations in scale, orientation, and lighting conditions commonly found in UAV imagery.



Figure 60. Sample images from the VisDrone2019 dataset showing small-scale human instances in aerial scenes

4.3 Person Detection Model

The core component of the proposed system is the person detection module, which is responsible for identifying individuals in aerial images. This module is implemented using the YOLOv8s (You Only Look Once version 8 - small) object detection model.

4.3.1 Model Selection

The YOLOv8s model was selected due to its efficiency and suitability for real-time applications. Since the system is intended to run on resource-constrained edge devices such as the Jetson Nano, a lightweight model is required.

The selection of YOLOv8s was based on the following advantages:

- Lightweight architecture, enabling faster inference
- Real-time performance, suitable for live UAV video processing
- Good balance between accuracy and speed
- Compatibility with edge deployment environments

Heavier models such as YOLOv8m were not selected, as they introduce higher computational cost and reduce real-time performance.

4.3.2 Model Training

The YOLOv8s model was trained using the preprocessed VisDrone dataset, focusing exclusively on the person class.

The training process involved:

- Configuring the model using the YOLOv8 framework
- Loading pretrained weights to accelerate convergence
- Training on aerial images containing small-scale human instances

- Monitoring training and validation performance to ensure proper learning

The model is trained to detect individuals by predicting bounding boxes around human instances in each input image.

Training Configuration:

The training process was explicitly configured as summarized in table 52:

Parameter	Value
Model	YOLOv8s
Number of Classes	1 (Person)
Input Image Size	1280 × 1280
Epochs	30
Batch Size	8
Optimizer	AdamW
Initial Learning Rate	0.0008
Mosaic Augmentation	1.0
MixUp Augmentation	0.1
Rotation (Degrees)	$\pm 5^\circ$
Scale Factor	0.5
Horizontal Flip Probability	0.5
HSV Hue	0.015
HSV Saturation	0.7
HSV Value	0.4
Pretrained Weights	YOLOv8s Pretrained Model
Framework	Ultralytics YOLOv8
Hardware	GPU (CUDA device 0)

Table 52: Training Configuration

The training progress and model performance are illustrated in Figures 61-65.

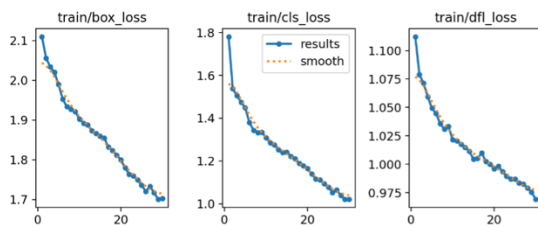


Figure 62 Training loss

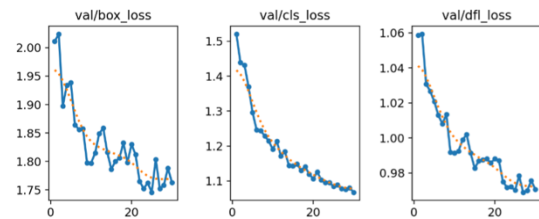


Figure 61 Validation loss

Figures 60 and 61 demonstrate a consistent decrease in both training and validation losses, indicating successful model convergence and stable learning behaviour.

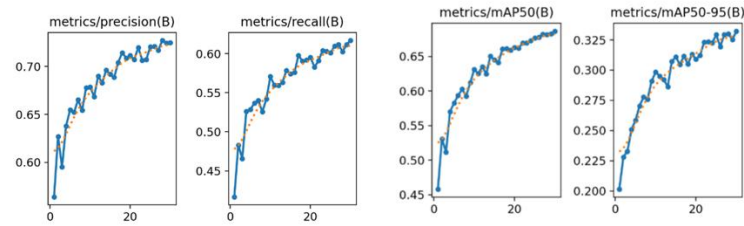


Figure 63: Performance metric curves

Figure 63 shows improvements in precision, recall, and mAP values throughout training, demonstrating the effectiveness of the proposed YOLOv8s model for person detection.

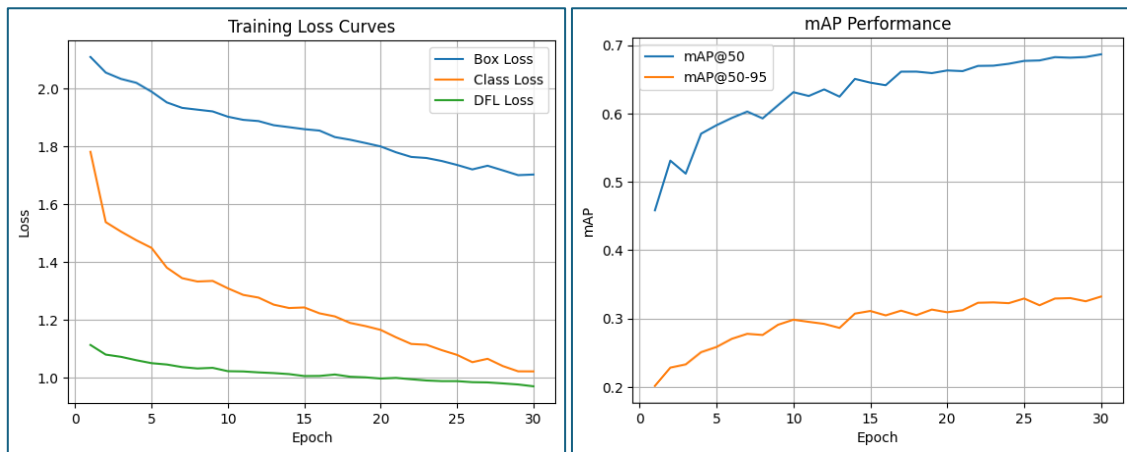


Figure 64: Training loss curves.

Figure 65: Mean Average Precision (mAP) curves.

Figure 64 illustrates the behavior of individual loss components, all of which decrease steadily, indicating that the model is learning meaningful features.

Figure 65 shows that both mAP@50 and mAP@50–95 improve throughout training, demonstrating enhanced detection accuracy.

Overall, the training results demonstrate that the model achieves a good balance between accuracy and efficiency, making it suitable for real-time UAV-based crowd monitoring applications.

4.3.3 Model Evaluation

The performance of the trained model was evaluated using standard object detection metrics. The achieved results are as follows:

- **Precision:** 72.76%
- **Recall:** 61.68%
- **mAP@50:** 68.94%
- **mAP@50-95:** 33.52%

These results indicate that the model is capable of detecting individuals in aerial scenes with acceptable accuracy for real-time UAV-based applications. However, the recall value suggests

that some small or densely packed individuals may not be detected, which is a common challenge in UAV-based imagery.

4.3.4 Inference Performance

In addition to detection accuracy, inference speed is critical for real-time applications. The model achieved:

- **Average inference time:** 23 ms per image
- **Video processing speed:** approximately 34.87 FPS

These results demonstrate that the model is suitable for near real-time crowd monitoring, meeting the performance requirements of the system.



Figure 66: Detection results of the YOLOv8s model showing identified individuals in aerial imagery with bounding boxes and confidence scores

4.3.5 Qualitative Failure Case Analysis

Although the trained YOLOv8s model achieved acceptable quantitative performance on the VisDrone validation set (Precision = 72.76%, Recall = 61.68%, mAP@50 = 68.94%, and mAP@50-95 = 33.52%), qualitative inspection shows that the model still has limitations in visually complex UAV scenes. These limitations are most visible when people appear very small, are partially occluded by other people or scene objects, or are captured under degraded visibility conditions such as low light, fog, rain, or dust. Since the proposed system uses detection outputs as the basis for crowd counting, grid-based density estimation, and risk classification, missed detections can directly reduce the estimated crowd density in affected grid cells.

Table 53: Observed Failure Patterns

Case	Visual condition	Observed limitation	Effect on system output
Small or distant people	High-altitude or wide field-of-view UAV frames.	Some individuals become too small for stable localization, especially near image edges or in visually cluttered regions.	Recall decreases and the grid-based person count may underestimate local crowd density.
Dense or overlapping crowds	People appear close together, partially occluded, or visually merged.	Bounding boxes can overlap heavily; some people are missed, while others receive low-confidence predictions.	Crowd density may be smoothed or shifted into neighboring cells instead of reflecting the real distribution.
Low visibility	Fog, dust, rain, and low-light frames reduce contrast between people and the background.	The detector produces fewer confident detections and may miss people whose outlines blend into the scene.	Risk classification can become less reliable under difficult environmental conditions.

Example A: Dense-scene detection output:

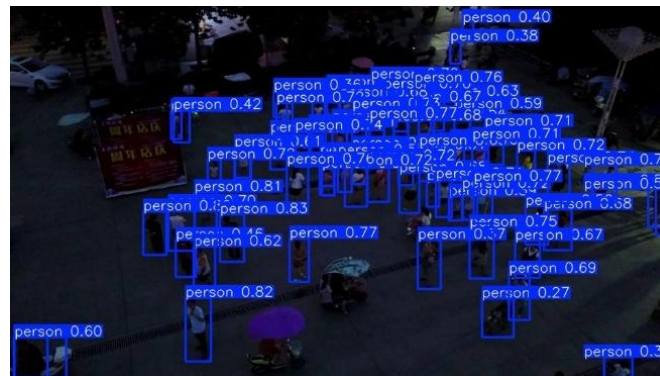


Figure 67: Dense aerial imagery creates many overlapping detections. In such cases, very small or partially occluded people may be missed, which is consistent with the model's moderate recall value.

Example B: Altitude-related detection loss:

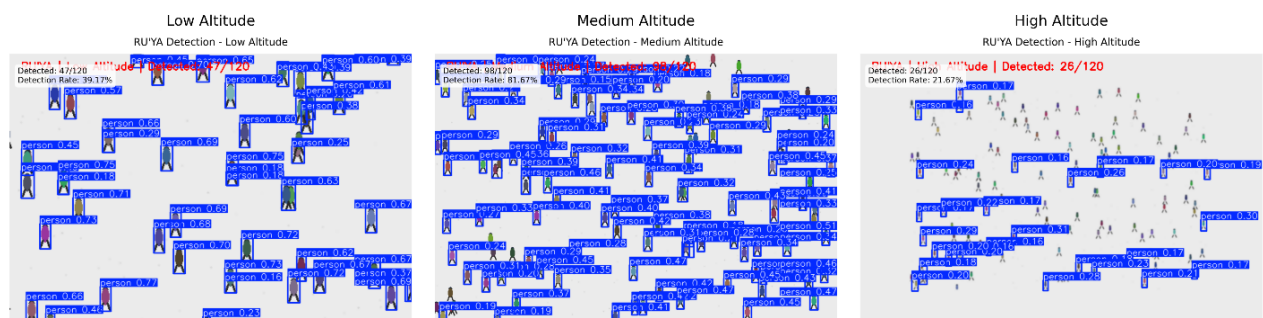


Figure 68: Detection examples across altitude levels show the trade-off between scene coverage and object detail. At high altitude, people occupy fewer pixels, making missed detections more likely.

Example C: Low-light and degraded visibility:

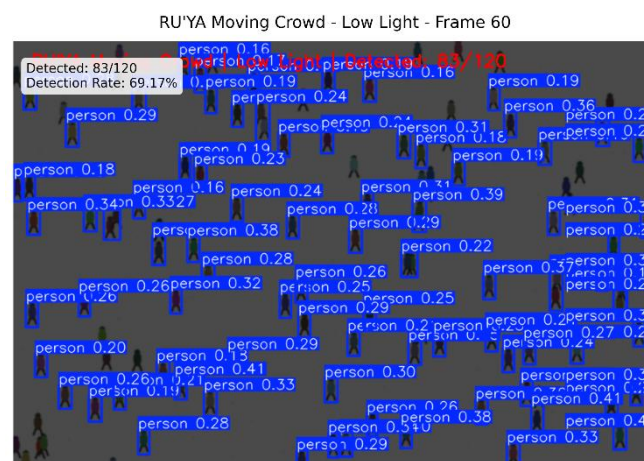


Figure 69: Low-light conditions reduce visual contrast and lead to weaker confidence scores or missed detections, especially for small people in crowded regions.

Example D: Weather-based performance trend:

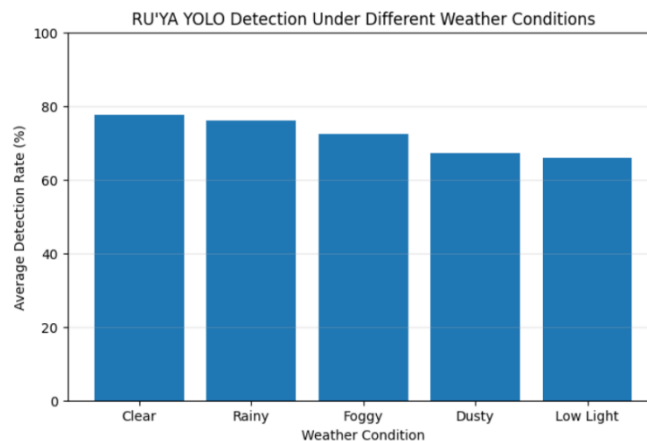


Figure 70: Weather-based evaluation confirms that clear scenes achieve the highest detection rate, while dusty and low-light conditions reduce the average detection rate.

The failure cases indicate that YOLOv8s provides a practical balance between speed and accuracy for the Ru'ya pipeline, but its predictions are less reliable when the visual evidence for each person is weak. The recall value of 61.68% supports this observation because it means the system detects many individuals but still misses a meaningful portion of ground-truth persons in difficult frames. For real-time crowd monitoring, this limitation should be considered when interpreting density maps and safety alerts. A conservative operational approach is to treat low-visibility or high-altitude outputs as lower-confidence estimates rather than exact crowd counts.

Future improvements should focus on training with more challenging examples, including high-altitude, night-time, foggy, rainy, and heavily occluded crowd scenes. Additional strategies include multi-scale training, higher-resolution inference for dense frames, temporal smoothing across consecutive video frames, and confidence-aware alert thresholds that account for environmental conditions.

4.4 Crowd Density Estimation

Following the detection of individuals using the YOLOv8s model, a crowd density estimation module was implemented to analyze the spatial distribution of people within each frame. This module enables localized crowd analysis rather than relying solely on total person count.

4.4.1 Grid-Based Partitioning

To estimate crowd density, each input frame is divided into a uniform 6×6 grid, resulting in 36 equal-sized cells. This spatial partitioning allows the system to analyze different regions of the scene independently.

The frame is divided into a 6×6 grid structure. Each detected individual is assigned to a grid cell according to the center point of the bounding box. This enables localized density estimation instead of relying solely on a global crowd count.



Figure 71: Grid-based partitioning of the input frame into 6×6 cells for localized density estimation

4.4.2 Local Person Counting

For each grid cell $G_{i,j}$, the number of detected individuals is computed. Let:

- $N_{i,j}$: number of persons detected in cell $G_{i,j}$

The number of detected individuals is calculated independently for each grid cell, allowing the system to identify localized crowd concentrations and potentially congested regions.

4.4.3 Density Computation

The crowd density for each grid cell is calculated using the following formula:

$$D_{i,j} = \frac{N_{i,j}}{A_{cell}}$$

Where:

- $D_{i,j}$: density of cell $G_{i,j}$
- $N_{i,j}$: number of detected individuals in the cell
- A_{cell} : area of the grid cell (in pixel units)

Since camera calibration parameters are not available, the density is computed using pixel-based area, resulting in a relative density measure rather than an absolute real-world value.

4.4.4 Interpretation of Density Values

The computed density values provide insight into how crowded each region of the frame is. Higher values indicate areas with a greater concentration of individuals.

This localized approach enables the system to:

- Identify crowded regions within the scene
- Support risk assessment based on density distribution
- Improve decision-making compared to using only total crowd count

4.5 Risk Classification

Following the estimation of crowd density for each grid cell, a risk classification module was implemented to assess the safety level of different regions within the scene. This step translates numerical density values into meaningful categories that can support real-time decision-making.

4.5.1 Risk Classification Criteria

Each grid cell is assigned a risk level based on the number of detected individuals within that cell. Let N represent the number of people detected in a given grid cell.

The classification is defined as follows:

- **Low Risk:** $N < 2$
- **Medium Risk:** $2 \leq N < 4$
- **High Risk:** $N \geq 4$

These thresholds were selected empirically based on observed crowd distributions in the dataset and experimental results.

4.5.2 Risk Mapping and Visualization

After determining the risk level for each grid cell, the results are visualized directly on the video frame. Each cell is color-coded according to its risk category:

- **Green:** Low Risk

- **Yellow:** Medium Risk
- **Red:** High Risk

Additionally, textual labels (“LOW”, “MEDIUM”, “HIGH”) are displayed within each grid cell to clearly indicate the classification outcome.

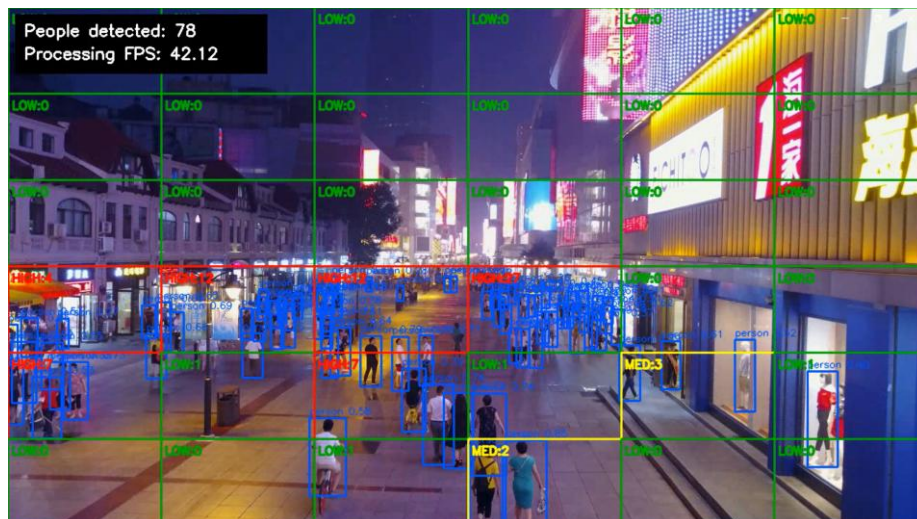


Figure 72: Crowd risk visualization showing grid-based partitioning with low-risk regions highlighted across the frame

4.5.3 Interpretation of Results

The risk classification output provides a clear representation of crowd distribution across the monitored area. Regions with higher density are easily identifiable through color-coded visualization, enabling rapid assessment of potential crowding risks.

In the current implementation, most regions in the sample output are classified as low risk, indicating sparse crowd presence. However, the system is capable of detecting higher-risk zones when crowd density increases.

This approach enhances situational awareness compared to traditional counting methods, as it highlights not only the number of individuals but also their spatial concentration.

4.6 Hardware Setup and Integration

4.6.1 Hardware Assembly

In this section, we will walk through the process of both the hardware construction and system configuration. Figure 69 presents an overview of the final Ru’ya UAV platform and its main components including the F450 frame, GPS module, brushless motors, electronic speed controllers (ESCs), flight controller, telemetry module, RC receiver, and onboard camera.

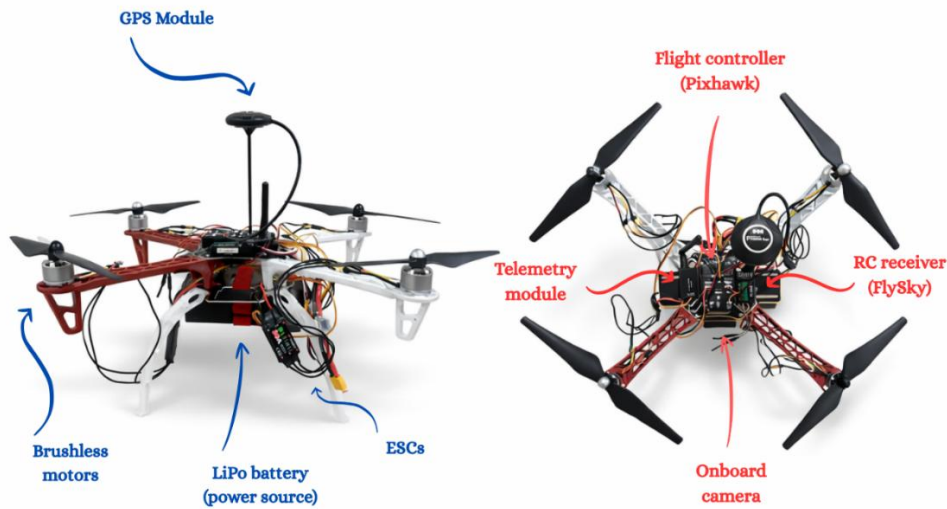


Figure 73. Final Ru'ya UAV system overview.

1. Hardware Construction

1.1. Mechanical Assembly and Power Integration

This subsection covers the initial setup of the drone, where the main structure is built and the power components are connected. We started by soldering the ESCs to the **power distribution board (PDB)** shown in figure 70, to handle power delivery.

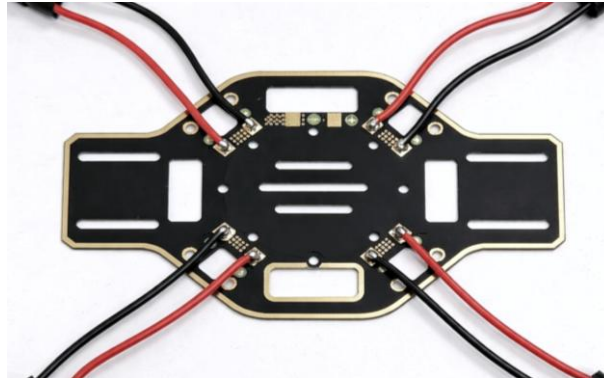


Figure 74. ESCs Soldered to the Power Distribution Board (PDB)

After that, we assembled the drone frame and secured all its parts such as the **arms** and the **legs**. Once the frame was ready, we mounted the **motors** on each arm and fixed them using screws. Then, we soldered bullet connectors to the motor wires and connected them to the ESCs, as shown in Figure 71. Finally, we secured the **battery** under the frame using straps to keep it stable during operation.



Figure 75. Bullet connectors soldered to motor wires for connection with the ESC.

1.2. Flight Controller and Electronics Integration

This subsection focuses on integrating the main control and communication components of the drone. We started by mounting the **Pixhawk** flight controller at the centre of the board to ensure stability and accurate sensing. Then the **GPS** module was mounted on an elevated stick to improve signal reception and ensure more accurate readings. Also, it is worth noting that both the GPS and the Pixhawk were placed to face forward to maintain correct direction, as shown in Figure 72.

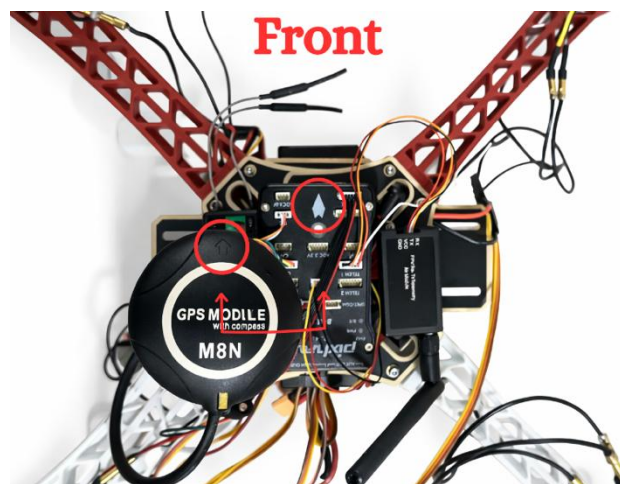


Figure 76. GPS and Pixhawk aligned facing forward.

Then, the **ESC** signal wires were connected to the Pixhawk to allow control of the motors. After that we connected the **radio receiver** to enable communication with the remote

transmitter, and a **camera module** was integrated into the drone to capture visual data, which is used later for processing. also, the **telemetry module** was installed to allow real-time communication with Mission Planner.

1.3.Safety and Final Components

This subsection covers the final added components to complete the drone assembly and ensure safe and smooth operation. We installed the **buzzer** and **safety switch** to provide alerts and allow safe testing. After completing all wire connections, the propellers were attached as the final step of the assembly. To make sure everything was working properly, we first charged the Lipo battery, then connected it to power the system. Once powered, we observed that all components turned on successfully, as indicated by the lights shown in figure 73, confirming that the connections and soldering were done correctly.



Figure 77. Ru'ya UAV powered on showing active system status

With that, the hardware construction phase was completed, where all components were assembled and connected. The system was powered on to confirm that the setup was functioning at a basic level. We now move on to the system configuration using Mission Planner in the next section.

2. Mission Planner Configuration

In this section, we will show the process of system configuration using Mission Planner, which is an open-source Ground Control Station (GCS) for ArduPilot-based autopilot systems.

2.1. Mission Planner Setup and Connection

We started by installing Mission Planner and connecting the Pixhawk to the laptop using a USB cable. Once connected, the system successfully established communication, and as shown in Figure 74, the interface displays the UAV's live telemetry data, including position, altitude, and system status, confirming that we have a successful communication between the UAV and Mission Planner.

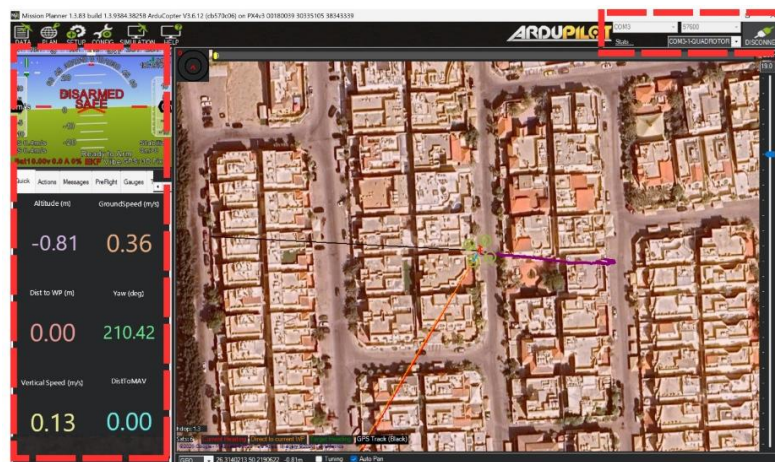


Figure 78. Mission Planner showing UAV connection and GPS location.

2.2. Initial UAV Setup

After connecting the UAV to Mission Planner, the initial setup was completed by selecting the **correct frame type**. Since our system is a quadcopter in **X** configuration, this option was chosen in Mission Planner to match the physical structure of the drone and ensure proper control behaviour.

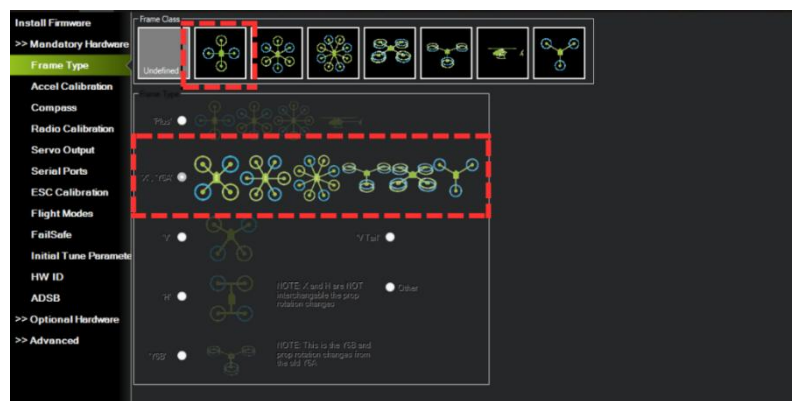


Figure 79. Frame type selection in Mission Planner.

2.3. Sensor and Radio Calibration

2.3.1. Accelerometer Calibration

The accelerometer was calibrated to ensure the UAV can correctly estimate its orientation and maintain stable flight. This was done using Mission Planner by placing the drone in different positions demonstrated in figure 76, as required by the calibration process.

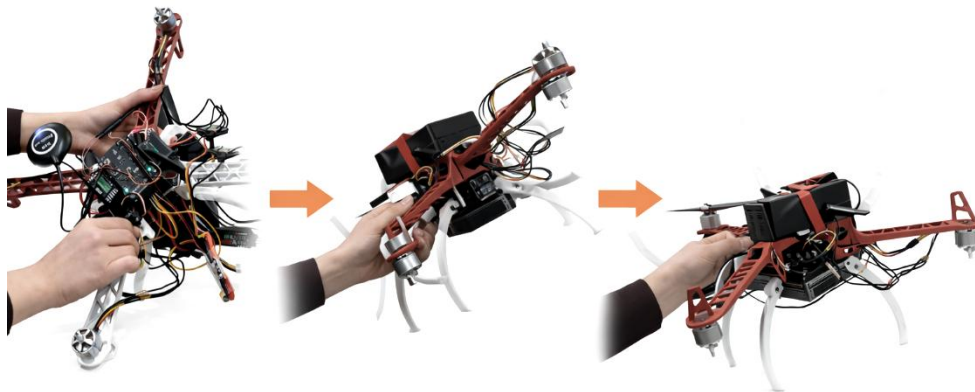


Figure 80. UAV orientation during accelerometer calibration.

After performing the manual accelerometer calibration, we verified the correctness of the UAV's positioning using the Mission Planner interface, as shown in Figure 77. The interface confirmed that the calibration process was successfully completed, indicating that the sensor readings were properly aligned and ready for accurate flight estimation.

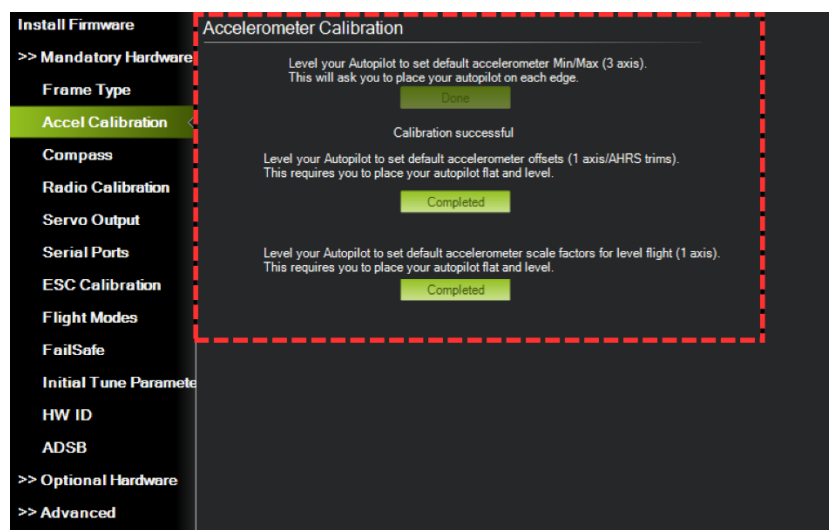


Figure 81. Accelerometer calibration interface in Mission Planner

2.3.2. Compass Calibration

The compass was then calibrated to provide accurate heading information for navigation. This step is important for maintaining correct direction during flight, especially when GPS is used. Figure 78 shows the interface of Mission Planner during Compass calibration.

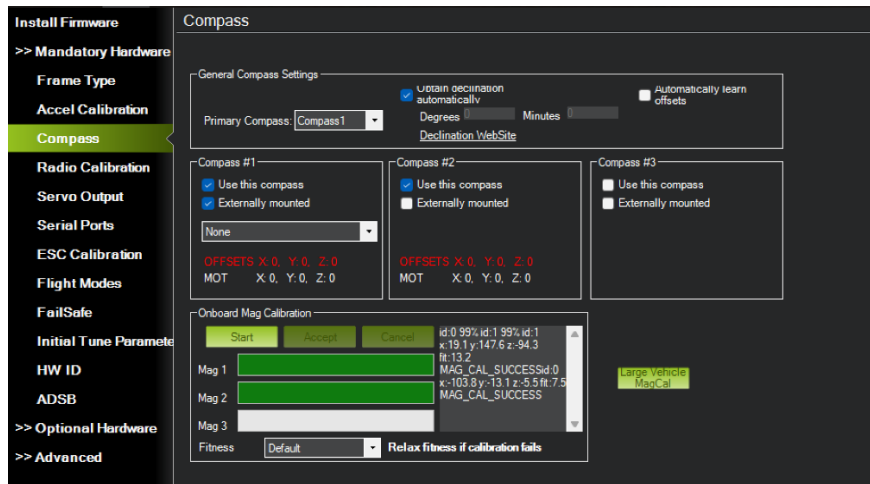


Figure 82. Compass calibration interface in Mission Planner.

To support this, the system interface provides feedback during calibration and helps ensure correct configuration. Figure 79 shows how the movement of the UAV is directly reflected in Mission Planner. The different directions (forward, backward, left, and right) represent how the drone responds to control inputs, while the interface on the laptop displays the corresponding telemetry data and position in real time.



Figure 83. UAV movement directions and telemetry interface.

2.3.3. Radio Calibration

Finally, the radio controller was calibrated to correctly map the transmitter inputs (throttle, roll, pitch, and yaw) to the UAV. This was done by manually moving the controls on the remote while observing the corresponding signals in Mission Planner shown in Figure 80.

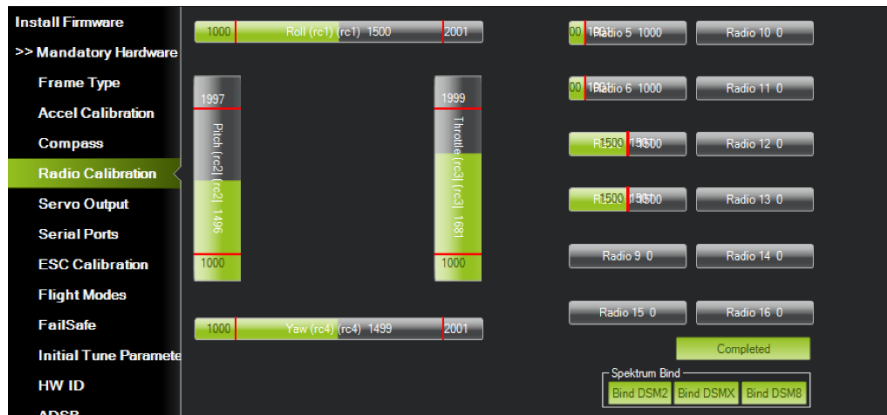


Figure 84. Radio calibration signals in Mission Planner

With these steps completed, the UAV sensors and controls were properly set up. This makes sure the drone can correctly understand its position, direction, and the inputs from the remote, preparing it for the next configuration steps.

2.4. Flight Mode Setup and Motor Testing

In this subsection, we focused on making sure the UAV hardware responds correctly and defining how the drone will behave during operation using Mission Planner.

2.4.1. Mode setup

We began with configuring the flight modes, which define how the UAV behaves during flight. We selected modes such as **Stabilize**, **AltHold**, and **Loiter**. Shown in Figure 81.

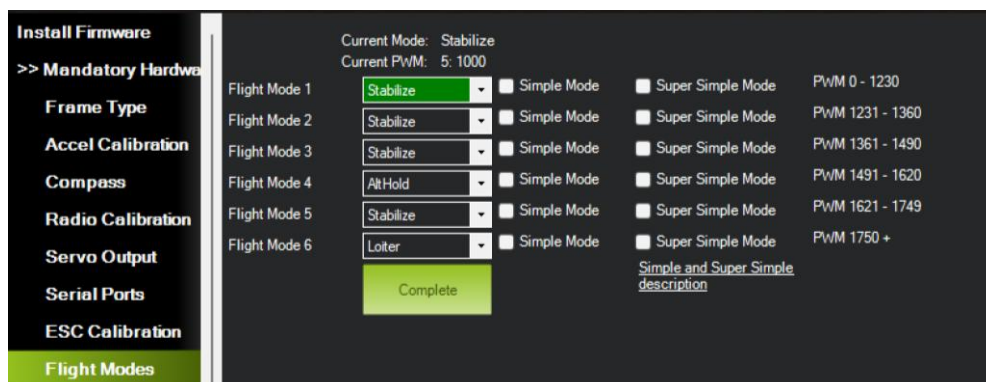


Figure 85. Flight mode configuration in Mission Planner

Stabilize mode allows manual control with basic balancing, AltHold keeps the altitude fixed, and Loiter allows the UAV to hold its position using GPS. These modes were chosen to give us both control and stability depending on the situation.

2.4.2. Motor Testing

In this subsection, we validate the motors and ensure that each motor follows the correct rotation direction, as illustrated in Figure 82. This step is critical because quadrotor stability depends on having alternating clockwise (CW) and counterclockwise (CCW) rotations, which balance the generated torque. Any mismatch in rotation would cause the UAV to spin uncontrollably instead of lifting and maintaining stable flight.

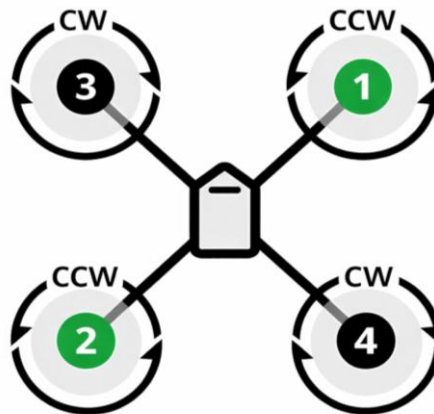


Figure 86. Quadrotor motor rotation directions (CW and CCW)

We tested the motors to ensure they are working properly. This was done using the Motor Test feature, where each motor was activated individually to check that it spins correctly and in the right direction. Figure 83 shows that all motors are in the correct rotation. Hence this step confirms that the motor connections are correct and ready for operation.

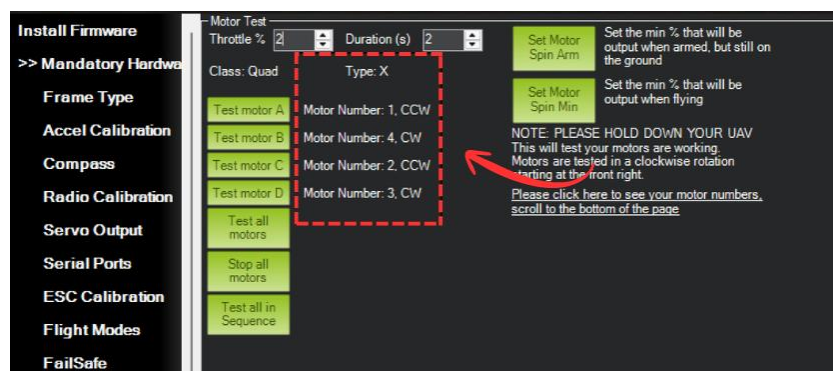


Figure 87. Motor testing interface in Mission Planner

Finally, we checked the system status and controller inputs through the interface. This step was used to confirm that the UAV is connected correctly and that the signals from the remote controller are being received properly. By observing the input values shown in Figure 84 we made sure that all controls are working as expected and that the system is ready for operation.

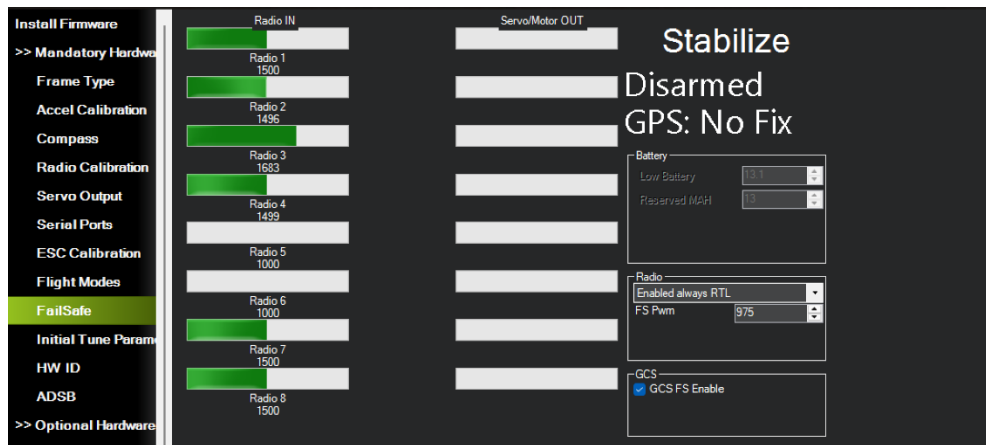


Figure 88. UAV connection and input signals in Mission Planner

Overall, this section covered the hardware construction and system configuration of the UAV. Through this process, we gained hands-on experience in assembling the drone, calibrating the sensors, and configuring it using Mission Planner. This helped us better understand how the system works in practice and how different components interact to achieve stable flight.

5. Issues Faced

During the implementation of the proposed UAV-based crowd monitoring system, several technical and practical challenges were encountered. These issues were addressed throughout the development process and contributed to improving the overall system design.

5.1 Dataset Challenges

One of the main challenges was working with aerial datasets such as VisDrone. The dataset contains complex scenes with small and densely packed objects, making person detection more difficult compared to standard ground-level datasets.

Additionally, filtering the dataset to focus only on the person class required careful preprocessing to ensure compatibility with the YOLOv8 training pipeline.

5.2 Model Training and Performance

Training the YOLOv8s model required careful parameter tuning to achieve satisfactory detection performance. Challenges included:

- Detecting small-scale individuals in aerial views
- Handling occlusions in crowded scenes
- Balancing detection accuracy and computational efficiency

Ensuring stable training and avoiding overfitting also required multiple training iterations and validation checks.

5.3 Implementation and Integration

Integrating different components of the system into a unified pipeline presented several challenges. These included:

- Connecting detection outputs with grid-based density estimation
- Mapping bounding boxes accurately to grid cells
- Ensuring consistent processing across video frames

Debugging these components required iterative testing and refinement.

5.4 Real-Time Processing Constraints

Achieving real-time performance was a key challenge. Processing video frames while maintaining acceptable frame rates required optimization of the model and efficient handling of computations.

Trade-offs had to be made between detection accuracy and processing speed to ensure smooth system performance.

5.5 Hardware Challenges

The hardware implementation introduced additional challenges, particularly during the drone assembly process. These included:

- Complexity of wiring and component connections
- Ensuring correct motor configuration (CW/CCW)
- Limited experience with UAV hardware systems

Although a partial hardware prototype was achieved, full system integration with the AI model remains a challenge for future work.

Chapter 7: Testing & Evaluation

7.1 Test Items

Ru'ya is a UAV-based crowd monitoring system designed to detect individuals and estimate crowd density in real time.

The purpose of this software testing section is to verify that all system requirements specified in the Software Requirements Specification (SRS) are functionally sound. In addition, the system design and implementation described in the Software Design Specification (SDS) will be validated to ensure correct operation.

7.2 Features to be Tested

All features to be tested:

- AI-based crowd detection (YOLOv8s).
- Crowd counting and density estimation.
- Risk classification (Low / Medium / High).
- UAV flight and stability.
- System pipeline execution.
- Real-time processing on Jetson Nano.
- Integration between UAV hardware and AI system.

7.3 Approach

All test types that will be implemented on the Ru'ya system are listed below:

- Data and dataset integrity testing
- Component testing
- Integration testing
- Interface testing
- Validation and verification testing
- Performance testing
- Constraints
- Acceptance testing

7.3.1 Data and Dataset Integrity Testing

A test is implemented on the dataset used in the system to ensure its compatibility and reliability.

Techniques:

- Apply preprocessing (resizing, normalization)
- Verify annotation format (YOLO bounding boxes)
- Ensure dataset consistency and correctness
- Validate compatibility with the model

Completion Criteria:

- The dataset must be free of inconsistent or corrupted data.

Special Considerations:

- The entire dataset is used during testing to ensure comprehensive validation and better generalization of the model.

7.3.1.1 Test Cases

The test cases are summarized in Table 53.

Table 54. Data and Dataset Integrity Test Cases

Test ID	Test Description	Expected Result	Verified
1	Check annotation format	All labels follow correct YOLO bounding box format	Yes
2	Check image size	Images are compatible with model input size	

7.3.2 Component Testing

Component testing is used to test each system module independently.

Components:

- YOLOv8 detection model
- Counting module
- Density estimation module
- Risk classification module

Techniques:

- Verify bounding box accuracy against ground truth.

Completion Criteria:

- Each component must function correctly without errors.

Special Considerations:

- Each component is tested independently before integration

7.3.2.1 Test Cases (YOLOv8 Detection Model)

The test case for the YOLOv8 detection model is presented in Table 54.

Table 55 Component Test Case for YOLOv8 Detection Model

Test ID	Yolov8s
Prerequisite	• VisDrone2019-DET dataset selected and preprocessed (person class only). • Dataset split into training, validation, and testing sets. • YOLOv8s model configured with pretrained weights. • Input images resized automatically by YOLOv8 framework.
Test Procedure	1. Input aerial images from VisDrone dataset. 2. Run YOLOv8s detection model. 3. Detect individuals using bounding boxes. 4. Evaluate results using Precision, Recall, and mAP.
Expected Result	Correct detection of persons in aerial images. Bounding boxes localized around individuals.
Actual Result	Precision: 72.76% Recall: 61.68% mAP@50: 68.94% mAP@50-95: 33.52%
Verified	Yes

7.3.2.2 Test Cases (Density & Risk Module)

The test case for the density estimation and risk classification module is presented in Table 55.

Table 56 Component Test Case for Density and Risk Classification Module

Test ID	Density & Risk Module
Prerequisite	- Detection results available from YOLOv8s model. - Frame divided into a 6×6 grid (36 cells). - Bounding boxes mapped to corresponding grid cells.
Test Procedure	- Divide input frame into 6×6 grid cells - Count number of detected persons $N_{i,j}$ in each cell - Compute density using $D = \frac{N}{A}$ - Classify risk level based on thresholds - Generate color-coded risk map
Expected Result	Correct counting of persons per grid cell. Accurate density estimation for each region. Correct risk classification (Low / Medium / High). Proper visualization using color-coded grid.
Actual Result	System successfully counted individuals per cell. Density computed correctly using grid-based approach. Risk levels assigned as: - Low: $N < 2$ - Medium: $2 \leq N < 4$ - High: $N \geq 4$ - Output visualized with color-coded grid (Green/Yellow/Red)
Verified	Yes

7.3.3 Integration Testing

Integration testing is performed after component testing to ensure proper interaction between all system modules. This step verifies that the complete pipeline functions correctly when all components are combined.

The integration process focuses on validating the flow of data from one stage to another, starting from person detection through risk classification and visualization.

Hardware integration testing is currently in progress and will be further evaluated through UAV flight testing, live video streaming, and in real-world conditions.

Techniques:

- Integrate modules sequentially (detection → counting → density → risk classification)
- Verify correct data transfer between modules
- Test the full pipeline using sample images and video frames
- Validate consistency of intermediate outputs

Completion Criteria:

- The system operates correctly as a complete pipeline without errors
- Outputs from each stage are accurate and consistent

Special Considerations:

- If errors occur, individual components are retested and debugged before reintegration

7.3.4 Interface Testing

This test verifies communication between system components.

Basic data flow between software components has been successfully tested, while full hardware communication (camera, Jetson Nano, Pixhawk) is currently in progress and will be validated in later stages.

Techniques:

- Verify data flow from the input (camera frames) to the processing unit (Jetson Nano) without latency or data loss.
- Ensure detection outputs are correctly passed to the counting, density estimation, and risk classification modules.
- Check that results are consistently processed and visualized across the pipeline.

Completion Criteria:

- All components communicate correctly without delays or data loss

7.3.5 Validation and Verification (V&V) Testing

Validation and verification testing ensure that the system is both correctly implemented and aligned with the project objectives.

Verification confirms that each component is implemented according to the system design, while validation ensures that the system meets its intended purpose of real-time crowd monitoring.

Techniques:

- Verification: compare implementation with system design and specifications.

- Validation: evaluate system outputs against expected behavior.

Completion Criteria:

- The system produces correct and reliable outputs.

7.3.6 Performance Testing

Performance testing evaluates the efficiency of the system and its ability to operate in real-time conditions, which is critical for UAV-based applications.

This includes measuring the speed and responsiveness of the detection model and the overall pipeline.

Techniques:

- Measure inference time per frame
- Calculate frames per second (FPS)
- Evaluate performance on target hardware (Jetson Nano)

Completion Criteria:

- The system achieves real-time performance (≥ 10 FPS)
- Processing is stable without significant delays

7.3.7 Constraints

The system is subject to the following constraints:

- Limited prior experience with UAV hardware integration.

7.3.8 Acceptance Testing

Acceptance testing ensures that the system meets all required specifications.

Completion Criteria:

- Accurate crowd detection
- Correct density estimation and risk classification
- Real-time performance achieved
- Successful integration with UAV hardware

7.4 Testing Environment

The testing environment includes:

- **Hardware:** Jetson Nano, F450 quadcopter drone kit, Pixhawk flight controller, Windows 10 or Mac OS.
- **Software:** Python, YOLOv8 (Ultralytics), PyTorch.

7.5 Testing Hardware

This section evaluates the functionality and reliability of the UAV hardware components integrated within the RU'YA system. The purpose of these tests is to ensure proper electrical connections, stable UAV operation, reliable communication between components, and readiness for real-time deployment.

7.5.1 ESC and Motor Testing

The Electronic Speed Controllers (ESCs) and brushless motors were tested to verify correct operation and ensure compatibility with the quadcopter configuration. Since stable UAV movement depends heavily on proper motor synchronization and rotation direction, several preliminary hardware tests were conducted during assembly.

The testing process included verifying the rotation direction of each motor according to the standard quadcopter configuration. Clockwise (CW) and counterclockwise (CCW) motor orientations were checked and adjusted to ensure stable flight behavior and proper thrust balancing. Additionally, ESC connections were examined to confirm correct signal and power wiring between the motors, power distribution board, and flight controller.

Basic motor response tests were performed after ESC calibration to verify that all motors responded consistently to throttle input without abnormal delay or instability.



Figure 89. Motor direction diagram (CW/CCW layout)

7.5.2 Power Distribution Testing

Power distribution testing was conducted to ensure stable electrical power delivery across all UAV hardware components. The power distribution board (PDB) was connected to the ESCs and battery system using XT60 connectors and soldered power lines.

Voltage stability and connection integrity were inspected to minimize the risk of power interruption during UAV operation. The testing process also verified that all ESCs received sufficient and consistent power from the battery through the distribution board.

Special attention was given to soldering quality and connector stability to reduce potential electrical failures during real-time deployment.

Completion Criteria

- Stable voltage delivery across all ESC connections.
- Proper XT60 connector functionality.
- No loose wiring or unstable electrical connections detected.

7.5.3 Pixhawk Integration Testing

Pixhawk integration testing was performed to validate communication between the flight controller and the UAV hardware components. This testing phase focused on ensuring that the Pixhawk controller correctly interfaces with the ESCs, motors, and power modules.

Preliminary calibration procedures were conducted, including accelerometer calibration and ESC signal verification. Communication between the Pixhawk flight controller and Mission Planner software was also tested to confirm successful parameter configuration and hardware recognition.

The testing process aimed to ensure that the UAV platform could support stable flight operations and future integration with onboard AI processing systems.

Completion Criteria

- Successful communication between Pixhawk and Mission Planner.
- Correct recognition of connected hardware components.
- Stable calibration results without communication errors.

7.5.4 Camera and Video Transmission Testing

Camera and video transmission testing were conducted to verify the ability of the RU'YA system to capture and process live aerial video streams in real time. The onboard camera

system was connected to the processing unit to evaluate frame acquisition stability and video transmission reliability.

The tests focused on validating live video feed continuity, frame quality, and communication latency between the camera and the processing pipeline. Preliminary streaming tests confirmed that aerial frames could be successfully transmitted for crowd detection and density estimation.

Latency observations were also considered during testing to ensure that the system remains suitable for near real-time monitoring applications.

Completion Criteria

- Stable live video feed obtained successfully.
- Frames transmitted without significant interruption.
- Acceptable latency achieved for real-time processing.

7.6 Altitude & Weather Simulation Testing

1. Simulation Overview

A simulation environment was developed to evaluate the feasibility and robustness of the RU'YA system prior to real-world deployment. The simulation integrates synthetic aerial crowd generation with real YOLO-based person detection to assess system performance under dynamic conditions.

The objective is to analyze how both **drone altitude** and **environmental conditions** affect the accuracy of crowd detection from a UAV perspective.

2. Simulation Design

2.1 Synthetic Crowd Generation

A synthetic aerial scene was generated to simulate a top-view drone perspective of a crowd. Each individual was represented using simplified human-like shapes to approximate real-world visual characteristics. The environment includes a textured ground and randomized spatial distribution of individuals.



Figure 90. Example of the generated synthetic aerial crowd used in the simulation

2.2 Crowd Movement Modeling

To simulate realistic crowd behavior, each individual was assigned a random velocity vector, allowing continuous movement across the scene. Boundary conditions were applied to ensure individuals remain within the environment, creating a dynamic and time-varying crowd scenario.

2.3 YOLO Detection Setup

The YOLOv8n model was used for real-time person detection. Detection was performed on each frame of the simulation using a confidence threshold of 0.15, focusing exclusively on the “person” class. This allows evaluation of detection performance under varying visual conditions.

3. Altitude-Based Analysis

3.1 Objective

The objective of this experiment is to evaluate how drone altitude affects crowd detection performance.

3.2 Conceptual Illustration

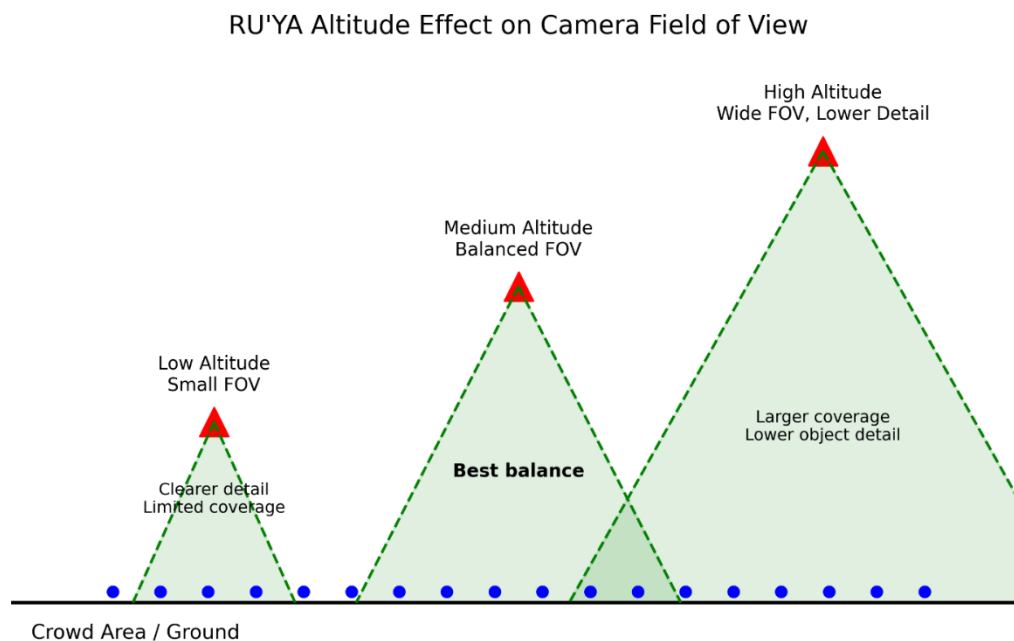


Figure 91. Conceptual illustration of drone altitude effect on camera field of view. Low altitude provides clearer object details but limited coverage, high altitude increases coverage but reduces object detail, while medium altitude provides the best

3.3 Simulation Setup

Three altitude levels were simulated:

- Low altitude
- Medium altitude
- High altitude

Increasing altitude increases the camera field of view but reduces the apparent size and clarity of individuals.

3.4 Results

Table 57. Comparison of Crowd Detection Accuracy Across Different UAV Altitudes

	Altitude	Ground Truth People	Average YOLO Detected People	Average Detection Rate (%)
0	Low	120	50.42	42.02
1	Medium	120	93.02	77.51
2	High	120	23.69	19.74

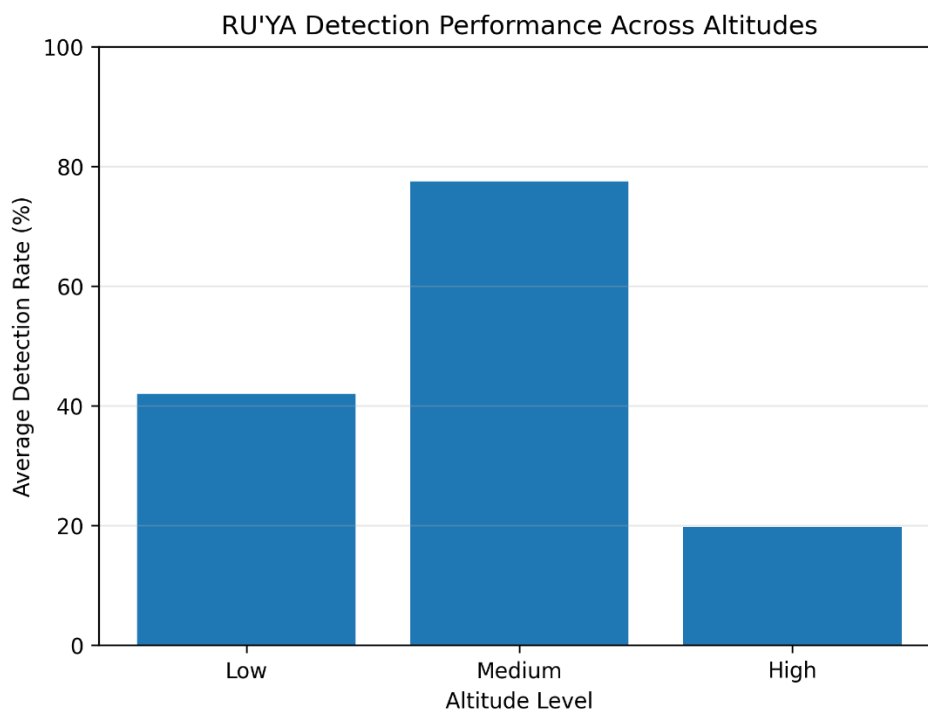


Figure 92. Detection Performance of the RU'YA System Across Different UAV Flight Altitudes

3.5 Visual Results

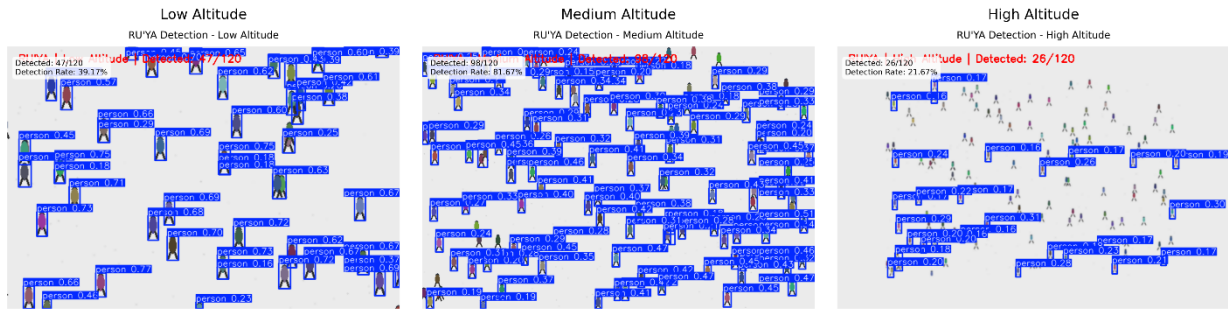


Figure 93. Detection results at different altitudes. Medium altitude provides the best balance between coverage and detection accuracy.

3.6 Discussion

The results show that low altitude provides clearer visibility of individuals, but limits the field of view. High altitude increases coverage but reduces detection accuracy due to smaller object size. Medium altitude achieves the best balance between visibility and coverage, resulting in the highest detection performance.

4. Weather Simulation

4.1 Weather Conditions

The simulation includes the following environmental conditions:

- Clear (baseline)
- Rainy
- Foggy
- Dusty
- Low Light

4.2 Weather Modeling Approach

Weather effects were simulated by modifying image properties to replicate real-world visual degradation. Fog was simulated using brightness blending and blur, rain using streak patterns and slight darkening, dust using color haze and reduced contrast, and low light by reducing brightness and visibility. These transformations allow analysis of detection robustness under adverse conditions.

5. Evaluation Metrics

5.1 Detection Rate

The detection rate is defined as:

Equation 2. detection rate

$$\text{Detection Rate} = \frac{\text{Detected People}}{\text{Total People}} \times 100$$

5.2 Average Detection Rate

The average detection rate was computed across all frames to evaluate overall system performance under each weather condition.

6. Experimental Results

6.1 Quantitative Results

Table 58. Weather-Based Evaluation of the RU'YA Crowd Detection System

	Weather Condition	Ground Truth People	Average YOLO Detected People	Average Detection Rate (%)
0	Clear	120	93.02	77.51
1	Rainy	120	91.16	75.97
2	Foggy	120	86.96	72.47
3	Dusty	120	80.65	67.21
4	Low Light	120	79.05	65.88

Table 57 shows the average number of detected individuals and detection rates across different weather conditions.

6.2 Visual Results

Detection performance under different weather conditions. The results show that YOLO achieves the highest accuracy under clear conditions, while performance degrades under foggy, dusty, and low-light environments due to reduced visibility and contrast.

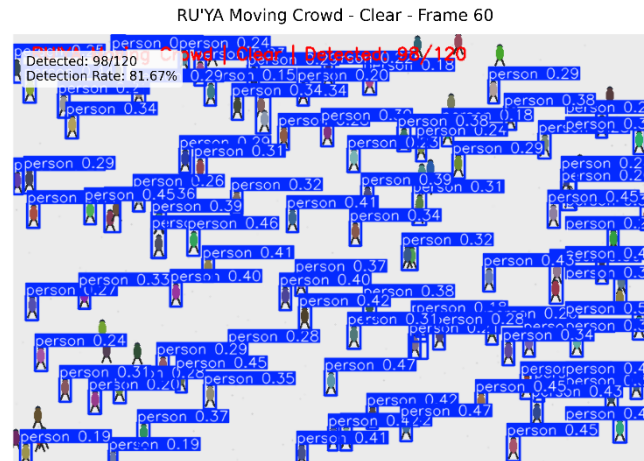


Figure 94. Crowd Detection Output Under Clear Weather Conditions Using the RU'YA System

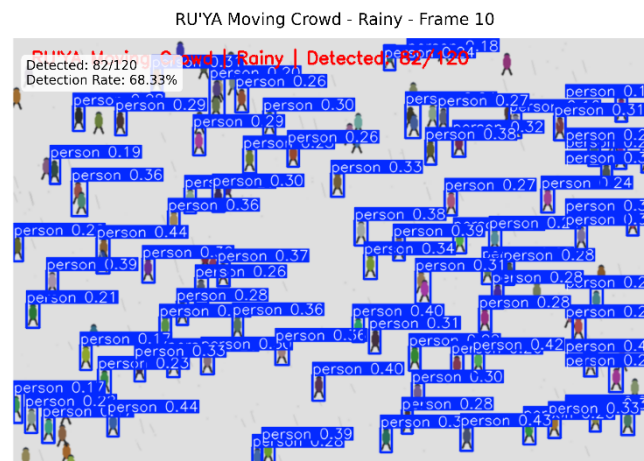


Figure 95. Crowd Detection Output Under Rainy Weather Conditions Using the RU'YA System

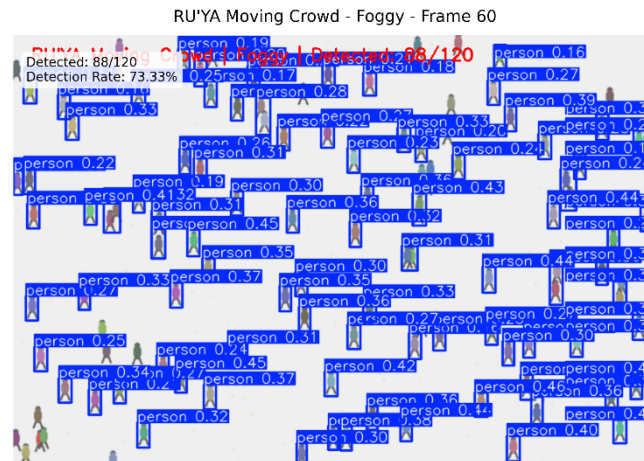


Figure 96. Crowd Detection Output Under Foggy Weather Conditions Using the RU'YA System

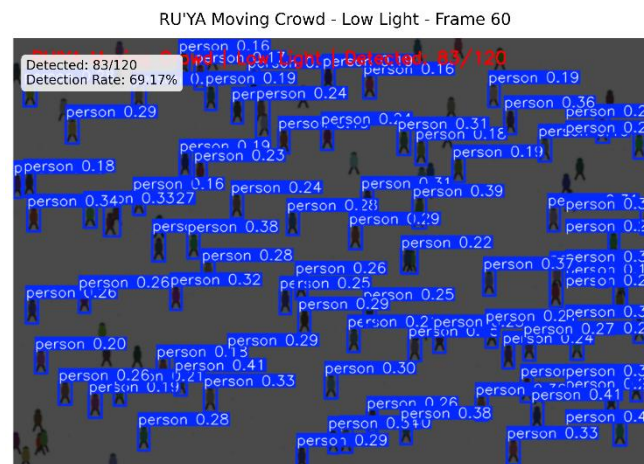


Figure 97. Crowd Detection Output Under low light Weather Conditions Using the RU'YA System

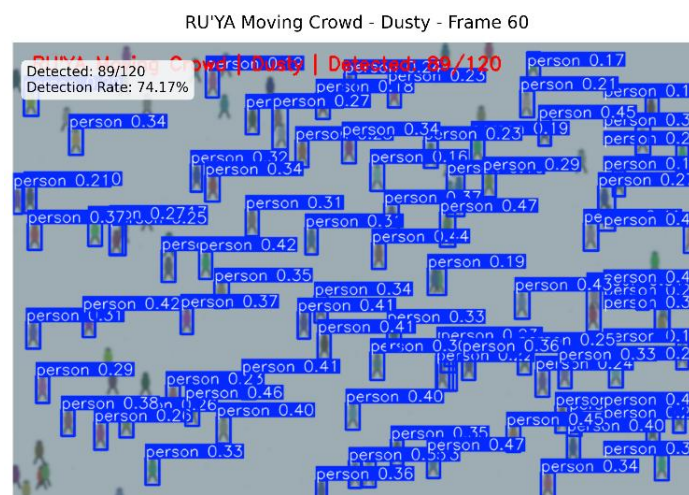


Figure 98. Crowd Detection Output Under Dusty Weather Conditions Using the RU'YA System

4.3 Discussion

The results indicate that detection performance decreases as environmental conditions become more challenging. Clear conditions achieve the highest accuracy, while fog, dust, and low-light conditions reduce visibility and negatively impact detection performance.

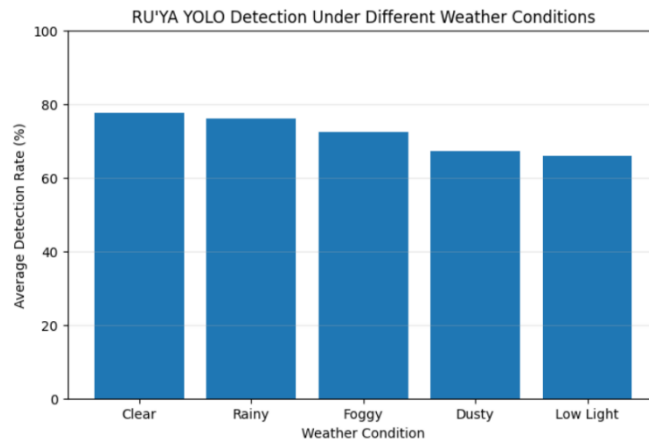


Figure 99. Weather-Based Evaluation of YOLOv8s Crowd Detection Performance

6. Conclusion of Simulation

The simulation confirms that the RU'YA system is capable of performing real-time crowd detection under a variety of operational conditions. Although detection accuracy decreases under higher altitudes and adverse weather conditions, the system maintains acceptable performance across all tested scenarios.

These results demonstrate the feasibility of the proposed approach and highlight the importance of optimizing both flight parameters and environmental considerations for real-world deployment.

8. Conclusion

This report presents the progress made in developing the Ru'ya system. At this stage, the main components of the system have been successfully implemented. The results show that the system is capable of performing crowd detection and analysis in a practical and efficient manner suitable for real-time applications.

The use of a lightweight model has proven to be a good choice, as it balances performance with computational limitations. During the implementation process, several challenges were encountered, especially in handling aerial images, optimizing system performance, and integrating the hardware components with the software pipeline.

Overall, the project is progressing as planned, with a working system pipeline already in place. Future work will focus on improving detection accuracy, completing full hardware integration, and testing the system under real-world conditions.

Resources

- [1] Saudi Vision 2030, "Saudi Vision 2030," *Vision2030.gov.sa*, 2024. <https://www.vision2030.gov.sa/en>
- [2] "Home | World Cup 2034 Hosting Higher Authority," *Wca34.gov.sa*, 2025. <https://www.wca34.gov.sa/en#goals> (accessed Sep. 17, 2025).
- [3] "General Authority for Statistics," *General Authority for Statistics*. <https://www.stats.gov.sa/en/>
- [4] "Ludlow Engineers and Associates," *Ludlowengineers.com*, Mar. 13, 2025. <https://ludlowengineers.com/evaluating-drones-vs-traditional-techniques-in-land-surveying/> (accessed Sep. 21, 2025).
- [5] C. Feliciani, A. Corbetta, M. Haghani, and K. Nishinari, "Trends in crowd accidents based on an analysis of press reports," *Safety Science*, vol. 164, p. 106174, Apr. 2023, doi: 10.1016/j.ssci.2023.106174.
- [6] K. Al-Dosari, Z. Hunaiti, and W. Balachandran, "Systematic Review on Civilian Drones in Safety and Security Applications," *Drones*, vol. 7, no. 3, p. 210, Mar. 2023, doi: 10.3390/drones7030210.
- [7] D. B. Resnik and K. C. Elliott, "Using Drones to Study Human Beings: Ethical and Regulatory Issues," *Science and Engineering Ethics*, vol. 25, no. 3, pp. 707–718, Feb. 2018, doi: <https://doi.org/10.1007/s11948-018-0032-6>.
- [8] H. Shakhathreh *et al.*, "Unmanned Aerial Vehicles (UAVs): A Survey on Civil Applications and Key Research Challenges," *IEEE Access*, vol. 7, pp. 48572–48634, 2019, doi: <https://doi.org/10.1109/access.2019.2909530>.
- [9] Y. Qin, M. A. Kishk, and M.-S. Alouini, "Performance Evaluation of UAV-Enabled Cellular Networks With Battery-Limited Drones," *IEEE Communications Letters*, vol. 24, no. 12, pp. 2664–2668, Dec. 2020, doi: <https://doi.org/10.1109/lcomm.2020.3013286>.
- [10] J. P. Duffy *et al.*, "Location, location, location: considerations when using lightweight drones in challenging environments," *Remote Sensing in Ecology and Conservation*, vol. 4, no. 1, pp. 7–19, Aug. 2017, doi: 10.1002/rse2.58.
- [11] Y. Zhang, A. Carballo, H. Yang, and K. Takeda, "Perception and sensing for autonomous vehicles under adverse weather conditions: A survey," *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 196, pp. 146–177, Jan. 2023, doi: 10.1016/j.isprsjprs.2022.12.021.
- [12] C. Jiang *et al.*, "Object detection from UAV thermal infrared images and videos using YOLO models," *International Journal of Applied Earth Observation and Geoinformation*, vol. 112, p. 102912, Jul. 2022, doi: 10.1016/j.jag.2022.102912.
- [13] A. Kuzdeuov, D. Aubakirova, D. Koishigarina, and H. A. Varol, "TFW: Annotated thermal faces in the wild dataset," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 2084–2094, Jan. 2022, doi: 10.1109/tifs.2022.3177949.
- [14] "The General Authority of Civil Aviation (GACA) | الهيئة العامة للطيران المدني." <https://gaca.gov.sa/rules-and-regulations-category/aviation-safety-and-environmental-sustainability/gacar-safety-regulations/chapter-f-general-operating-rules-part-107---non-recreational-civil-small-unmanned-aircraft-systems>
- [15] D. Caballero-Martin, J. M. Lopez-Guede, J. Estevez, and M. Graña, "Artificial Intelligence Applied to Drone Control: A State of the Art," *Drones*, vol. 8, no. 7, p. 296, Jul. 2024, doi: 10.3390/drones8070296.
- [16] M. Tzelepi and A. Tefas, "Graph embedded convolutional neural networks in human crowd detection for drone flight safety," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 5, no. 2, pp. 191–204, Mar. 2019, doi: 10.1109/tetci.2019.2897815.

- [17] C. Herrera et al., “Drone Insights: Unveiling Beach Usage through AI-Powered People Counting,” *Drones*, vol. 8, no. 10, p. 579, Oct. 2024, doi: 10.3390/drones8100579.
- [18] B. Ptak, D. Pieczyński, M. Piechocki, and M. Kraft, “On-Board Crowd Counting and Density Estimation Using Low Altitude Unmanned Aerial Vehicles—Looking beyond Beating the Benchmark,” *Remote Sensing*, vol. 14, no. 10, p. 2288, May 2022, doi:10.3390/rs14102288.
- [19] G. Castellano, C. Castiello, C. Mencar, and G. Vessio, “Crowd Detection in Aerial Images Using Spatial Graphs and Fully-Convolutional Neural Networks,” *IEEE Access*, vol. 8, pp. 64534–64544, 2020, doi: <https://doi.org/10.1109/access.2020.2984768>.
- [20] O. Elharrouss et al., “Drone-SCNet: Sealed Cascade Network for Crowd Counting on Drone Images,” *IEEE Transactions on Aerospace and Electronic Systems*, vol. 57, no. 6, pp. 3988–4001, Dec. 2021, doi: <https://doi.org/10.1109/taes.2021.3087821>.
- [21] L. Zhao, Z. Bao, Z. Xie, G. Huang, and Z. U. Rehman, “A point and density map hybrid network for crowd counting and localization based on unmanned aerial vehicles,” *Connection Science*, vol. 34, no. 1, pp. 2481–2499, Oct. 2022, doi: <https://doi.org/10.1080/09540091.2022.2130878>.
- [22] C. Papaioannidis, I. Mademlis, and I. Pitas, “Autonomous UAV safety by visual human crowd detection using Multi-Task deep neural networks,” *IEEE*, May 2021, doi: 10.1109/icra48506.2021.956083
- [23] M. A. Khan, H. Menouar, and R. Hamila, “LCDnet: a lightweight crowd density estimation model for real-time video surveillance,” *Journal of Real-Time Image Processing*, vol. 20, no. 2, Mar. 2023, doi: <https://doi.org/10.1007/s11554-023-01286-8>.
- [24] A. N. Alhawsawi, S. D. Khan, and F. U. Rehman, “Enhanced YOLOv8-Based Model with Context Enrichment Module for Crowd Counting in Complex Drone Imagery,” *Remote Sensing*, vol. 16, no. 22, p. 4175, Nov. 2024, doi: <https://doi.org/10.3390/rs16224175>.
- [25] V. R. Yallamraju and S. Jana, “Dynamic Object Detection and Tracking System on Unmanned Aerial Vehicles for Surveillance Applications Using RegionViT-Based Adaptive Multi-Scale YOLOv8,” *Computational Intelligence*, vol. 41, no. 4, pp. –, 2025, doi: <https://doi.org/10.1111/coin.70101>.
- [26] M. A. Khan, Hamid Menouar, Ridha Hamila, and Adnan Abu-Dayya, “Crowd counting at the edge using weighted knowledge distillation,” *Scientific Reports*, vol. 15, no. 1, Apr. 2025, doi: <https://doi.org/10.1038/s41598-025-90750-5>.
- [27] Z. Pengyi, Z. Yunxin, and L. Xiaoqiong, “SlimYOLOV3: Narrower, Faster and Better for Real-Time UAV Applications,” *IEEE Conference Proceedings*, vol. 2019, pp. 37–45, Jan. 2019, [Online]. Available: https://jglobal.jst.go.jp/en/detail?JGLOBAL_ID=202002217598207347
- [28] H. Zhong, Y. Zhang, Z. Shi, Y. Zhang, and L. Zhao, “PS-YOLO: a lighter and faster network for UAV object detection,” *Remote Sensing*, vol. 17, no. 9, p. 1641, May 2025, doi: 10.3390/rs17091641.
- [29] D. Du et al., “The Unmanned Aerial Vehicle Benchmark: Object Detection and Tracking,” *arXiv.org*, Mar. 26, 2018. <https://arxiv.org/abs/1804.00518>
- [30] J. H. Chan, K. Liu, Y. Chen, A. S. M. S. Sagar, and Y.-G. Kim, “Reinforcement learning-based drone simulators: survey, practice, and challenge,” *Artificial Intelligence Review*, vol. 57, no. 10, Sep. 2024, doi: 10.1007/s10462-024-10933-w.
- [31] C. A. Dimmig et al., “Survey of Simulators for Aerial Robots: An Overview and In-Depth Systematic Comparisons,” *IEEE Robotics & Automation Magazine*, pp. 2–15, Jan. 2024, doi: 10.1109/mra.2024.3433171.

- [32] S. Shah, D. Dey, C. Lovett, and A. Kapoor, "AirSim: High-Fidelity Visual and Physical Simulation for Autonomous Vehicles," arXiv preprint arXiv:1705.05065, 2017. [Online]. Available: <https://arxiv.org/abs/1705.05065>
- [33] J. Panerati, H. Zheng, S. Zhou, J. Xu, A. Prorok, and A. P. Schoellig, "Learning to Fly—a Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-agent Quadcopter Control," 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Sep. 2021, doi: 10.1109/iros51168.2021.9635857.
- [34] The MathWorks, Inc. "UAV Toolbox — Design, simulate, and deploy UAV applications," MATLAB & Simulink, accessed: Sep. 26, 2025. [Online]. Available: <https://www.mathworks.com/help/uav/index.html>
- [35] M. H. Rahman, M. A. S. Sejan, M. A. Aziz, R. Tabassum, J.-I. Baik, and H.-K. Song, "A Comprehensive survey of unmanned aerial vehicles detection and classification using Machine learning approach: challenges, solutions, and future directions," *Remote Sensing*, vol. 16, no. 5, p. 879, Mar. 2024, doi: <https://doi.org/10.3390/rs16050879>.
- [36] T. Ahmad et al., "Future UAV/Drone systems for intelligent active surveillance and monitoring," *ACM Computing Surveys*, Aug. 2025, doi: <https://doi.org/10.1145/3763089>.
- [37] H. Hildmann and E. Kovacs, "Review: Using Unmanned Aerial Vehicles (UAVs) as Mobile Sensing Platforms (MSPs) for disaster response, civil security and public safety," *Drones*, vol. 3, no. 3, p. 59, Jul. 2019, doi: 10.3390/drones3030059
- [38] Y. Shao, W. Li, H. Chu, Z. Chang, X. Zhang, and H. Zhan, "A Multitask Cascading CNN with MultiScale Infrared Optical Flow Feature Fusion-Based Abnormal Crowd Behavior Monitoring UAV," *Sensors*, vol. 20, no. 19, p. 5550, Sep. 2020, doi: <https://doi.org/10.3390/s20195550>
- [39] X. Qiu, Y. Chen, W. Cai, M. Niu, and J. Li, "LD-YOLOV10: A lightweight target detection algorithm for drone scenarios based on YOLOV10," *Electronics*, vol. 13, no. 16, p. 3269, Aug. 2024, doi: <https://doi.org/10.3390/electronics13163269>
- [40] IEEE, "IEEE Standard for Software Project Management Plans," in IEEE Std 1058-1-2021, pp.1-108, 2021.
- [41] IEEE, "IEEE Recommended Practice for Software Requirements Specifications," in IEEE Std 830-1998, pp.1-40, 1998.
- [42] R. Pressman and B. Maxim, *Software Engineering: A Practitioner's Approach*. New York: McGraw-Hill Education, 2014.
- [43] G. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*. John Wiley & Sons, 2011.
- [44] W. Royce, "Managing the development of large software systems," in *Proceedings of the IEEE WESCON*, vol. 26, no. 8, pp. 1-9, 1970.
- [45] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 6th ed. Newtown Square, PA: Project Management Institute, 2017.
- [46] IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.
- [47] IEEE Std 1016-2009: *IEEE Standard for Information Technology—Systems Design—Software Design Descriptions*.
- [48] in Computer, "CRCV | Center for Research in Computer Vision at the University of Central Florida," *Ucf.edu*, 2014. <https://www.crcv.ucf.edu/data/ucf-qnr/>

[49] B. Ptak, D. Pieczyński, M. Piechocki, and M. Kraft, “On-Board Crowd Counting and Density Estimation Using Low Altitude Unmanned Aerial Vehicles—Looking beyond Beating the Benchmark,” *Remote Sensing*, vol. 14, no. 10, p. 2288, May 2022, doi:10.3390/rs14102288.

[50] “Face Blur Techniques for Privacy in Machine Learning,” *viso.ai*, 2023. <https://viso.ai/deep-learning/face-blur-for-privacy-aware-deep-learning/> (accessed Nov. 08, 2025).

[51] P. Siva, Garige Bhavani, G. S. Krishna, Gadde Hemanth, and S. S. V, “Smart Surveillance Systems Using YOLOv8: A Scalable Approach for Crowd and Threat Detection,” vol. 14, no. 1, pp. 52–62, Apr. 2025, Available:
https://www.researchgate.net/publication/390920968_Smart_Surveillance_Systems_Using_YOLOv8_A_Scalable_Approach_for_Crowd_and_Threat_Detection

[52] K. Pandya, “VisDrone Dataset,” Kaggle. [Online]. Available:
<https://www.kaggle.com/datasets/kushagrapandya/visdrone-dataset>

Appendix 1: Response to evaluators' comments

This section includes the comments we received from the evaluators and our responses.

Term 1 comments

Table 59: Evaluators comments term1

Comment 1	By
Avoid using sentences like “The table above summarizes”. The best practice is to cite the table number.	Dr. Wadha Almattar
Response	
Tables are now explicitly cited using their corresponding numbers.	
Comment 2	By
The terms “heavyweight” and “lightweight” AI models are relative and context-dependent. This weakens the reader’s ability to understand the rationale behind your system design choices. Without clear definitions, it’s unclear whether you are referring to model size (e.g., number of parameters), computational complexity (e.g., FLOPs), memory footprint, latency, or deployment feasibility on embedded systems.	Dr. Wadha Almattar
Response	
A concise clarification defining heavyweight and lightweight AI models based on computational complexity is added at chapter 2 in theme 2 “AI models”. A thorough comparison was added and discussed explicitly that describes exactly what is “heavyweight” and “lightweight”.	
Comment 3	By
What is the classical method to operate UAV drones that is not considered an AI solution? Emphasize this in the document.	Dr. Wadha Almattar
Response	
A brief paragraph has been added to section 1.2 to describe classical non-AI UAV operation methods.	
Comment 4	By
Collect and analyze datasets: Mention candidate datasets, and describe their characteristics.	Dr. Wadha Almattar
Response	

Already addressed in Chapter 2 Section 2.2, where candidate datasets and their characteristics are clearly discussed in detail.	
Comment 5	By
Develop a Lightweight AI Model: Specify Baseline and Target Model: Indicate which architecture you plan to start from (e.g., YOLOv5n, MobileNetV3, EfficientDet-D0). Clarify what compression techniques methods will be used, e.g., quantization, pruning, knowledge distillation.	Dr. Wadha Almattar
Response	
The report specifies the baseline and target AI model architectures in Chapter 6 and explicitly states the model compression techniques to be used in Chapter 2 Section 2.2.	
Comment 6	By
Test the Model in Simulation: Mention Simulation Tools: Name specific simulators like AirSim, Gazebo, or RotorS, and how you will feed visual input into the AI model. Simulated Conditions: Specify if you will test under different altitudes, weather, and crowd density scenarios.	Dr. Wadha Almattar
Response	
The simulation input pipeline and testing conditions are now added in Chapter 2 Section 2.3.	
Comment 7	By
Deploy and Test in a Controlled Environment: Test Protocol Design: Describe how many flight trials will be conducted, and what metrics will be recorded. Environmental Controls: Note how you'll ensure variability in crowd formation or lighting to mimic real-world complexity.	Dr. Wadha Almattar
Response	
Addressed in section 1.5.	
Comment 8	By
Reproducibility Plan: Describe how others can replicate your work using the GitHub repository. Include model weights, training scripts, configuration files, and documentation.	Dr. Wadha Almattar
Response	
A full reproducibility setup will be implemented in the second semester, with the GitHub repository providing final model weights, training scripts, configuration files, and documentation. Current code is supplementary and does not represent final results.	
Comment 9	By

Documentation Standards: Ensure that code comments, README files, and version control practices (e.g., Git branching strategy) follow industry conventions.	Dr. Wadha Almattar
Response	
Documentation and version control standards will be finalized in the second semester. The current code is supplementary not the final project implementation.	

Term 2 comments

Table 60: Evaluators comments term2

Comment 1	By
Figure 2: Please revise the system pipeline using a clearer and more standard schematic representation commonly used to illustrate system workflows. The current layout is difficult to follow and may confuse readers.	Dr. Wadha Almattar
Response	
Figure 58 has been updated and restructured for improved clarity and readability.	
Comment 2	By
AI Pipeline Figure: It is recommended to include a dedicated figure illustrating the overall AI pipeline. This is essential in AI-based studies to clearly present the workflow and the sequence of processes involved.	Dr. Wadha Almattar
Response	
The AI pipeline has been incorporated into Figure 58, illustrating the complete workflow from dataset preparation and model training to deployment and crowd monitoring.	
Comment 3	By
Section 4.2.2 – Data Preprocessing: Please add a figure that visually demonstrates the preprocessing steps applied to the dataset. Where applicable, include sample images before and after preprocessing. This will help readers better understand how these techniques improve image quality and support model performance during training	Dr. Wadha Almattar
Response	
Figure 59 has been added to visually demonstrate the class filtering step applied during dataset preprocessing.	
Comment 4	By
Annotation Conversion and Class Filtering:	Dr. Wadha Almattar

Please provide a more detailed explanation of the original annotation format and how it differs from the required YOLO format. Clarify the conversion process and the rationale behind class filtering.	
Response	
The section has been expanded to clarify the original annotation format, conversion process, and class filtering procedure.	
Comment 5	By
Training Configuration: It is advisable to present the training configuration details in a structured table format for better readability and clarity.	Dr. Wadha Almattar
Response	
A training configuration table [52] has been added.	
Comment 6	By
Figure 4: The figure caption indicates that it presents training and validation progress; however, the legend in one of the subfigures labels the red curve as “smooth” and the blue curve as “result,” which may cause confusion. The figure is also overly large and lacks clarity. It is recommended to break it down into smaller, well-organized subfigures and present the results in a more systematic and scientifically clear manner.	Dr. Wadha Almattar
Response	
All Figures [61-65] has been reorganized into smaller, well-structured subfigures with improved captions and presentation.	
Comment 7	By
Include a brief qualitative analysis of failure cases, with a few example images showing where the system misses detections or makes incorrect predictions. This would help illustrate the model’s limitations.	Dr. Sarah alyami
Response	
The report was revised by adding a new subsection titled “4.3.5 Qualitative Failure Case Analysis”. This subsection discusses the main failure cases observed in the YOLOv8s detection pipeline, including missed detections of small or distant people, occlusion in dense crowds, and reduced detection reliability under low-light or poor-weather conditions. Example output images were also added to illustrate these cases, including dense-scene detections, altitude-related detection loss, and low-visibility detection results. The discussion explains how these limitations may affect grid-based crowd counting and risk classification, and it identifies possible improvements for future work.	

Appendix 2: Biweekly Report: 1st semester

ARTI 511 - Project Proposal
Term 1 – 2024/2025
Status Report 1 #
 Period from 1/09/2025 to 18/09/2025

By

#	Name	ID	Role
1	Nora Abdullah Aljomuh	2220004452	Leader
2	Mozoon Alkhalis	2220002873	Member
3	Ritaj Alhamli	2220002809	Member
4	Joud Alahmari	2220001872	Member
5	Sarah Alashgar	2220005589	Member

Supervised by: Dr.Rabab Alkhalifa

Co-supervised by: Ms.Mehwash Farooqi

✓ **Tasks planned to work on during the specified period:**

Week 2 tasks:

1. Prepare Excel sheet for papers collection

Include columns: Title, Authors, Year, Source, Link, Summary, Notes, and a column for marking inclusion/exclusion.

2. Define Inclusion and Exclusion Criteria

○ **Inclusion Criteria:**

- Relevant to our project scope such as drones hardware/sensors, AI, survey studies, or code implementations.
- Published in reputable journals/conferences
- Published within the last 5–7 years

- Written in English

○ **Exclusion Criteria:**

- Irrelevant topics
- Low credibility sources
- Very old publications (more than 7–10 years old unless foundational)
- Incomplete papers

3. Upload Excel sheet to the Documents section

Ensure all team members can access and edit it.

4. Collect 5 research papers (1 per member)

Each team member is responsible for selecting one paper according to the criteria above.

5. Fill out the columns for the 5 collected papers

Complete all required details in the sheet for these papers.

Week 3:

1. Reorganize collected papers into the 3 LR subsections

Divide all previously collected papers into these thematic sections:

A. Drones in Public Safety and Crowd Management

- Papers about drone usage in large events, demonstrations, or safety operations
- Comparisons with traditional methods (fixed cameras, ground patrols)
- Case studies and real-world applications

B. Artificial Intelligence in Drone Systems

- Papers on computer vision for detection/tracking/crowd density estimation
- Machine learning for crowd behavior prediction
- Deep or reinforcement learning for autonomous decision-making

C. Challenges and Ethical Considerations

- Papers discussing technical limitations (battery, flight time, accuracy issues)
- Privacy and ethical implications of surveillance
- Legal and regulatory challenges of using drones in public spaces

2. Prepare a survey summary (1 per member)

3. Maintain alignment with LR structure

✓ **Status of each task including the names of members worked on each task:**

Task	Assigned members	Status
Prepare Excel sheet for papers collection	All members	Completed

Define inclusion & exclusion criteria	All members	Completed
Reorganize collected papers	All members	Completed
Prepare survey summary (1/member)	All members	Completed
Start writing the Literature Review (LR)	All members (split by subsections)	Completed

✓ Difficulties:

- 1. New and Unfamiliar Topic Area:** one of the main difficulties is that this is a completely new chapter and research area for us.
- 2. Identifying Relevant and High-Quality Papers:** It was challenging to find papers that exactly match our scope, as many are either too general or focus on unrelated drone applications.
- 3. Coordinating the Literature Review Structure:** Since this is our first time writing a structured Literature Review, it was difficult to plan the flow, divide sections, and avoid overlapping content between members.
- 4. Time Management:** Balancing this research work with other coursework and responsibilities has been challenging, especially while building the foundational understanding first.

✓ Pending tasks:

- 1. Complete Writing All LR Subsections and merge all drafts into one document**
- 2. Internal Review & Editing:** Proofread the merged LR and Fix grammar, spelling, formatting
- 3. Prepare the References Section:** Create a properly formatted references list (IEEE style)
Finalize Figures, Tables, or Conceptual Diagram: by adding any supportive diagrams, tables, or summaries if needed and Label and reference them correctly
- 4. Supervisor Feedback**

✓ Next planned tasks:

- 1. Improve the Literature Review based on supervisor feedback**
- 2. Start using simulators to develop a prototype** by exploring and select a suitable drone simulation environment.
- 3. Begin working on Chapter 3: SPMP (Software Project Management Plan)**

✓ **Number of group meetings you had during the specified time:** Two



✓ **Attached document(s):**

1. Excel sheet for the selected papers
2. Literature review

✓ Additional notes:

----- For supervisor use only -----

Date:

Comments:

Term 1 – 2024/2025
Status Report 2 #
 Period from 21/09/2025 to 05/10/2025

By

#	Name	ID	Role
1	Nora Abdullah Aljomuh	2220004452	Leader
2	Mozoon Alkhalis	2220002873	Member
3	Ritaj Alhamli	2220002809	Member
4	Joud Alahmari	2220001872	Member
5	Sarah Alashgar	2220005589	Member

Supervised by: Dr.Rabab Alkhalifa

Co-supervised by: Ms.Mehwash Farooqi

✓ **Tasks planned to work on during the specified period:**

Week 4

- 1- Rewrite the literature review based on the feedback given.
- 2- Prepare and deliver a presentation for our instructor to show recent updates.

Week 5

1. Rewrite the literature review again based on the new feedback.
- 2- Restructure the literature review into 4 main themes:
 - **Theme 1:** AI Models – Heavyweight AI Models for UAVs
 - **Theme 2:** Comparative Analysis of Datasets for Crowd Monitoring
 - **Theme 3:** Simulation as a Testbed for UAV Crowd Detection Models
 - **Theme 4:** Deployment of Lightweight Models on UAV Hardware

✓ Status of each task including the names of members worked on each task:

Task	Assigned members	Status
Rewrite and restructure the themes	All members (split by subsections)	Completed
Prepare slides for the presentation	All members	Completed

✓ **Difficulties:**

- Restructuring the literature review around the four themes was challenging, as it required reorganizing and rewriting large sections to ensure smooth flow.
- Time management was difficult, especially balancing rewriting, preparing presentations, and addressing detailed feedback.
- Trying to achieve exactly what the instructor requested in the feedback was challenging, since some points needed interpretation and careful alignment with the project scope.

✓ **Pending tasks:**

1. Review the entire document carefully to ensure all instructor feedback has been fully addressed.
2. Make final adjustments for clarity and consistency across all four themes.
3. Supervisor Feedback

✓ **Next planned tasks:**

1. Prepare updated slides for the next progress presentation.
2. Begin working on Chapter 3: SPMP (Software Project Management Plan)

✓ Number of group meetings you had during the specified time: **Two**

✓ Attached document(s):

1. Literature review
2. Bi-weekly report

Additional notes:



----- For supervisor use only -----

----- Date:

Comments:

Term 1 – 2024/2025

Status Report 3

Period from 16/10/2025 – 02/11/2025

By

#	Name	ID	Role
1	Nora Abdullah Aljomuh	2220004452	Leader
2	Mozoon Alkhalis	2220002873	Member
3	Ritaj Alhamli	2220002809	Member
4	Joud Alahmari	2220001872	Member
5	Sarah Alashgar	2220005589	Member

Supervised by: Dr.Rabab Alkhalifa

Co-supervised by: Ms.Mehwash Farooqi

✓ Tasks planned to work on during the specified period:

Week 8

1. Finalize and submit the Software Project Management Plan (SPMP).

Week 9-10

1. Review the SRS template and distribute sections among members.
2. Begin drafting the Software Requirements Specification (SRS) document.
3. Conduct group discussions.

✓ Status of each task including the names of members worked on each task:

Task	Assigned members	Status
Finalize and submit SPMP	All members	Completed
Review SRS template and divide sections	All members	Completed

Review and refine SRS content	All members	Completed
-------------------------------	-------------	-----------

✓ **Difficulties:**

- Aligning all requirements with the system scope took time due to detailed cross-checking between user needs and system constraints.
- Coordinating edits among team members on shared sections of the SRS document caused occasional version conflicts.

✓ **Pending tasks:**

1. 2. Review the complete SRS document before submission.
- Incorporate supervisor feedback if received before the deadline.

✓ **Next planned tasks:**

1. Submit the final version of the SRS document by Sunday, 2 November 2025.

✓ **Number of group meetings during the period:** Two

Attached documents:

1. SPMP
2. Bi-weekly report
3. SRS

✓ Additional notes:

----- For supervisor use only -----

----- Date:

Comments:

Term 1 – 2024/2025

Status Report 4#

Period from 2/11/2025 to 13/11/2025

Tasks planned to work on during the specified period:

Week 11:

- Divide the SDS sections.
- Each member worked on their assigned parts.

Week 12:

- o Check the progress of each member.
- o Finalized the SDS sections.

✓ Status of each task including the names of members worked on each task:

Task	Assigned members	Status
Divide the SDS sections. Each member worked on their assigned parts.	All members	Completed
Check the progress of each member. Finalized the SDS sections.	All members	Completed

Difficulties we faced:

- o Uncertainty on how to describe our AI-based system in an SDS, since it's mainly a deep learning project, not a traditional software system.
- o Minor delays due to section dependencies between team members.

How We Tackled These Difficulties:

- o We had meetings with our supervisor regarding the SDS structure and discussed appropriate approaches to represent Ru'ya project.
- o Shared our work drafts early between members to keep the work consistent.

Pending tasks:

- o Review the SDS document to ensure all instructor feedback has been fully addressed.
- o Receive our supervisor's final feedback after adjustments.

Next planned tasks:

- o Submit the SDS after team review.
- o Start working on the Final project design.

Number of group meetings you had during the specified time:

Two meetings held.

Attached document(s):

- o Software Design Specification (SDS)



ARTI 511

o Bi-weekly report

Additional notes:

----- For supervisor use only -----

----- Date: -----

Appendix 3: Paper Submission to conference

This appendix documents the successful submission of the research paper titled “*Ru’ya: A Lightweight AI Model for UAV-Based Crowd Detection and Monitoring*” to the **ICIGP 2026 Conference** under the **Computer Vision and Pattern Recognition** track. The submission was completed through the EasyChair system, and the paper was successfully saved and recorded with a submission ID. This confirms the formal participation of the project in an international academic conference and demonstrates the readiness of the research for peer review and dissemination.

12/16/25, 6:23 PM

ICIGP 2026 Submission 67


ICIGP 2026 (author)
[User guide](#) / [Log out](#)

New Submission

Submission 67

Help

Conference

News

EasyChair

ICIGP 2026 Submission 67

[Update information](#)
[Update authors](#)
[Update file](#)

The submission has been saved!

Submission 67	
Title	Ru'ya: A Lightweight AI Model for UAV-Based Crowd Detection and Monitoring
Paper	 (Dec 16, 15:23)
Track	Track 2. Computer Vision and Pattern Recognition
Author keywords	Artificial Intelligence Deep Learning Computer Vision UAVs Lightweight Models
Abstract	The Ru'ya Drone Project focuses on developing an intelligent lightweight UAV-based system capable of performing real-time crowd detection and density estimation using artificial intelligence (AI) and computer vision techniques. The project integrates drone technology with advanced deep learning models such as TensorFlow and PyTorch to enhance public safety, surveillance efficiency, and situational awareness in large gatherings. The system captures aerial footage through onboard cameras, preprocesses the data, and applies AI models to identify and analyze crowd behavior dynamically. A userfriendly interface allows real-time monitoring, while cloud integration ensures efficient data management and scalability. This project contributes to the growing field of AI-driven autonomous systems and aligns with Saudi Vision 2030, promoting smart technologies for safety and urban management. The results demonstrate the potential of combining UAV platforms with AI to achieve accurate, adaptable, and reliable crowd monitoring solutions for realworld applications
Submitted	Dec 16, 15:23
Last update	

Figure 100.ICIGP 2026 Conference

Ru'ya conference paper:

https://www.researchgate.net/publication/400800904_Ru'ya_A_Lightweight_AI_Model_for_UAV-Based_Crowd_Detection_and_Monitoring

Appendix 4: Patent Submission to Imam Abdulrahman Bin Faisal University

A patent titled “Mandatory Hardware-Based Compliance Enforcement Device for Unmanned Aerial Vehicles in Saudi Airspace” has been submitted through Imam Abdulrahman Bin Faisal University and is currently under review awaiting acceptance and potential filing

[Hardware-Based Compliance and Authorization Enforcement Device for Unmanned Aerial Vehicles \(submitted\) - Online LaTeX Editor Overleaf](#)

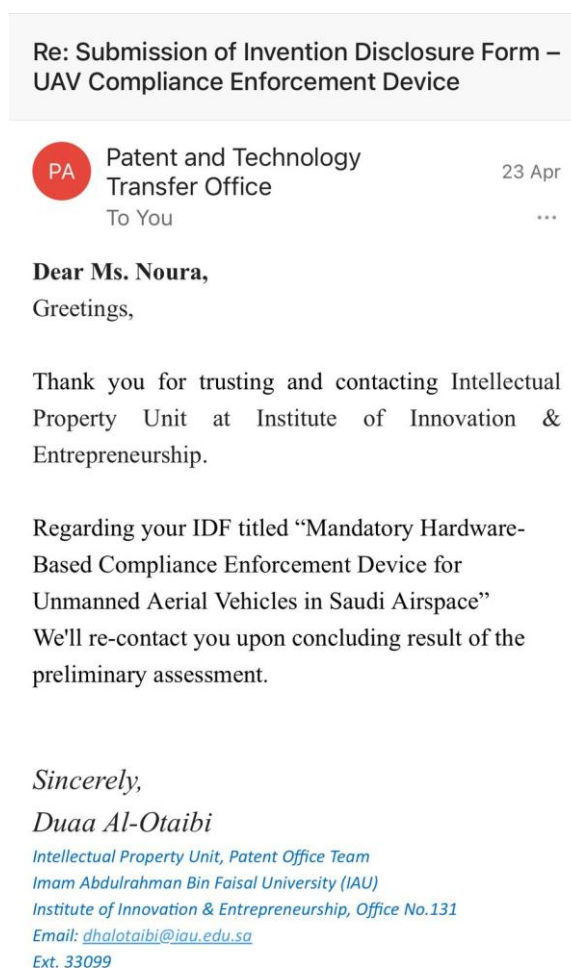


Figure 101. Patent Submission Acknowledgment Email for the UAV Compliance Enforcement Device

Appendix 5: Biweekly Report: 2nd semester

Kingdom of Saudi Arabia
Ministry of Education
Imam Abdulrahman Bin Faisal University
College of Computer Science & Information
Technology

جامعة الإمام عبد الرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY



ARTI 521 – Project Implementation

Term 2 – 2024/2025

Status Report 1

Period from 18/1/2026 to 25/1/2026

Ru'ya: Lightweight AI Models for UAV-Based Crowd Monitorin

- ✓ Group 1
- ✓ Members of the group:

#	Name	ID	Role
1	Nora Abdullah Aljomuh	2220004452	Leader
2	Ritaj Alhamli	2220002809	Member
3	Mozoon Alkhalis	2220002873	Member
4	Sarah Alashgar	2220005589	Member
5	Joud Alahmari	2220001872	Member

Supervised by: Dr.Rabab Alkhalifa

Co-supervised by: Ms.Mehwash Farooqi

✓ **The outcome of the meeting:**

During this period, our group held an initial meeting to mark the beginning of the implementation phase. Our discussion focused on reviewing the overall project direction and confirming group member roles. This meeting helped ensure that all members were prepared to move forward and had a shared understanding of the next steps.

✓ **Tasks planned to work on during the specified period:**

- Confirm roles and responsibilities among group members.
- Plan the transition from the previous phase to implementation.
- Identify priorities and expected milestones for the term.

✓ **Pending tasks:**

- Begin technical implementation.

✓ **Number of group meetings you had during the specified time:**

1 meeting

----- For supervisor use only -----

Date:

Comments:

Kingdom of Saudi Arabia
Ministry of Education
Imam Abdulrahman Bin Faisal University
College of Computer Science & Information
Technology



ARTI 521 – Project Implementation

Term 2 – 2024/2025

Status Report 2

Period from 1/2/2026 to 8/2/2026

Ru'ya: Lightweight AI Models for UAV-Based Crowd Monitorin

- ✓ Group 1
- ✓ Members of the group:

#	Name	ID	Role
1	Nora Abdullah Aljomuh	2220004452	Leader
2	Ritaj Alhamli	2220002809	Member
3	Mozoon Alkhalis	2220002873	Member
4	Sarah Alashgar	2220005589	Member
5	Joud Alahmari	2220001872	Member

Supervised by: Dr.Rabab Alkhalifa

Co-supervised by: Ms.Mehwash Farooqi

✓ **The outcome of the meeting**

During this period, our group discussed recent academic and administrative updates related to the project. The main focus was on the acceptance of our research paper in a conference and reviewing the status of the university's financial funding request. These developments were acknowledged as important milestones that support the continuation of the project.

✓ **Tasks planned to work on during the specified period:**

- Received confirmation of paper acceptance in an academic conference.
- Followed up on the university's approval process for the financial funding request.
- Reviewed conference requirements and next steps related to the accepted paper.

✓ **Pending tasks:**

- Final confirmation of financial funding approval.
- Begin technical implementation.

✓ **Number of group meetings you had during the specified time:**

1 meeting

----- For supervisor use only -----

Date:

Comments:

Kingdom of Saudi Arabia
Ministry of Education
Imam Abdulrahman Bin Faisal University
College of Computer Science & Information
Technology



ARTI 521 – Project Implementation

Term 2 – 2024/2025

Status Report 3

Period from 15/2/2026 to 22/2/2026

Ru'ya: Lightweight AI Models for UAV-Based Crowd Monitorin

- ✓ Group 1
- ✓ Members of the group:

#	Name	ID	Role
1	Nora Abdullah Aljomuh	2220004452	Leader
2	Ritaj Alhamli	2220002809	Member
3	Mozoon Alkhalis	2220002873	Member
4	Sarah Alashgar	2220005589	Member
5	Joud Alahmari	2220001872	Member

Supervised by: Dr.Rabab Alkhalifa

Co-supervised by: Ms.Mehwash Farooqi

✓ **The outcome of the meeting**

During this period, Our discussion focused on intellectual property considerations, academic dissemination, and the practical development of the project. We explored the process and requirements for patent registration in both Saudi Arabia and the United States. Also, we talked about start writing our conference paper and begin the hardware development phase.

✓ **Tasks planned to work on during the specified period:**

- Reviewed patent requirements in Saudi Arabia and the United States.
- Began writing and organizing the conference paper.
- Initiated early development of the project model.
- Began initial preparation for the hardware setup

✓ **Pending tasks:**

- Finalize the work related to the patent.
- Conduct early testing of the developed components.

✓ **Number of group meetings you had during the specified time:**

1 meeting

Kingdom of Saudi Arabia
Ministry of Education
Imam Abdulrahman Bin Faisal University
College of Computer Science & Information
Technology

جامعة الإمام عبد الرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY



ARTI 521 – Project Implementation

Term 2 – 2025/2026

Status Report #4

Period from 22/02/2026 to 29/03/2026

✓ **Group # TF10**

✓ **Members of the group:**

#	Name	ID	Role
1	Nora Abdullah Aljomuh	2220004452	Leader
2	Mozoon Alkhalis	2220002873	Member
3	Ritaj Alhamli	2210002809	Member
4	Joud Alahmari	2220001872	Member
5	Sarah Alashgar	2220005589	Member

✓ **The outcome of the meeting**

We met to work on constructing the RU'YA drone and made progress on the implementation mid-term report. We also completed and submitted the ICIGP paper and finalized the presentation, which we successfully delivered. Additionally, we organized our deadlines and continued developing the written sections of the project. To support the hardware setup, we reviewed multiple instructional videos on assembling the F450 drone kit and began the construction process.

✓ **Tasks planned to work on during the specified period:**

- Finalize the construction of the drone
- Integrate and implement our AI model on the drone system

✓ Pending tasks:

- Investigating the patent application process and guidelines provided by the Saudi Authority for Intellectual Property, including required documentation and submission steps.
- Testing and validating the drone system to ensure stable performance and accurate real-time detection under different environmental conditions.

✓ Number of group meetings you had during the specified time: 1

----- For supervisor use only -----

Date:

Comments:



ARTI 511 - Project Proposal

Term 1 – 2024/2025



Status Report # 5

Period from 12/04/2026 to 19/04/2026

- ✓ Group #: 1
- ✓ Supervisor's Name: Dr.Rabab Alkhalifa
- ✓ Members of the group:

#	Name	ID	Role
1	Nora Abdullah Aljomuh	2220004452	Leader
2	Mozoon Alkhalis	2220002873	Member
3	Ritaj Alhamli	2210002809	Member
4	Joud Alahmari	2220001872	Member
5	Sara Alashgar	2220005589	Member

✓ **Tasks planned to work on during the specified period:**

- Reviewed progress of the **testing phase** and simulation results.
- Discussed the impact of environmental conditions (fog, rain) on detection performance.
- Confirmed successful **construction and assembly of the drone hardware (F450 frame, motors, Pixhawk, and wiring)**.
- Agreed on moving forward to real-world testing and hardware integration.



✓ **Tasks planned to work on during the specified period:**

- Integrate the AI model with the drone hardware system.
- Conduct real-world testing using the drone camera feed.
- Improve detection accuracy under different environmental conditions.
- Optimize real-time processing performance.

✓ **Pending tasks:**

- Full deployment of the model on embedded hardware (e.g., Jetson / onboard processing).
- Extensive real-world testing and validation.
- Final performance evaluation and result documentation.

✓ **Number of group meetings you had during the specified time:**